

大模型技术之主流大模型原理

第 1 章 大模型介绍

1.1 定义

目前关于大模型（Large Models）并没有统一的定义，通常是指**参数规模巨大**、**训练数据庞大**、**能力强大**的深度神经网络模型。

1.2 特点

1.2.1 参数规模巨大

参数量通常在十亿级至千亿级之间，远高于传统的深度学习模型。

- GPT-3: 1750 亿
- Llama1-7B: 70 亿
- DeepSeek-V3-1226: 6710 亿

1.2.2 训练数据庞大

- GPT-1: 使用了筛选后的 BooksCorpus 数据集，后者包含超过 7000 本未出版书籍，近 10 亿 token。
- GPT-3: 使用了清洗后的 Common Crawl（开源爬虫项目，见[附录](#)）数据集、WebText2 网络文本数据集、Book1 和 Book2（基于互联网的书籍语料库）数据集和维基百科数据集，token 总量约 5000 亿。

1.2.3 能力强大

具备通用性强、跨任务泛化能力高的特点。

1.3 分类

1.3.1 大语言模型

大语言模型（Large Language Models, LLM）是目前名气最大的模型，专注于自然语言处理。

1.3.1.1 定义

大语言模型（LLM）是一种基于深度学习的大规模语言模型，通常采用 Transformer 架构，在**海量文本数据**上进行训练，能够**理解**、**生成和推理**自然语言。其核心特点包括**参数量巨大**、**通用性强**（可处理多种 NLP 任务）以及**涌现能力**（在规模达到一定程度后表现出小模型不具备的能力）。

1.3.1.2 核心特征

- 1) 大规模参数：通常具有数十亿到数千亿参数。
- 2) 基于海量文本训练
- 3) 通常基于 Transformer 架构

- 4) 通用性强，可处理多种 NLP 任务
- 5) 具备涌现能力

1.3.2 多模态大模型

如 DALL·E（文本生成图像）、Flamingo（图文交互）、通义万相。

1.3.3 科学计算大模型

如 AlphaFold（蛋白质结构预测）、气候预测模型。

1.4 模型的“开源”

1.4.1 OSI 的《开源定义》

开源倡议组织（Open Source Initiative, OSI）是一个成立于 1998 年的非营利组织，旨在推广和保护开源软件（Open Source Software, OSS）。它是开源运动的权威机构，负责制定和维护开源定义（Open Source Definition, OSD），并认证符合标准的开源许可证。

采用 OSI 标准的著名框架

采用 OSI 标准的著名框架	
许可证	代表项目
MIT	React (Meta), Ruby on Rails, Node.js
Apache 2.0	Android, Kubernetes, TensorFlow
GPL	Linux 内核, Git, WordPress
BSD	FreeBSD, Nginx（早期）
LGPL	GNU C 库, Blender

《开源定义》包括 10 项关键准则

- （1）自由再分发
- （2）源代码可用
- （3）允许衍生作品
- （4）作者源代码的完整性
- （5）不歧视个人或团体
- （6）不歧视领域
- （7）许可证分发
- （8）许可证不能针对特定产品
- （9）许可证不能限制其他软件
- （10）许可证必须是技术中立的

1.4.2 大模型社区的开源

如果按照《开源定义》，目前大多数“开源”大模型都不符合标准。

大模型的数据集、源代码和数据清洗 workflows 相当于传统软件的源代码，这是企业的核心竞争力，如果开源可能直接帮助竞争对手，且数据集可能涉及版权问题，所以主流的大模型的开源都不包含以上内容。

大模型的开源通常包含权重、模型结构定义、模型推理代码、分词器实现、微调代码等，这是使用大模型的必要条件。开发者可以利用这些资料独立部署和微调大模型，但无法完整复现预训练模型。

因此，目前开源大模型和闭源大模型的根本区别在于是否提供权重。

1.4.3 OSI 开源 AI 定义 1.0

根据 OSI（Open Source Initiative）2024 年 10 月发布的开源 AI 定义 1.0（简称：OSAID 1.0），其核心标准围绕“四大自由”展开，旨在确保 AI 系统的透明度、可访问性和可复用性。

1) 核心标准

OSAID 1.0 要求开源 AI 系统必须满足以下四大自由，且需提供完整的技术组件以支持这些自由：

- （1）使用自由：可将系统用于任何目的（包括商业用途），无需额外许可。
- （2）研究自由：可访问系统所有组件（包括设计、数据、代码），以研究其工作原理。
- （3）修改自由：允许修改系统，包括调整输出或内部机制。
- （4）分享自由：可自由共享原始或修改后的系统，供他人使用。

对于机器学习系统的技术要求如下：

（1）数据透明度：

提供训练数据的完整描述，包括来源、范围、处理方式（如过滤、标注方法）。
需公开所有可获取数据的途径（包括付费数据源），确保他人可重建“实质等效”的系统。

（2）完整代码：

开源训练、数据预处理、推理的完整源代码（如数据处理脚本、训练框架、模型架构代码），且需使用 OSI 批准的许可证。

（3）参数开放：

模型权重、配置参数必须开放，并提供中间训练检查点（如适用）。

核心目标：通过完全透明的技术堆栈，使任何人能复现、修改和迭代 AI 系统，而非仅提供“黑箱”使用权。

2) 当前开源大模型是否满足 OSAID-1.0 的标准

目前主流“开源大模型”均未完全满足 OSAID 1.0 标准。

以开源大模型 Llama 和 Qwen 为例。

评估维度	OSAID 1.0 要求	Llama 3	Qwen
数据透明度	✅ 完整数据来源、处理方法和获取途径	❌ 仅笼统描述数据构成	❌ 未公开数据处理细节

代码完整性	✅ 训练/数据处理/推理全栈代码开源	❌ 仅开源推理和微调代码	❌ 缺少核心训练代码
参数开放性	✅ 权重和中间检查点开放	✅ 权重开放	✅ 权重开放
许可证合规性	✅ 无使用限制(包括商业)	❌ 限制大平台商用(7 亿 MAU)	✅ Apache 2.0(无限制)
能否复现模型?	✅ 完全可复现	❌ 否	❌ 否

1.4.4 总结

无论是传统的 OSI《开源定义》还是 OSAID-1.0，都是理想化的严格标准，要求实现真正的透明与可复现。

当前所有主流“开源大模型”均未达标，主要因素可能是避免直接帮助竞争对手、避免版权问题等。

目前大模型的开源都包含**权重开源**，可能还会包含**模型设计代码、微调代码**等，但不包含**预训练代码、数据集和数据处理 workflow**。

1.5 常用网站

1.5.1 看论文

<https://arxiv.org/>
<https://modelscope.cn/papers>

1.5.2 模型天梯网站

OpenRouter 天梯
<https://openrouter.ai/rankings>

vellum 天梯
<https://www.vellum.ai/llm-leaderboard>

SuperGLUE 天梯
<https://super.gluebenchmark.com/leaderboard>

1.5.3 模型版本迭代网站

<https://www.datalearner.com/ai-models/>

1.5.4 论文翻译神器

沉浸式翻译（浏览器插件）

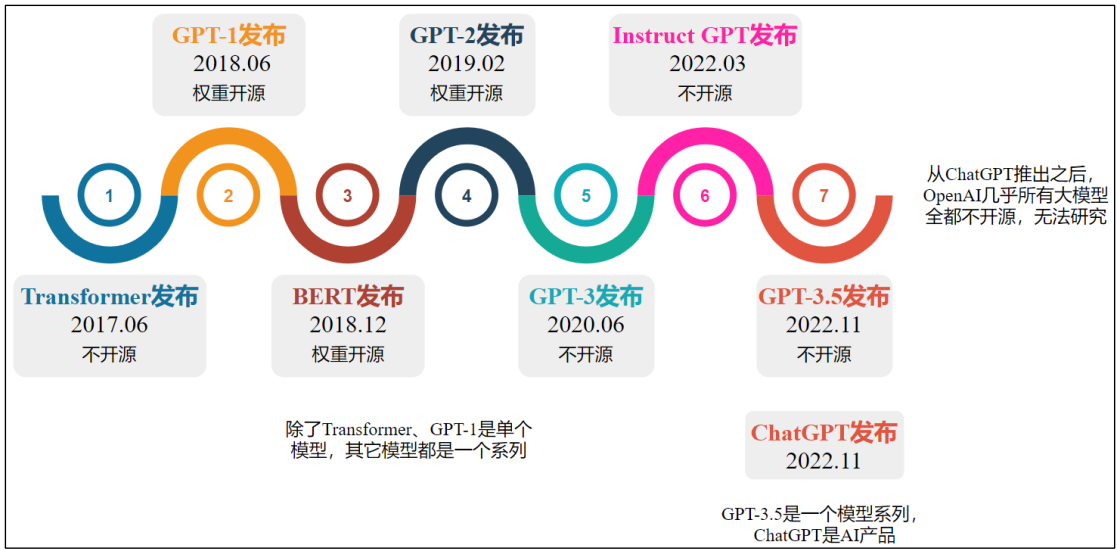
➤ 官网地址：<https://immersive-translate.owenyoung.com/>

➤ 插件安装地址（Chrome）：<https://chrome.google.com/webstore/detail/immersive-translate/bpoadfkcbjbfhfodiogcnhhpibjhbnh>

第 2 章 GPT 系列

ChatGPT-3 是第一个真正意义上的 LLM，它是基于之前的一系列模型演化而来。

2.1 GPT 系列模型发布时间线



2.2 GPT-1

2.2.1 知识储备（按需查阅）

2.2.1.1 迁移学习（Transfer Learning）

1) 定义

将源领域（Source Domain）或源任务（Source Task）中学到的知识（如模型参数、特征表示（如词嵌入）等），迁移到目标领域（Target Domain）或目标任务（Target Task）中，以提升目标模型的性能或训练效率。

（1）核心思想：利用已有知识解决新问题，减少对目标领域大量标注数据或计算资源的依赖。

（2）关键术语：

- 领域（Domain）：由数据及其分布组成（如医疗影像 vs. 自然图像）。
- 任务（Task）：指模型的具体目标（如分类、分割等）。

2) 迁移学习的典型场景

（1）预训练+微调（如 GPT、BERT）：在大规模无监督数据上预训练，再在小规模标注数据上微调目标任务。

（2）特征提取：固定预训练模型的部分层，仅训练新添加的分类层。

（3）领域自适应（Domain Adaptation）：将源领域（如合成图像）的知识迁移到目标领域（如真实照片）。

2.2.1.2 监督学习（Supervised Learning）

利用标注数据（输入-输出对）训练模型，学习从输入到输出的映射关系。模型通过最小化预测值与真实标签的误差进行优化。

2.2.1.3 无监督学习 (Unsupervised Learning)

从无标注数据中挖掘隐藏模式或结构（如聚类、降维、关联规则），无需人工标注指导。具体到 NLP 领域，无监督学习是通过上文预测下一个词或通过上下文预测中间词等形式实现的。

2.2.1.4 半监督学习 (Semi-Supervised Learning)

结合少量标注数据和大量无标注数据进行训练。标注数据成本高昂，通过这种方式节约大量的人力成本，利用海量低成本的无标注数据提升了模型性能。

2.2.1.5 文本蕴含任务 (Textual Entailment / Natural Language Inference, NLI)

判断一对句子之间的逻辑关系，主要分为三类：

- 蕴含 (Entailment)：前提句可以推导出假设句。
- 矛盾 (Contradiction)：前提句与假设句内容相矛盾。
- 中立 (Neutral)：前提句与假设句无明显关系，不能确定真假。

示例：

- 前提：“猫坐在垫子上。”
- 假设：“垫子上有动物。”
- 输出：蕴含。

2.2.1.6 问答任务 (Question Answering, QA)

让模型根据提供的上下文回答用户提出的问题。类型包括：

- 抽取式问答 (Extractive QA)：从文本中直接抽取答案。
- 生成式问答 (Generative QA)：基于知识生成新的自然语言答案。
- 开放域问答 (Open-domain QA)：不提供上下文，由模型自行查找知识来源回答。

示例：

- 上下文：“巴黎是法国的首都。”
- 问题：“法国的首都是哪里？”
- 答案：“巴黎”。

2.2.1.7 语义相似性评估 (Semantic Similarity)

评估两个句子或文本片段在语义上的相似程度，通常输出一个 05 或 01 之间的分值。常用于句子匹配、信息检索、重复检测等任务。

方法：

- 基于词重叠（如 TF-IDF）。
- 基于嵌入模型（如 BERT、Sentence-BERT）。

示例：

- 句子 1：“手机没电了。”
- 句子 2：“我的电话需要充电。”

- 相似度：0.9（高度相似）。

2.2.1.8 文档分类（Document Classification）

将文档归入一个或多个预定义类别中，任务包括：

- 单标签分类：每个文档只属于一个类别。
- 多标签分类：每个文档可能属于多个类别。

应用场景：垃圾邮件检测、新闻分类、情感分类等。

示例：

- 文本：“这款手机电池续航出色！”
- 类别：“正面评价”。

2.2.1.9 情感分析（Sentiment Analysis）

识别文本中表达的情感倾向（如正面、负面、中性）或更细粒度的情绪（如愤怒、高兴）。

方法：

- 基于词典规则（如情感词统计）。
- 基于机器学习（如 LSTM、Transformer）。

示例：

- 评论：“电影剧情糟糕，但特效很棒。”
- 输出：混合情感（负面+正面）。

情感分析是文本分类的一种，但因其独特性和广泛应用，常被单独列为 NLP 核心任务，与文档分类并列（如 GLUE 基准中同时包含主题分类和情感分析任务）

2.2.1.10 消融实验（Ablation Study）

通过逐步移除模型的某些组件（如层、特征、模块），评估其对性能的影响，以验证这些组件的必要性。

目的：

- 确定关键设计对模型效果的贡献。
- 简化模型结构（如去除冗余部分）。

示例：

- 在 BERT 模型中移除注意力机制后，准确率下降 15%，说明注意力机制至关重要。

2.2.1.11 TF-IDF（Term Frequency-Inverse Document Frequency）

TF-IDF 即词频-逆文档频率，它是一种统计方法，用于衡量一个词在文档中的重要性。这种方法认为一个词在单个文档中出现的频率越高、包含该词的文档数越少，它所包含的信息量越大，对应的 TF-IDF 值越大，后者由两部分组成：

1) 词频（Term Frequency, TF）

$$TF(t, d) = \frac{\text{词}t\text{在文档}d\text{中出现的次数}}{\text{文档}d\text{的总词数}}$$

2) 逆文档频率 (Inverse Term Frequency, IDF)

$$\text{IDF}(t) = \log\left(\frac{\text{总文档数}}{\text{包含词}t\text{的文档数}}\right)$$

3) IF-IDF 值

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

2.2.1.12 AdamW

1) Adam 算法回顾

让我们回顾一下传统的 Adam 算法的执行步骤。

(1) 计算纯梯度 (不含 L2) :

$$g_t = \nabla_{\theta} J(\theta_{t-1})$$

(2) 更新一阶矩:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

(3) 更新二阶矩:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

(4) 偏差修正:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

(5) 参数更新:

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right)$$

上式中的 ϵ 是一个极小值, 目的是防止分母为零。

2) L2 正则化回顾

L2 正则化通过修改损失函数实现, 修正后的损失函数如下

$$J_{\text{reg}}(\theta) = J(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

相应的, 梯度如下

$$g_t = \nabla_{\theta} J(\theta_{t-1}) + \lambda \theta_{t-1}$$

3) Adam 算法和 L2 正则化组合使用的问题

当 L2 正则化和 SGD 组合使用时, 参数更新公式如下

$$\theta_t = \theta_{t-1} - \eta \nabla_{\theta} J(\theta) - \eta \lambda \theta_{t-1}$$

此时等价于对参数直接施加权重衰减 ($-\eta \lambda \theta_{t-1}$ 项)。

当 Adam 算法和 L2 正则化组合使用时, 正则化项的梯度 $\lambda \theta$ 会被除以梯度二阶矩估计, 导致实际的正则化强度随训练动态衰减, 无法稳定约束权重。

4) AdamW 算法

为了解决上述问题，Ilya Loshchilov 等人提出了改进的 L2 正则化方法 AdamW，英文全称为 Adam with Weight Decay，即带有权重衰减的 Adam 算法。顾名思义，该算法和 Adam 相比，只是在参数更新阶段添加了权重衰减，其它部分完全相同。需要特别注意的是，AdamW 算法的梯度计算和 Adam 一样，不包含 L2 正则化项。

参数更新阶段改进后的公式如下

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right)$$

即

$$\theta_t = \theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon} - \eta \lambda \theta_{t-1}$$

AdamW 算法本质上只是添加了 $-\eta \lambda \theta_{t-1}$ ，等价于 Adam 算法与解耦权重衰减（ $-\eta \lambda \theta_{t-1}$ 项）相结合。保留了正则化的数学本质，恢复了权重衰减的物理意义（参数收缩）。

在 AdamW 中， λ 通常被称为权重衰减系数（Weight Decay Coefficient），而不是 L2 正则化强度系数。

AdamW 通常在实践中（训练基于 Transformer 架构的 LLM 时）比 Adam+L2 表现出更好的泛化性能，因此在 LLM 的训练中被广泛采用。

2.2.2 概述

1) 主要成果及创新

自然语言理解涵盖文本蕴含、问答、语义相似性评估和文档分类等多种任务。尽管无标注文本语料库资源丰富，但这些任务都需要特定的标注数据，大量的无标注数据无法被利用。在 GPT-1 之前，上述任务都是基于特定的标注数据训练特定的判别式模型完成的，由于标注数据的稀缺和高昂成本，这些判别式模型无法取得良好的表现。

（1）GPT-1 是**首个完全基于 Transformer 解码器架构构建的大规模语言模型**

首次证明纯解码器架构可胜任语言建模，奠定了后来纯解码器大语言模型的基础。

（2）GPT-1 是**首个自回归语言模型**

移除编码器-解码器交叉注意力，改为单向自回归。它的训练目标就是基于前面所有的词，预测序列中的下一个词。这种单向性是其生成能力的核心。

（3）**首次将 Transformer 架构与两阶段训练模式（无监督预训练+有监督微调）结合**

通过在多样化无标注文本上进行生成式预训练，再针对不同的任务使用特定的标注数据微调，可以显著提升大多数自然语言理解任务的性能。这是 GPT-1 的核心方法论贡献，充分利用了成本低廉的海量无标注文本语料库资源。

它极大地降低了每个任务对标注数据的依赖，显著提升了任务性能，并成为后续几乎所有大语言模型（LLM）的标准训练范式。需要注意的是，GPT-1 的微调通常需要在预训练模型基础上添加一个线性层，有别于未来的 prompt tuning。

（4）GPT-1 是**首个在大规模语言模型中成功应用 AdamW 优化器（等价于 Adam+解耦权重衰减）的先驱**

GPT-1 首次在大规模语言模型验证了 AdamW 优化器的可用性，这种设计使 GPT-1 仅在 5GB 数据上训练出强大的泛化能力，其正则化方案比模型架构创新影响更为深远，直接塑造了现代 LLM 的训练范式。后续的 BERT（通

常不认为是 LLM，但和 GPT-1 在一段时间内是“旗鼓相当的对手”）、GPT-1、GPT-2、GPT-3、Llama 系列、Qwen 系列、DeepSeek 系列的部分或全部模型采用的也都是这种优化器。

2) 训练范式

GPT-1 采用无监督预训练+监督微调的架构，称为半监督学习方法。采用两阶段训练流程：

首先在未标注数据上使用语言建模（预测下一个词）学习神经网络的初始参数

随后通过相应的标注数据将初始参数适配至目标任务。

3) 数据准备

（1）无监督预训练阶段采用清洗后的 BooksCorpus 数据集（一个全英文的文本数据集），包含了 7000 多本未出版的书籍，包含约 10 亿词（英文一个 token 约等于一个词），约 5GB 纯文本数据。

（2）监督微调阶段的数据集如下表所示

任务	数据集
自然语言理解	SNLI, MultiNLI, Question NLI, RTE, SciTail
问答	RACE, Story Cloze
句子相似度	MSR Paraphrase Corpus, Quora Question Pairs, STS Benchmark
文本分类	Stanford Sentiment Treebank-2, CoLA

4) 词表构建

词表通过 BPE 方式经过 40000 合并，而后添加一些特殊标记构建，大小为 40478

关于词表大小可参考 openAi 开源的 GPT-1 权重仓库，model/encoder_bpe_40000.json 文件给出了完整的词表，仓库链接：<https://github.com/openai/finetune-transformer-lm.git>。

5) 训练和参数更新策略

（1）训练批次为 64，上下文长度即 seq_len 为 512，共训练 100 轮（即 epoch 为 100）

（2）通过 $N(0,0.02)$ 初始化权重

前馈层的激活函数选用高斯误差线性单元 GELU

位置嵌入采用可学习参数版本

（3）学习率 η 在前 2000 次更新中从零线性增长，最大值为 2.5×10^{-4} ，随后通过余弦调度衰减至 0。

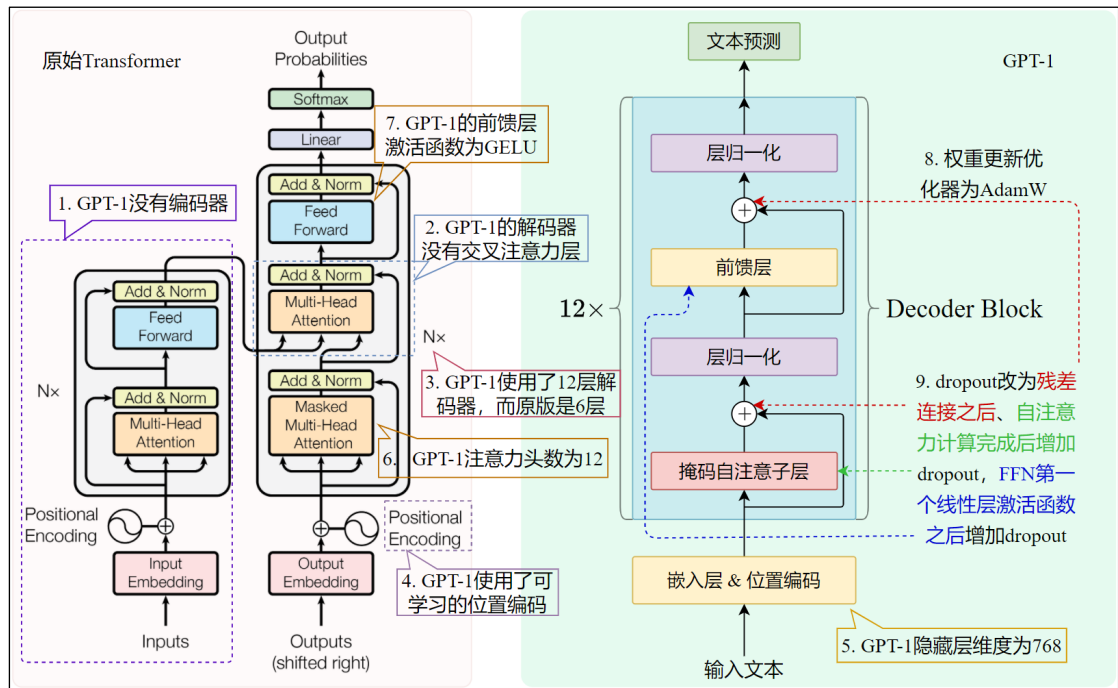
（4）采用 AdamW 优化策略更新权重，权重衰减系数为 0.01

（5）对残差连接、嵌入层和注意力层及 FFN 的第一个线性层之后应用比率为 0.1 的 dropout 正则化

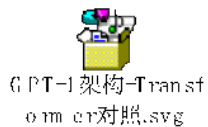
2.2.3 GPT-1 模型架构

2.2.3.1 GPT-1 和 Transformer 模型架构对照

GPT-1 采用了预训练+微调的训练范式，微调阶段的模型完整包含了预训练模型，此处对预训练模型和 Transformer 原始模型进行对照。



原图文件



	Transformer	GPT-1
组件类型	编码器+解码器	解码器
解码器层数	6 层	12 层
解码器内部架构	掩码自注意力+交叉注意力+前馈层	掩码自注意力+前馈层
位置编码	正余弦位置编码	可学习的位置编码
隐藏层维度 d_{model}	512	768
注意力头数	8	12
前馈层激活函数	RELU	GELU
前馈层中间隐藏层维度	4 倍 d_{model}	4 倍 d_{model}
嵌入层和输出线性层权重共享	是（需要注意的是，哈佛实现的 The Annotated Transformer 没有共享权重）	是
优化器	Adam	AdamW
Dropout 正则化位置	1.嵌入和位置编码之后 2.每个子层输出之后， 残差连接之前	1.嵌入和位置编码之后 2. 残差连接之后

		3.FFN 第一个线性层激活函数之后 4.注意力计算完成后，和 V 做矩阵乘法之前
层归一化	Post-LayerNorm	Post-LayerNorm

注意：

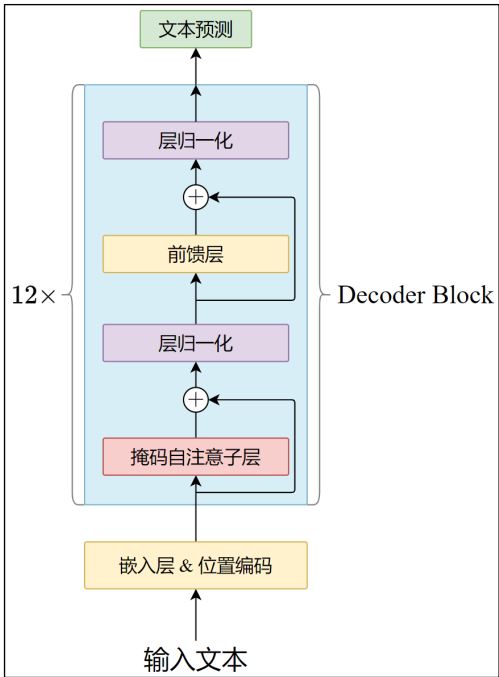
Transformer 原始论文未明确提及在 FFN 内部（特别是第一个线性层激活函数后）应用 Dropout。然而，由于 FFN 中间层维度（ $4 * d_{model}$ ）远大于输入维度，极易过拟合，因此在实践中（如 The Annotated Transformer 等主流实现），在 FFN 第一个线性层激活函数后应用 Dropout 已成为提升泛化能力的**标准且必要的做法**。

GPT-1 论文声称 “Our model largely follows the original transformer work”，但未明确说明是否在 FFN 内部应用了 Dropout。鉴于目前官方实现未公开模型定义细节，无法确定。

考虑到标准实践和模型设计对正则化的需求，普遍认为 GPT-1 在 FFN 子层的第一个线性层激活函数后也应用了 Dropout 正则化。本文档后续分析采用此观点。

2.2.3.2 预训练阶段

1) 模型架构



- (1) GPT-1 使用 12 层仅含解码器的 Transformer 模型
- (2) 使用掩码自注意力机制（也叫因果自注意力，Causal Self-Attention）
- (3) 隐藏状态为 768 维 (d_{model})，12 个注意力头， $d_k = d_q = d_v = d_{model} \div 12 = 64$
- (4) 前馈层采用 $4 \times d_{model} = 3072$ 维内部状态。

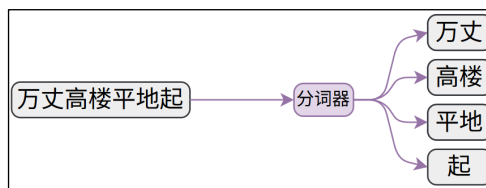
2) 训练数据的输入和目标构造

以“万丈高楼平地起”为例

(1) 原始文本输入模型后首先分词

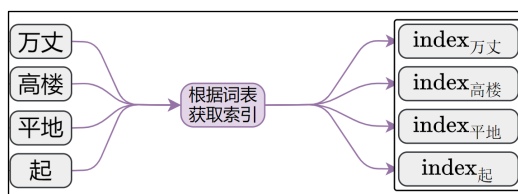
分词结果取决于分词器，假设分词结果为

["万丈", "高楼", "平地", "起"]

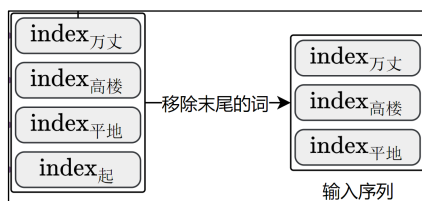


(2) 根据词表将文本转换为索引序列

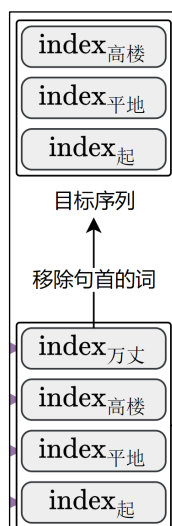
需要注意的是，不同于 BERT 等模型，GPT-1 在预训练阶段没有添加特殊的起始和结束标记如<SOS>、<EOS>，而是直接对原始文本建模。这是因为这一阶段的主要目的是训练语言理解能力，而不是生成连续文本，因此不需要知道何时开始和终止输出。



(3) 截取最后一个词之前的部分作为**输入**



(4) 截取第一个词之后的部分作为**目标序列**



3) 目标函数和损失函数

对于输入的 token 序列 $\mathcal{U} = \{u_1, \dots, u_n\}$ ，预训练阶段的目标是最大化似然函数，如下

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

这就是目标函数。

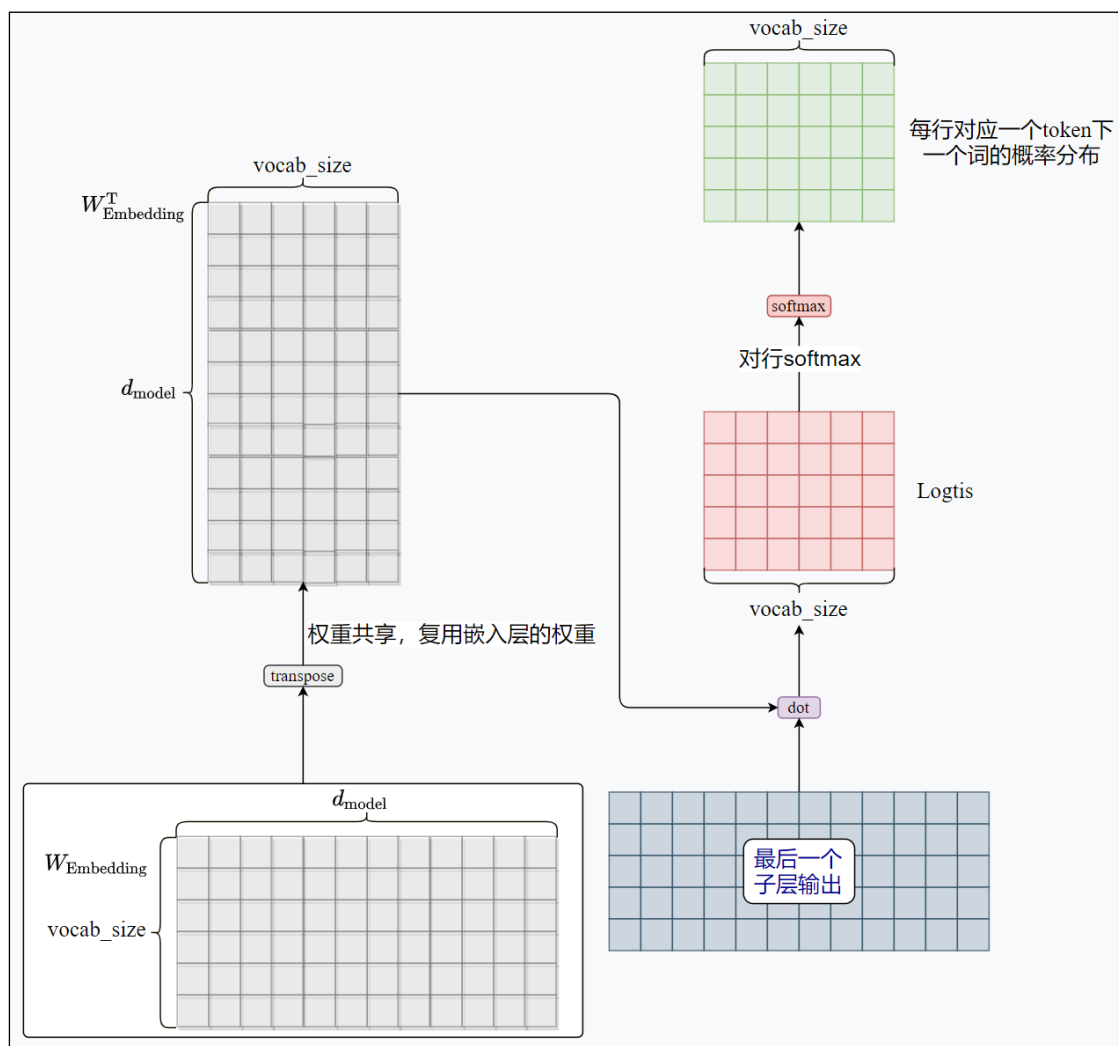
对上式取负值就是基于独热编码的交叉熵损失函数，如下

$$J_1(U) = - \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

4) 输出线性层权重优化

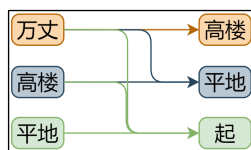
在哈佛大学实现的 The Annotated Transformer 实现中，最后一层解码器的输出经过一个独立的线性层，再通过 softmax 归一化，最终生成下一个 token 的概率分布。

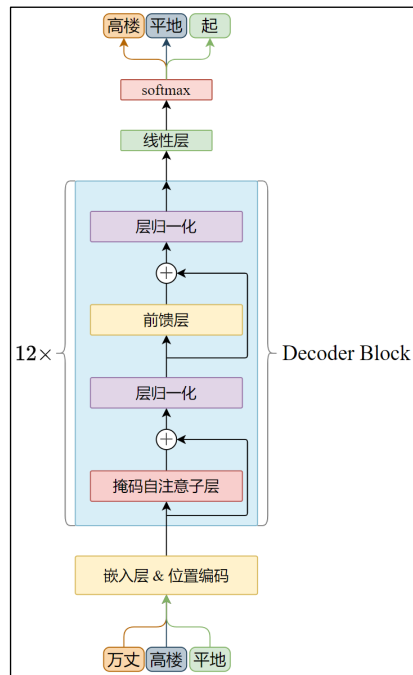
而在原版 Transformer 论文中，编码器、解码器的嵌入矩阵和上面的输出线性层共享相同的权重矩阵，GPT-1 使用了类似原版 Transformer 的方式，即输出线性层复用了解码层的嵌入矩阵，通过这种方式缩减了参数量。



5) 总结

综上，预训练阶段的目标是预测连续序列中的下一个词。

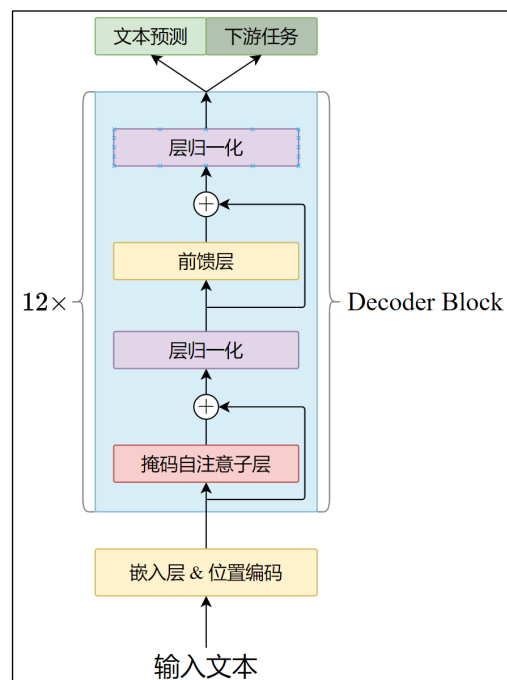




2.2.3.3 微调阶段

1) 模型架构

针对不同的下游任务，添加相应的线性层。



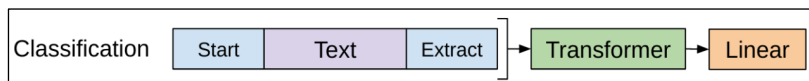
2) 训练数据的输入构造

文本蕴含、文本相似度判断等任务的标注数据都是结构化文本，如有序的句子对、问答组合等，GPT-1 将这些数据全部处理为连续的文本序列。

所有任务的原始输入首尾都要添加特殊的 Start 和 Extract，用于标记任务的开始和结束。

Start 和 Extract 是随机初始化的。

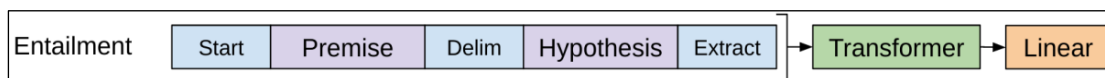
(1) 文本分类任务



上图中的 Transformer 表示预训练阶段的模型主体（不包含最终的线性层），Linear 是任务特定的输出层。下同。

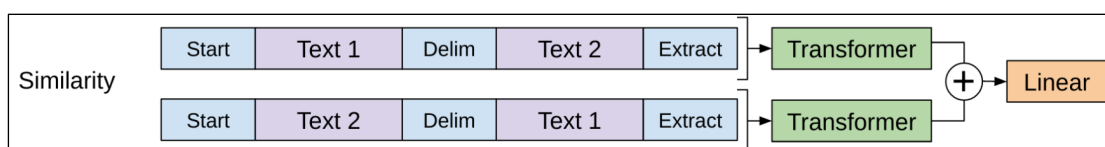
（2）文本蕴含任务

文本蕴含任务的输入由前提（Premise）和假设（Hypothesis）构成，通过特殊的分隔符\$拼接在一起。



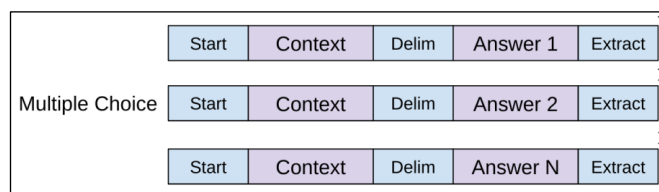
（3）语义相似度评估任务

文本相似度的输入是两个被比较句子，同样用分隔符（具体形式官方没有提及）连接。需要注意的是，二者没有先后顺序，因此调整输入序列以包含两种可能的句子排列，分别经过预训练模型的处理生成两个序列表示 h_l^m ，后者逐元素相加后输入任务特定的线性层。

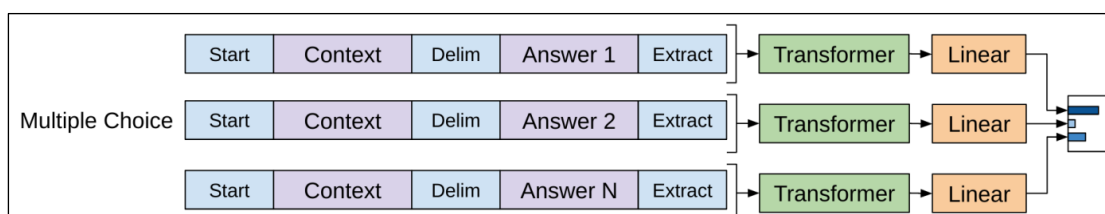


（4）问答任务

问答任务的结构化输入由上下文文档 z ，问题 q 和一系列可能的答案 a_k 构成。 z 和 q 共同构成上下文，通过分隔符依次与每个答案组合，得到和答案数量相同的 $[z; q; \$; a_k]$ 序列，如下图所示。



每个组合输入模型，经过任务特定的线性层计算得分，多个答案的得分拼接在一起，通过 softmax 得到归一化的分数，这就是最终的输出。



3）目标函数和损失函数

监督微调阶段的目标函数有两个：

（1）基于标注数据

假设有一个带标签的数据集 $\mathcal{C}$ ，其中每个实例由输入标记序列 $x^1, \dots, x^m$ 和标签 y 组成。按照上文介绍的方式构造输入，经过预训练模型处理后的隐藏层输出 h_l^m ，随后馈入新增的线性输出层（权重为 W_y ）以预测 y

$$P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y)$$

上式中的 l 表示最后一层 Decoder Block 的编号，此处为 12。

目标是最大化以下似然函数

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m)$$

(2) 基于语言建模

研发团队发现，在微调时将语言建模作为辅助目标有助于提升模型的泛化能力并加速收敛。因此，在监督微调阶段同时要语言建模，目标函数与上文提到的 $L_1(\mathcal{U})$ 一致，因为是基于监督微调的数据集建模，因此符号更改为 $L_1(\mathcal{C})$ 。

需要注意，语言建模只针对输入的 token 序列，而不针对输出序列，并且通常在计算损失时，会屏蔽 token 序列末尾的<Extract>的损失计算。

(3) 最终的目标函数

最终的目标函数为 L_1 和 L_2 的组合，如下。

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C})$$

其中 λ 是超参数，用于平衡监督目标和语言建模辅助目标的重要性。

(4) 损失函数

计算交叉熵损失，最终的损失函数刚好是 $L_3(\mathcal{C})$ 的相反数。

$$J_3(\{\mathcal{C}\}) = -L_3(\mathcal{C})$$

2.2.4 微调阶段数据处理流程

微调阶段包含了特定任务训练和语言建模，预训练阶段只有语言建模，只要清楚微调的数据处理流程，预训练的流程自然就懂了。

说明：为了降低复杂度，调整如下

- ① 不考虑批次维度
- ② 忽略偏置项
- ③ 在掩码自注意力阶段只绘制了 3 个头
- ④ d_{model} 和 d_k

2.2.4.1 微调任务及标注数据

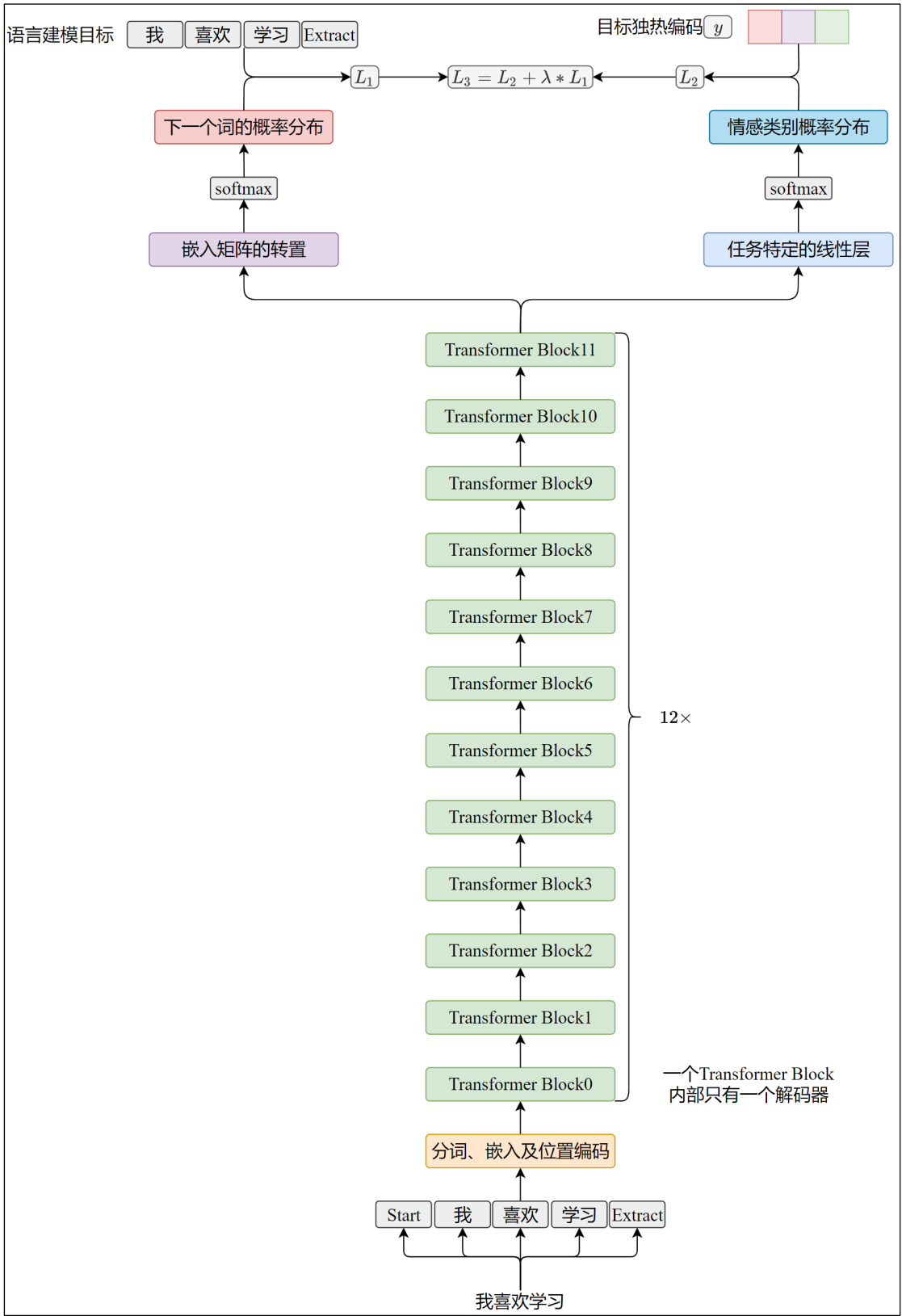
通过情感分析任务展示流程。

1) 结构化输入：我喜欢学习

2) 目标：[索引：0，名称：“积极”，独热编码：(1, 0, 0)]

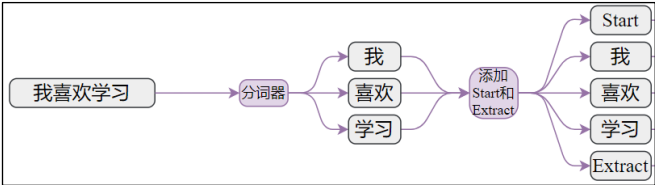
类别索引	类别名称	独热编码
0	积极	(1, 0, 0)
1	中立	(0, 1, 0)
2	消极	(0, 0, 1)

2.2.4.2 流程概览

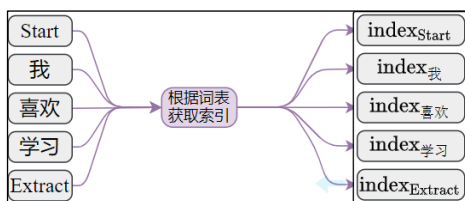


2.2.4.3 分词和嵌入

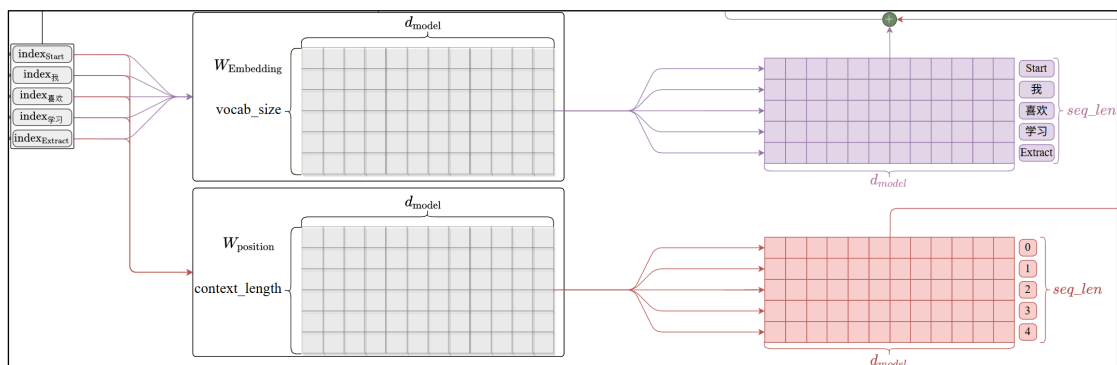
1) 分词后添加起始和结束标记



2) 将 token 映射为索引



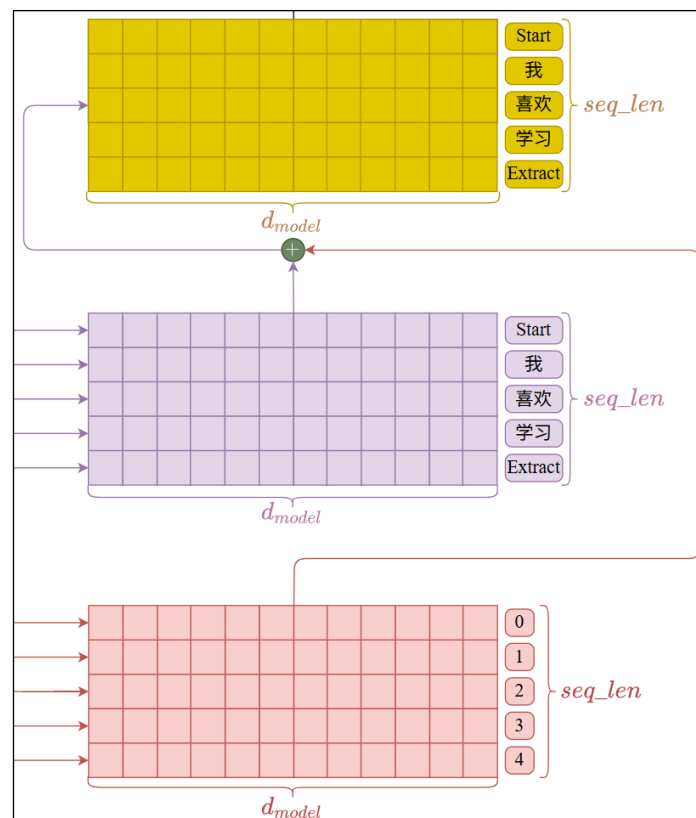
3) 基于词表索引从嵌入矩阵和位置矩阵获取向量



上图中

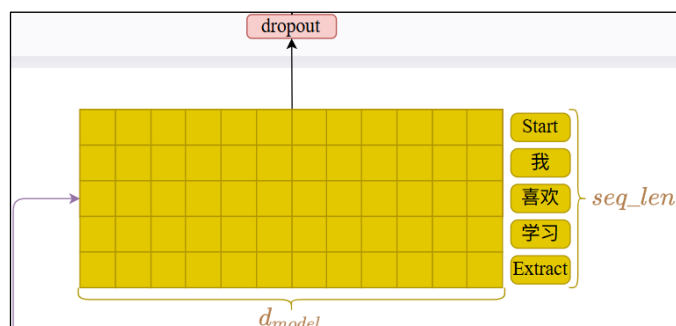
- 用 $W_{Embedding}$ 表示嵌入矩阵
 - 用 `vocab_size` 表示词表大小，GPT-1 的实际值为 40478。
 - 用 d_{model} 表示嵌入维度大小，GPT-1 的实际值为 768。
- 用 $W_{position}$ 表示可学习的位置矩阵
 - 用 `context_length` 表示上下文长度，GPT-1 为 512。不同序列的位置编码单独计算，因此位置编码矩阵的行数等于上下文长度即可。
- `seq_len` 表示序列长度，GPT-1 的序列长度即上下文长度为 512。

4) 嵌入向量和位置向量相加得到最终的 token 表示



5) dropout 正则化

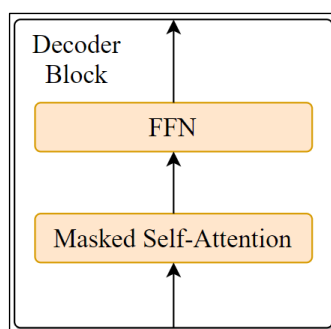
对上一步的输出应用比例为 0.1 的 dropout 正则化



2.2.4.4 Transformer Block

GPT-1 只有解码器，因此，Transformer Block 等价于 Decoder Block。

解码器由掩码自注意力层和前馈层构成

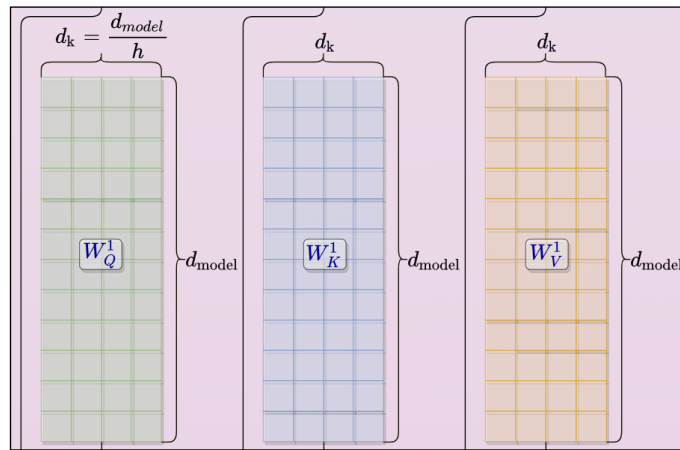


2.2.4.5 Masked Self-Attention

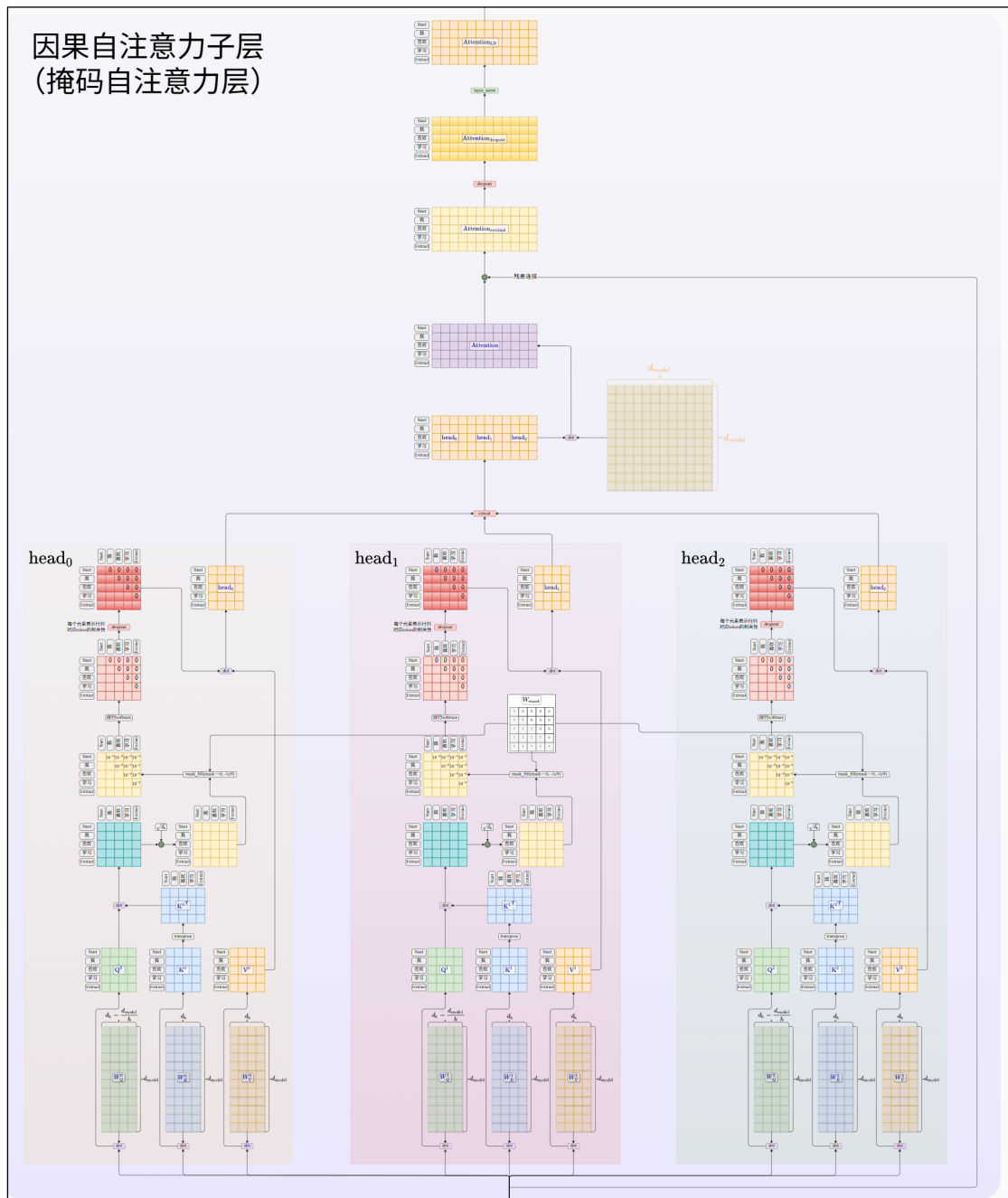
掩码自注意力，也叫（因果自注意力，Causal Self-Attention）通过引入掩码屏蔽 token 对于未来的依赖，前面的课程已有详细介绍。

GPT-1 的掩码自注意力层有 12 个自注意力头，每个头有独立的 Q、K、V 矩阵，它们的形状都是 $d_{model} * d_k$,

其中, $d_k = \frac{d_{model}}{h} = \frac{768}{12} = 64$, 如下图所示。



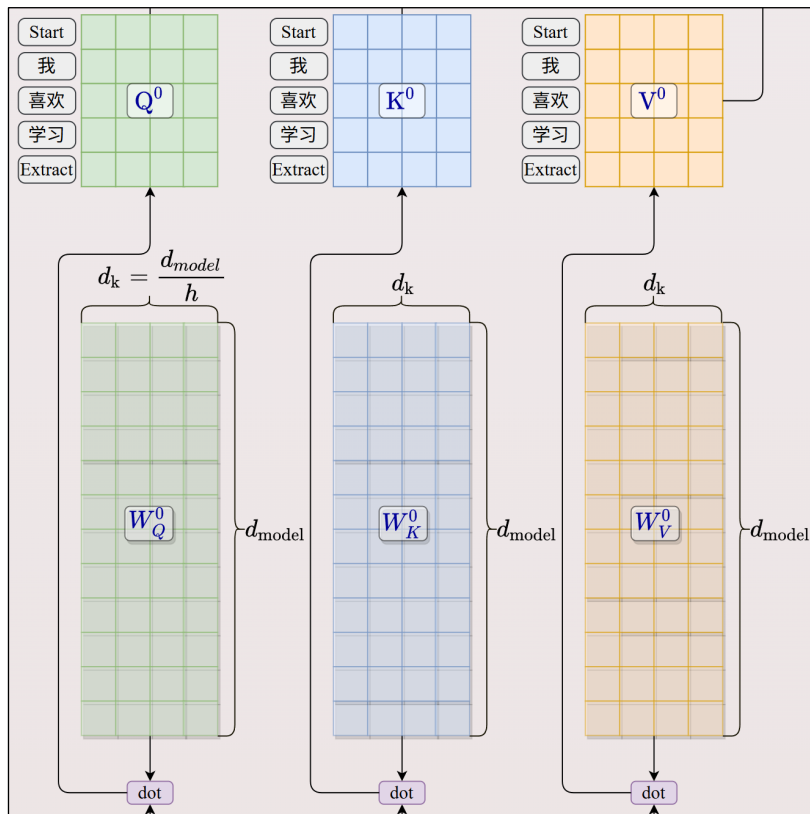
为了便于展示，此处只绘制了三个头。



以 head_0 介绍数据处理流程。

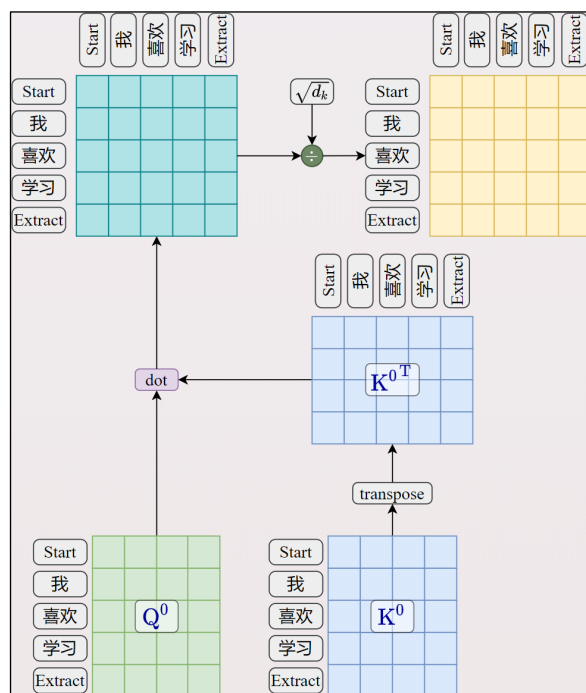
1) 投影 Q、K、V

嵌入层输出的数据分别和 W_Q^0 、 W_K^0 、 W_V^0 做矩阵乘法，得到投影矩阵 Q^0 、 K^0 、 V^0



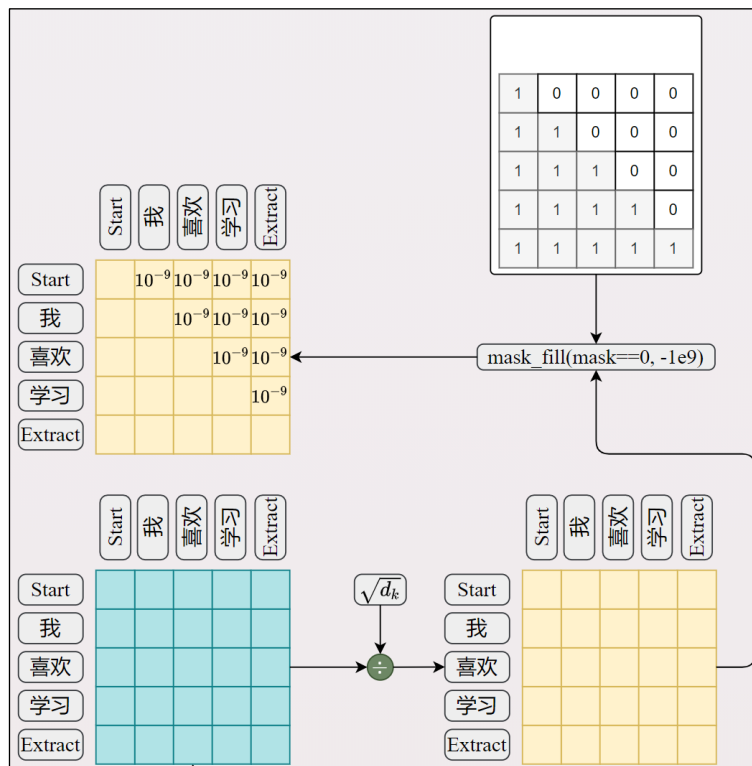
2) 计算注意力并缩放

$$\frac{Q^0 \cdot K^{0T}}{\sqrt{d_k}}$$

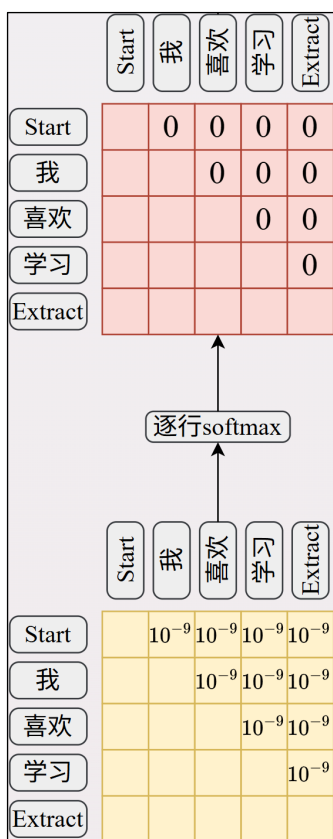


3) 掩码

根据掩码矩阵，将注意力矩阵对角线以上的部分置为 10^{-9}



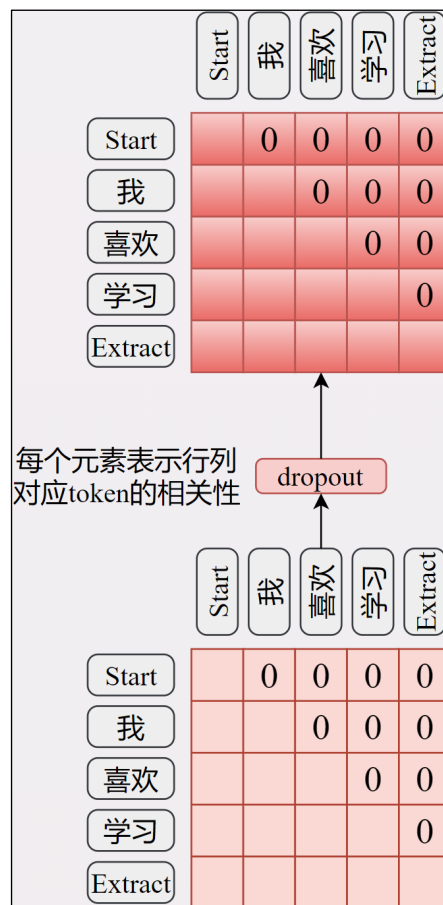
4) 逐行 softmax



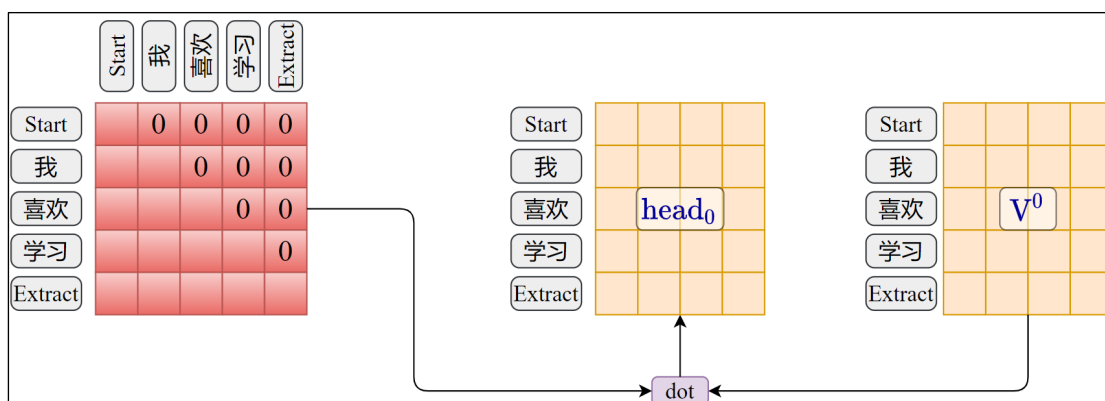
经过 softmax，上一步被置为极小值的部分归零。

此时每个单元格表示行对应 token 和列对应 token 的相关性，并经过归一化。

5) 按照 0.1 的比例 dropout 正则化

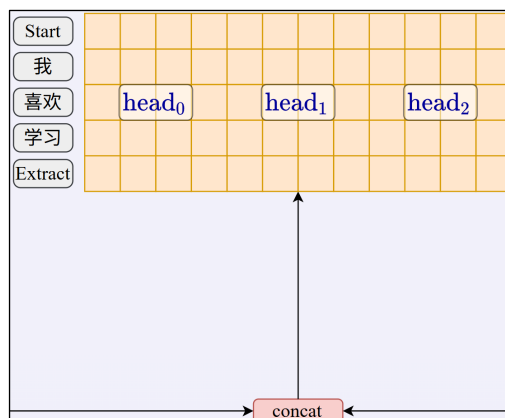


6) 对 V^0 加权得到 $head_0$



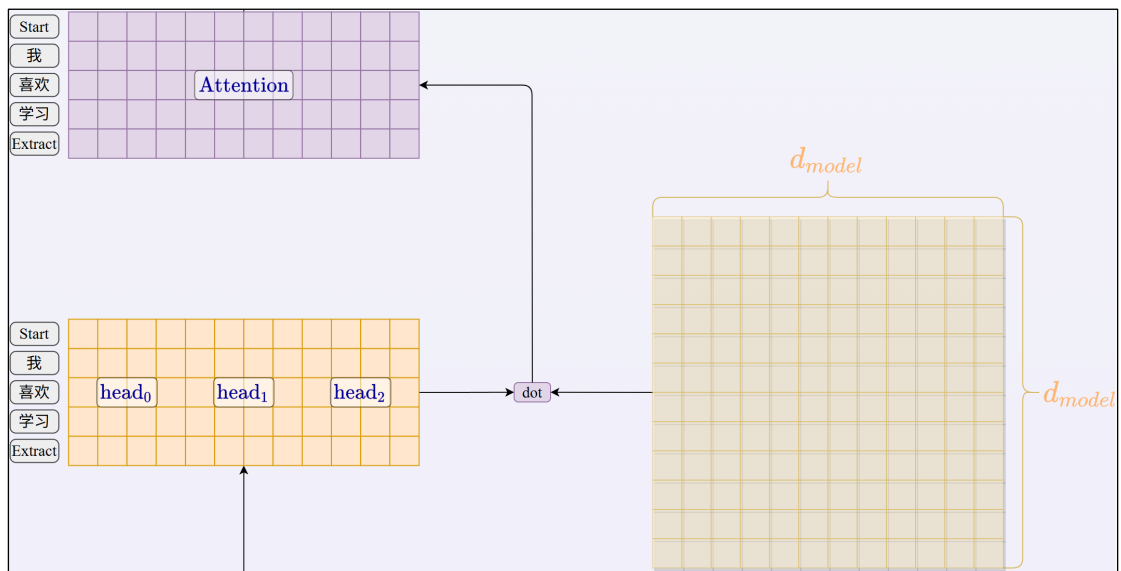
7) 通过 concat 合并所有头的输出

注意：此处仅绘制三个头，实际上 GPT-1 有 12 头。



8) 经过线性层投影

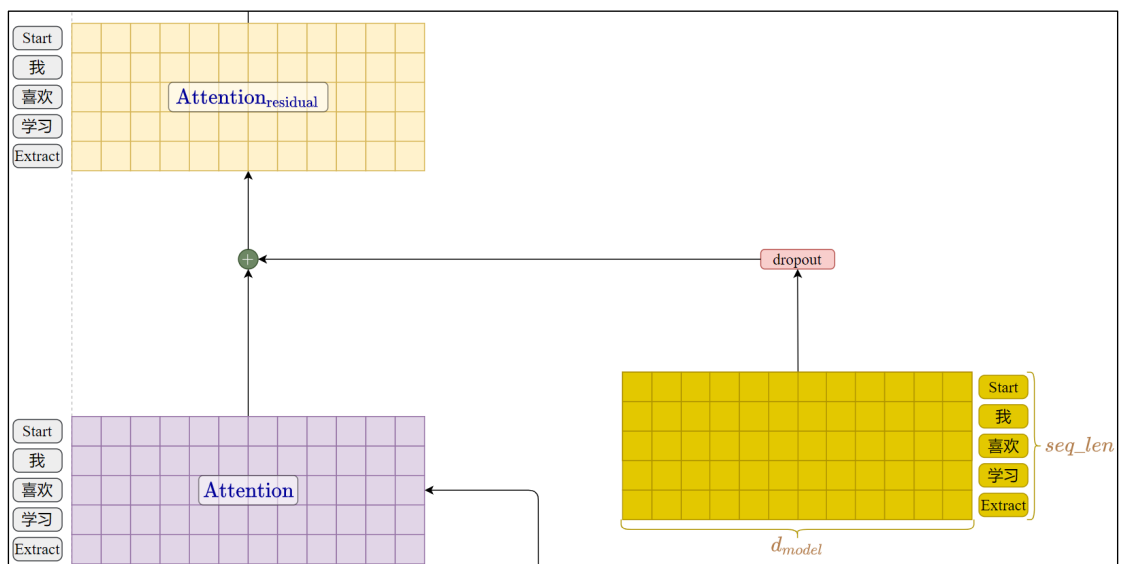
所有头的输出合并后经过大小为 $d_{model} * d_{model}$ 的权重矩阵转换得到 Attention 矩阵。



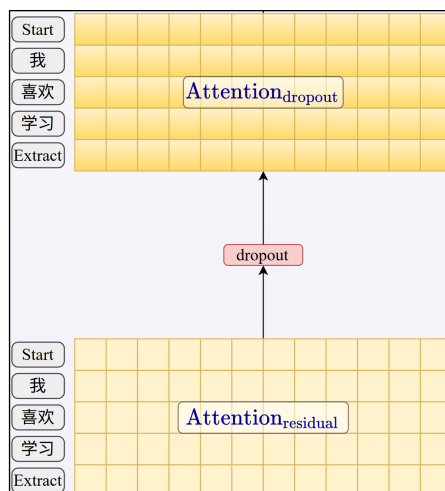
此处省略偏置项。

9) 残差连接

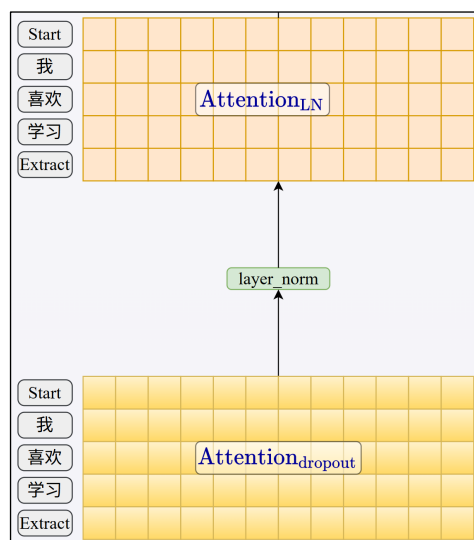
将自注意力层的输入和此处的 Attention 矩阵逐元素相加。



10) 按照 0.1 的比例 dropout 正则化



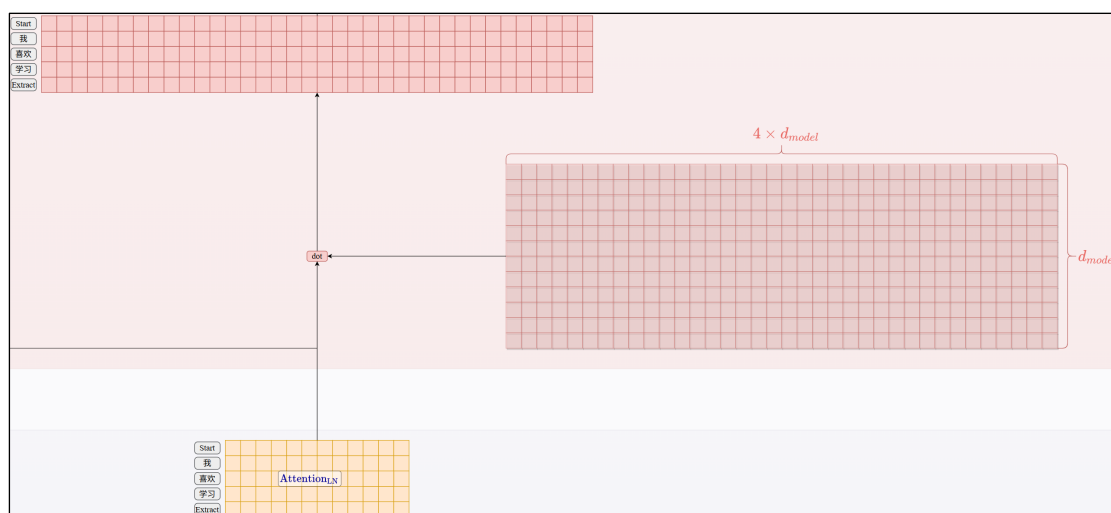
11) 层归一化



2.2.4.6 FFN（Feed-Forward Network）

这一层由两个线性层构成。

1) 第一个线性层

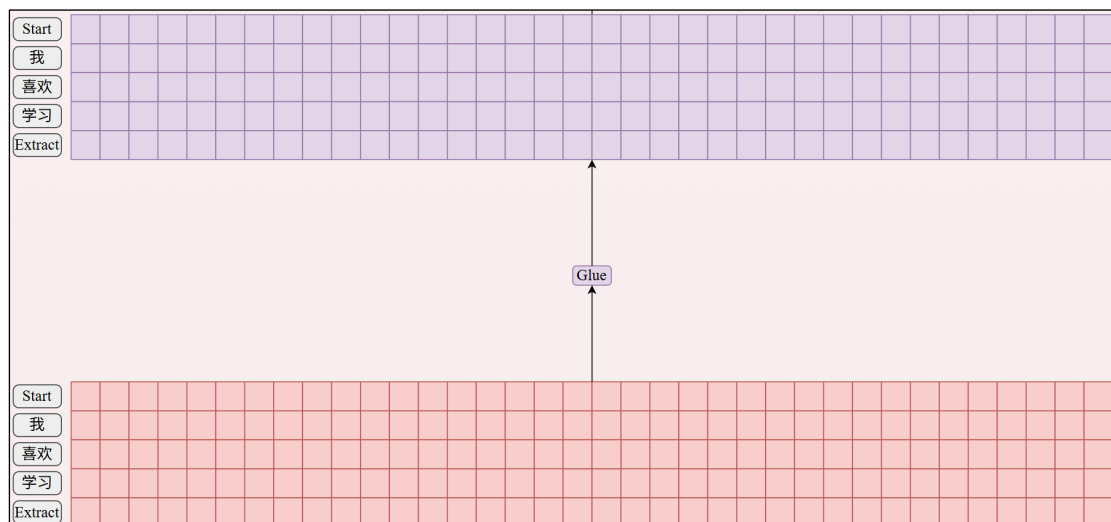


中间层的维度为 $4 * d_{model}$ ，即 3072。

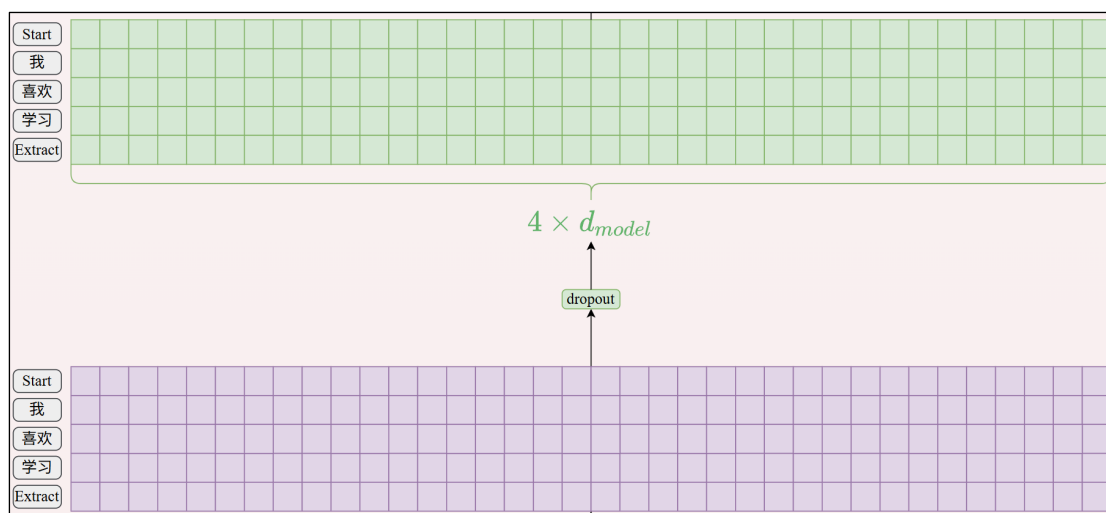
此处省略偏置项。

2) 激活函数

GPT-1 使用了不同于原始架构（RELU）的激活函数 GLUE。

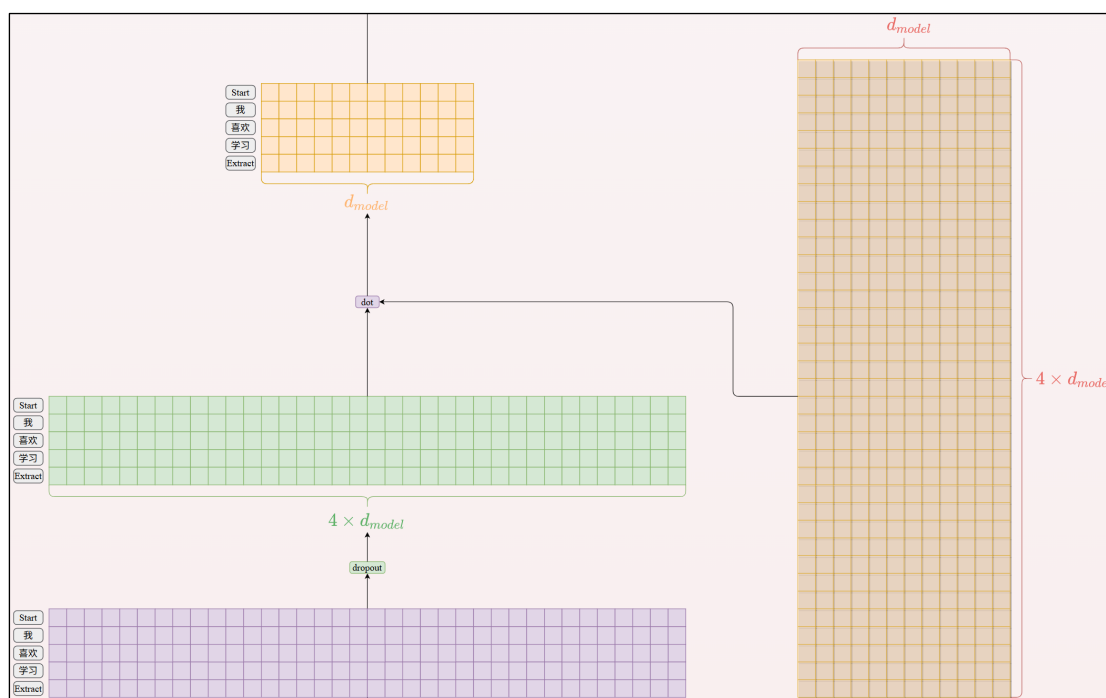


3) 按照 0.1 的比例 dropout 正则化

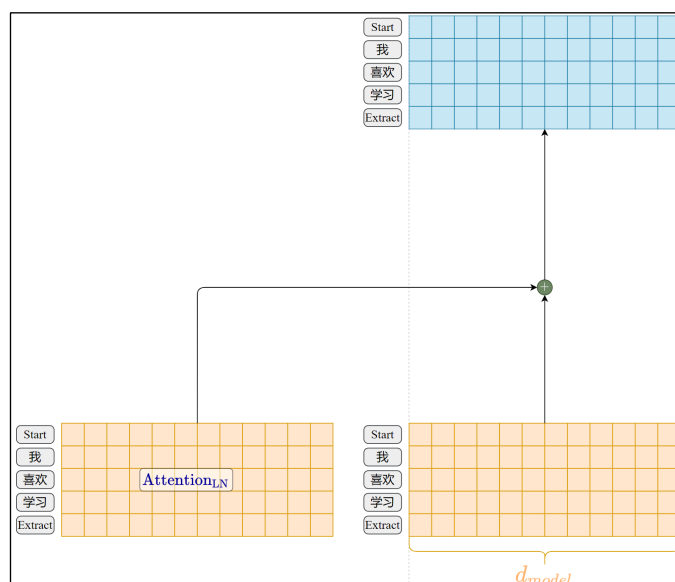


4) 第二个线性层

向量维度映射回 d_{model}

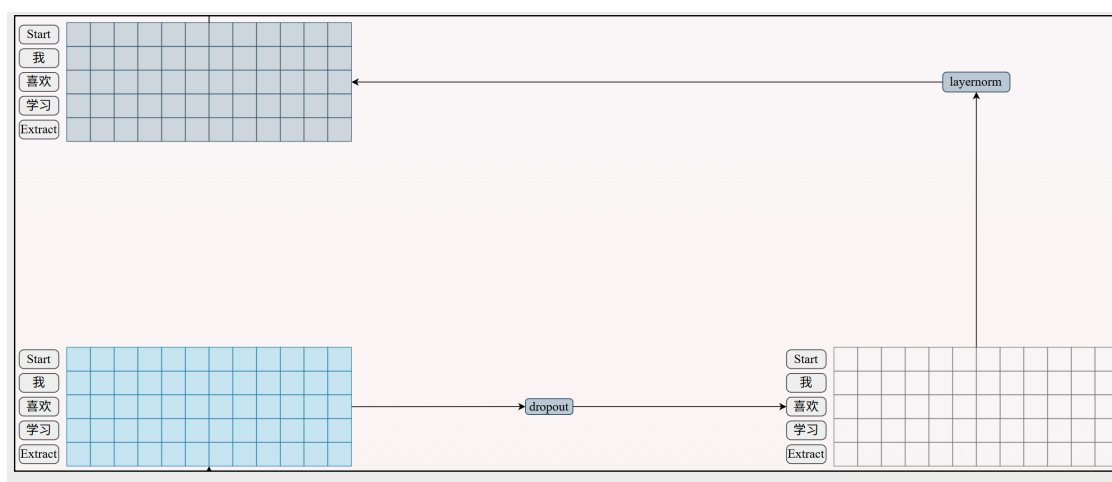


5) 残差连接



6) 按照 0.1 的比例 dropout 正则化

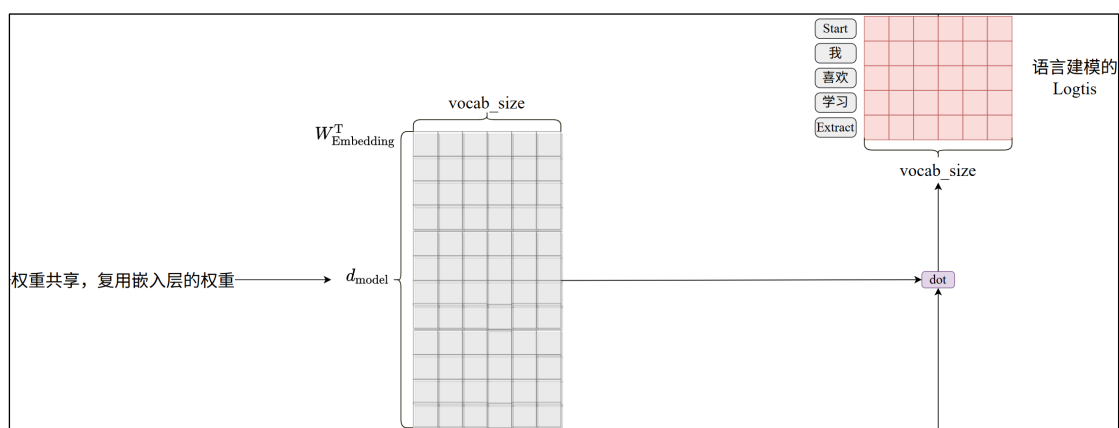
7) 层归一化



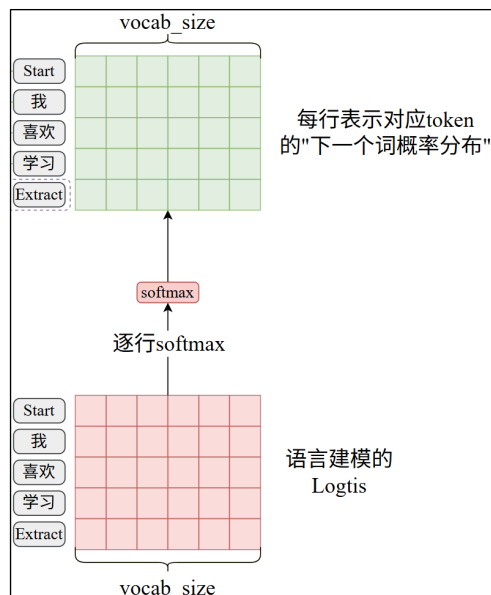
2.2.4.7 语言建模线性层

经过 12 个 Decoder Block 的处理，我们得到了最终的隐藏层输出 h_l^m 。同时进入语言建模线性层和特定任务线性层。

1) 复用嵌入矩阵计算 logits



2) 逐行 softmax 得到概率分布

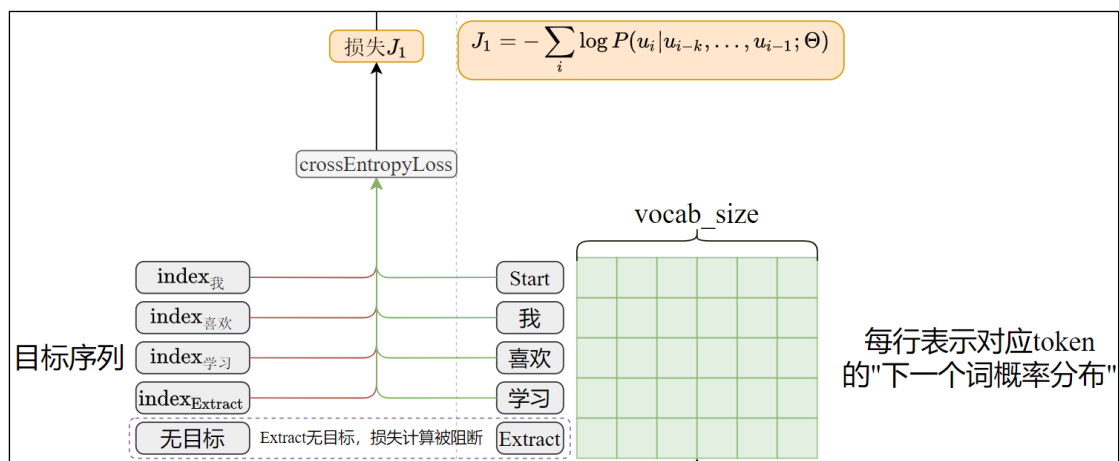


3) 目标序列

目标序列由输入序列左移一位得到，其中，末尾的 Extract 是没有目标的。



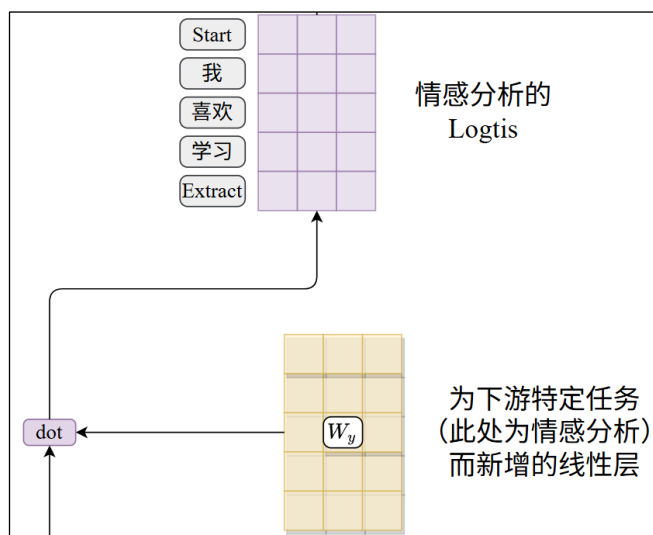
4) 交叉熵计算损失 J_1



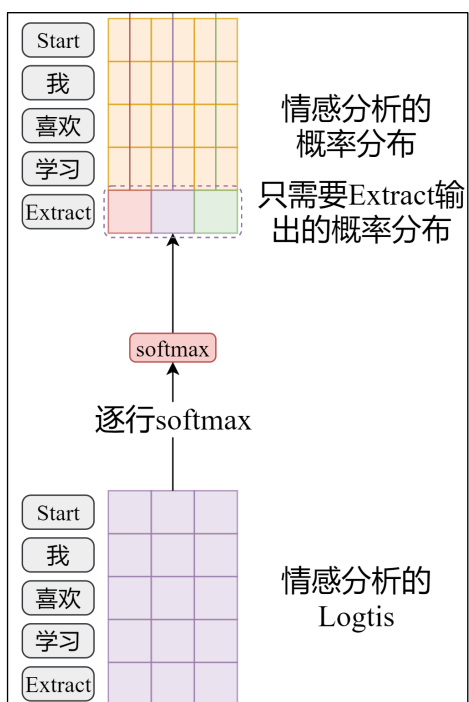
Extract 没有目标，计算损失时通过掩码阻断，反向传播的更新也不会考虑其输出。

2.2.4.8 特定任务线性层

1) h_l^m 经过 W_y 矩阵处理得到情感分析的 Logits

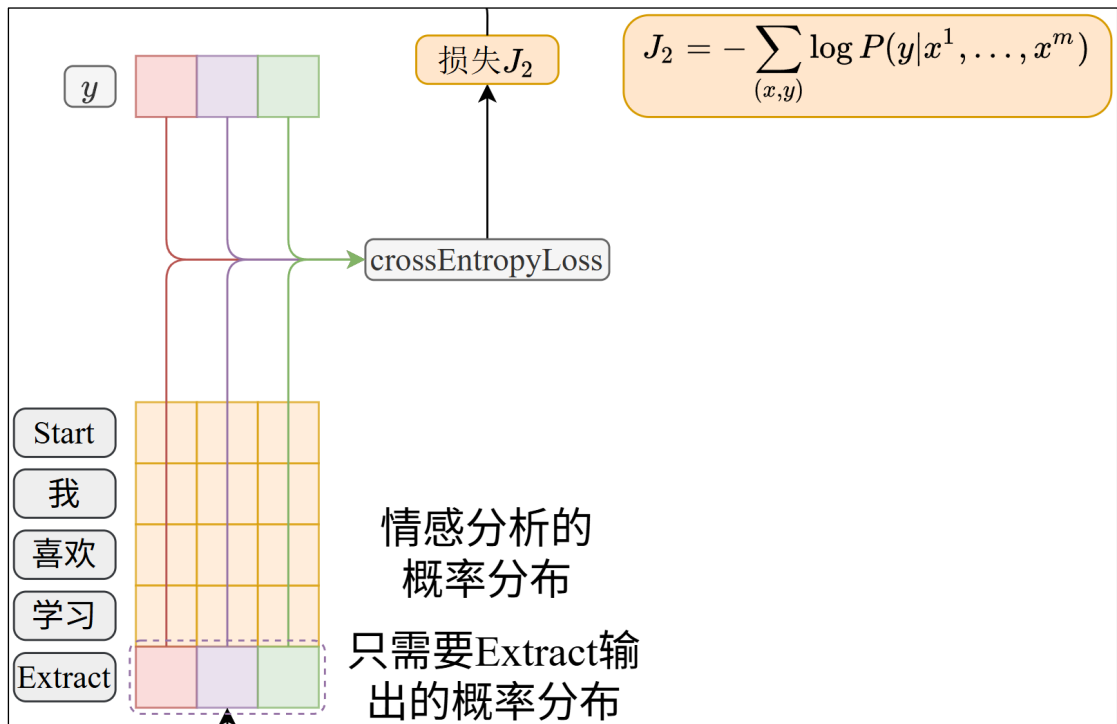


2) 逐行 softmax 计算概率分布



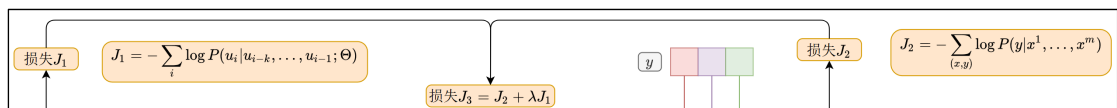
Extract 对应的输出向量包含了序列的所有信息，只有它是我们需要的概率分布，是模型判定的情感类别。

3) 交叉熵计算损失



上式中， y 标注数据中的目标，为独热编码形式，实际通常是索引，基于索引交叉熵的计算和上述过程在数学上是等价的。

4) 最终的损失函数 J_3



2.2.4.9 完整架构图



GPT-1 架构-细节-情感分析任务架构图.svg

2.2.5 参数量估算

偏置和监督微调阶段额外的线性层权重可以忽略不计，此处只计算预训练阶段的参数量。

GPT-1 的权重分为以下几个部分

2.2.5.1 嵌入矩阵和位置编码矩阵

1) 嵌入矩阵

参数量为 $\text{vocab_size} * d_{\text{model}}$

2) 可学习的位置编码矩阵

参数量为 $\text{content_length} * d_{\text{model}}$

2.2.5.2 Decoder Block

先计算单个 Decoder Block 的参数量。

1) 因果自注意力

(1) 首先计算单个头的参数量

三个投影矩阵的形状均为 $d_{model} * d_k$

所以一个注意力头的参数量为 $3 * d_{model} * d_k$

(2) 十二个头的参数量

$$12 * 3 * d_{model} * d_k$$

(3) 多头合并后的投影矩阵

$$d_{model} * d_{model}$$

(4) 单个因果自注意力子层的参数量

十二个头的参数量+投影矩阵的参数量:

$$\begin{aligned} & 12 * 3 * d_{model} * d_k + d_{model} * d_{model} \\ &= 3 * d_{model} * (12 * d_k) + d_{model} * d_{model} \\ &= 3 * d_{model} * d_{model} + d_{model} * d_{model} \\ &= 4 * d_{model}^2 \end{aligned}$$

2) FFN

(1) 第一个线性层的参数量

$$4 * d_{model} * d_{model}$$

(2) 第二个线性层的参数量

$$4 * d_{model} * d_{model}$$

(3) FFN 的总参数量

$$8 * d_{model} * d_{model} = 8 * d_{model}^2$$

3) 单个 Decoder Block 的参数量

$$4 * d_{model}^2 + 8 * d_{model}^2 = 12 * d_{model}^2$$

4) 总的 Decoder Block 参数量

$$12 * 12 * d_{model}^2$$

2.2.5.3 总的参数量

$$\begin{aligned} & \text{vocab_size} * d_{model} + \text{content_length} * d_{model} + 12 * 12 * d_{model}^2 \\ &= 40478 * 768 + 512 * 768 + 144 * 768 * 768 \\ &= 31,087,104 + 393,216 + 84,934,656 \\ &= 116,414,976 \end{aligned}$$

约为 1.16 亿，官方声称 GPT-1 的参数量为 1.17 亿，细微的差异可能是来源于偏置项和监督微调阶段任务特定的线性层。

2.3 GPT-2

2.3.1 概述

1) BERT 和 GPT 的较量

GPT-1 和 BERT 代表了 Transformer 架构早期两个关键的分支。GPT-1 基于纯 Transformer 解码器架构（使用掩码自注意力），在许多 NLP 任务上表现优异。然而，其单向注意力机制无法同时利用 token 前后的完整上下文信息。

BERT 为了克服这一限制，采用了纯 Transformer 编码器架构（使用无掩码/双向自注意力），推出了BERT_{BASE}（参数规模与 GPT-1 相同）和BERT_{LARGE}（约 3.4 亿参数）两种模型。实验证明，得益于双向上下文表示，BERT 在众多需要深度上下文理解的任务上显著超越了 GPT-1。

面对 BERT 的成功，OpenAI 团队首先尝试在他们坚持的解码器架构路线上，通过增加参数量来提升 GPT-1 的性能（例如训练一个类似 BERT_{LARGE} 规模的 GPT 模型）。虽然更大的规模带来了性能提升，但相对于 BERT 架构革新带来的巨大飞跃，这种仅靠规模增长的提升显得相对有限，未能全面反超 BERT。

这促使 OpenAI 团队思考如何更充分地挖掘生成式预训练模型的潜力。他们并未改变核心架构（仍然是纯解码器），而是提出了一个更激进的目标：验证当模型规模和数据量足够大时，单自回归语言模型能否通过 Zero-Shot 方式直接执行多种任务，而无需特定任务的微调。为此，他们推出了 GPT-2。

作者在 GPT-2 论文第六章的末尾说道：“GPT-2 的训练数据与模型容量足以克服 BERT 所展示的单向表征效率不足问题”。

2) 主要成果及创新

（1）巨量扩展：模型参数（最大 15 亿）和训练数据（WebText）大幅增加。

（2）Zero-Shot 能力：重点研究并展示了模型在未经任何下游任务微调（Zero-shot）的情况下，直接执行多种任务的能力。

（3）架构延续：保持了与 GPT-1 相同的 Transformer 解码器架构。

GPT-2 的诞生并非因为架构创新（它沿用了 GPT-1 架构），而是 OpenAI 为了最大化展现其坚持的生成式预训练路线的潜力，在规模和训练数据上的一次大胆跃进，并以此作为主要“新意”。GPT-2 的成功证明了这一方向的可行性，为后续的 GPT-3 及更大型模型铺平了道路。

3) BERT 和 GPT 对 LLM 发展的意义

BERT 并非生成式大模型，其本质是一个基于 Transformer 编码器的深度双向语言表示模型，核心优势在于语言理解。而 GPT 则是基于解码器的生成式语言模型，二者代表了截然不同的技术路线。

当今主流的大语言模型（LLM）普遍沿用了 GPT 的纯解码器架构，均属于生成式语言模型。若非 GPT 核心团队在当时 BERT 巨大成功压力下的坚持与探索（如推出 GPT-2/GPT-3），LLM 的蓬勃发展很可能会来得更晚一些。

当然，BERT 绝非 LLM 发展的障碍。它革命性地证明了双向上下文建模对于深度语言理解的巨大价值，在需要精准语义理解的任务上实现了质的飞跃。它的成功促使 GPT 路线进行了更激进的探索（如大规模参数扩展、Zero-shot 能力验证）。二者相辅相成，共同确立并巩固了大规模预训练 Transformer 模型作为现代 NLP/AI 基石的地位。

4) 数据准备

(1) 构建新的网络爬取系统，从 Reddit 上所有获得至少 3 颗星的外链开始抓取。共计抓取了 **4500 万个** 链接。不包含 2017.12 之后创建的链接。

(2) 结合使用 Dragnet 和 NewsPaper 内容提取器从抓取的原始数据中提取有价值的文本，并移除了所有维基百科文档。因为后者是其他数据集的常见来源，可能导致训练集和测试集的重复，从而影响分析。

(3) 经过去重和基于启发式（基于经验）的清理后，得到的数据集称为 WebText，包含 800 多万个文档，总计 **40GB** 文本数据。

题外话：Reddit 是国外一个大型的社交新闻聚合、讨论平台，用户可以在不同的主题板块中发布内容、投票、讨论和交流，它被称为“互联网的首页”，许多热门新闻都在这里传播到其它社交媒体。Reddit 的投票系统（Upvote/Downvote）本质上是社区共识的体现，高质量、有趣或有用的内容通常能获得更多点赞，通过限制点赞数可以筛选出质量较高的内容。

5) 词表构建

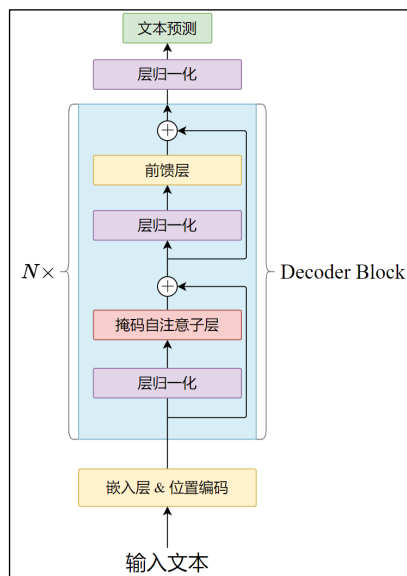
(1) 和 GPT-1 采用相同的 BPE 词表。

(2) 禁止 BPE 跨字符类别合并（即字模不能和标点符号合并），仅对空格例外处理。

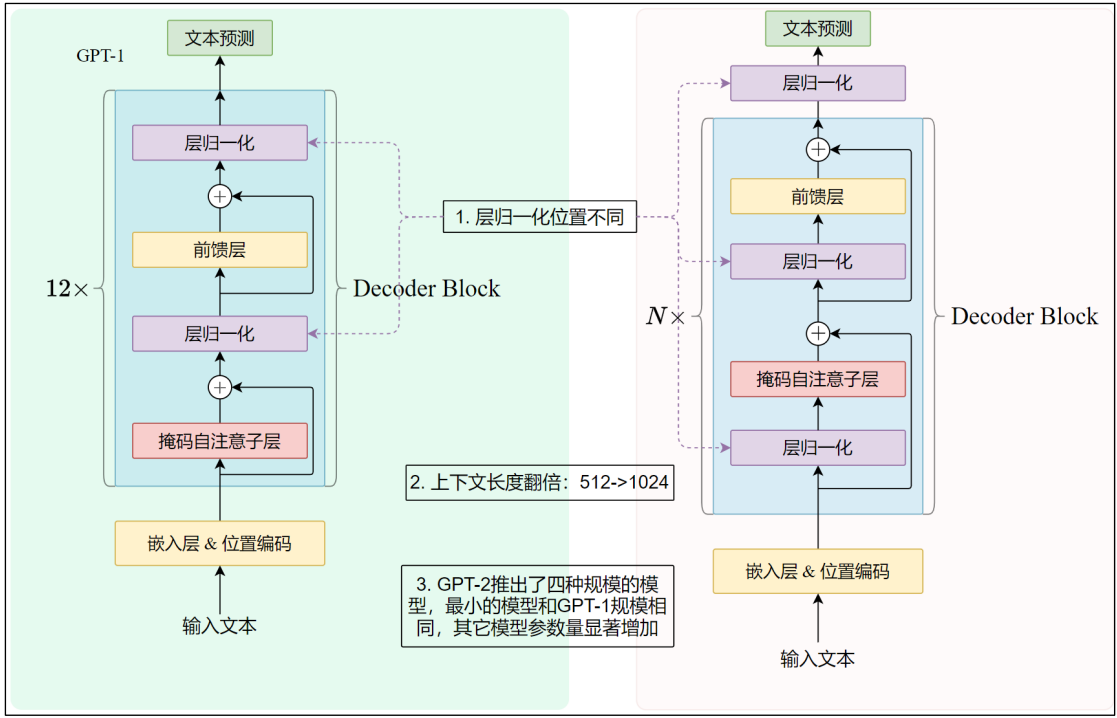
BPE 基于贪心启发式方法构建词汇表，研究团队观察到 BPE 会为常见词如 dog 生成多个版本：dog、dog!、dog?，浪费了词表槽位。因此做出了上述改进。

(3) 词表扩展至 50257 个 token。

2.3.2 模型架构



2.3.2.1 和 GPT-1 的差别

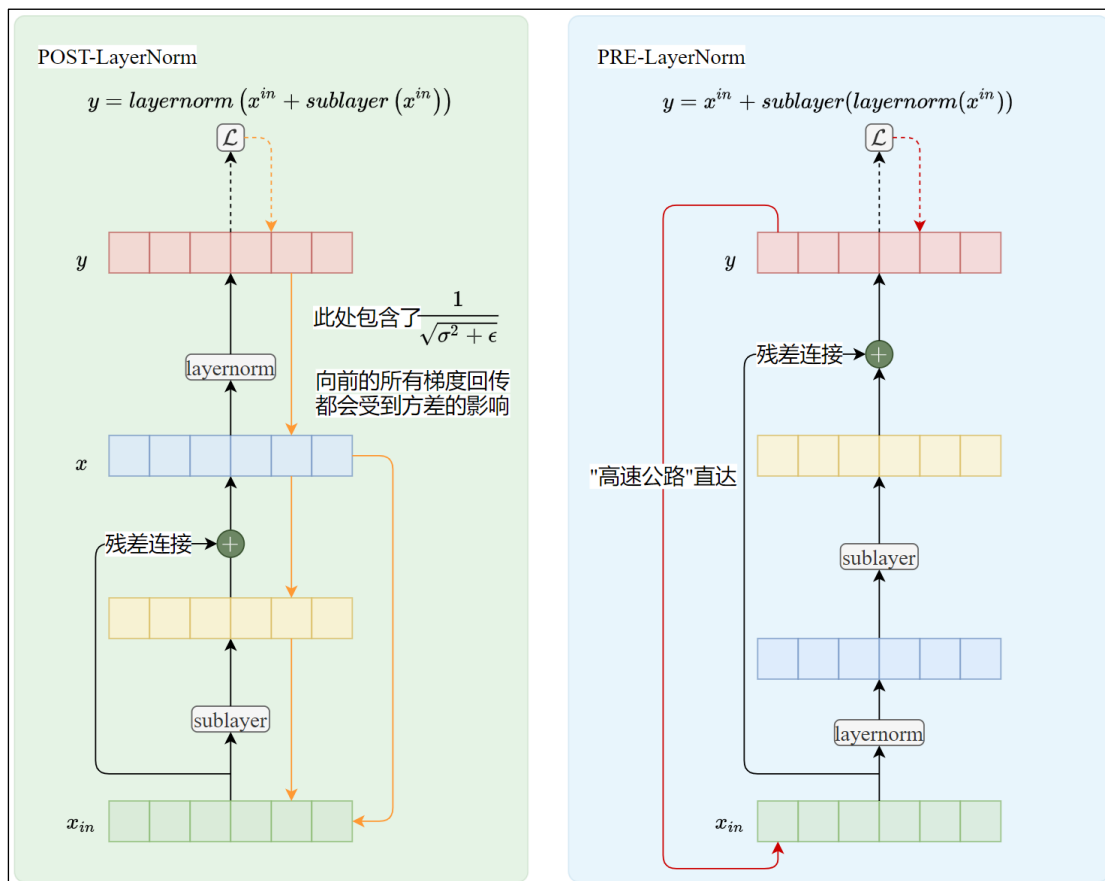


- GPT-2 模型基本沿用 GPT-1 的预训练架构，但做出以下修改
- (1) 将层归一化移至每个子块（因果自注意力层和 FFN 层）的输入之前，并在最终的隐藏层输出后添加额外的层归一化。
 - (2) 上下文长度由 512 个 token 增加到 1024 个 token。
 - (3) GPT-2 不再是单个模型，而是一个系列，共有四个不同参数量的模型，如下

参数量	层数	d_{model}
1.17 亿	12	768
3.45 亿	24	1024
7.62 亿	36	1280
15.42 亿	48	1600

2.3.2.2 Pre-LayerNorm vs Post-LayerNorm

在 GPT-1 和原版 Transformer 中使用的都是 Post-LayerNorm，即后归一化。GPT-2 转而使用 Pre-LayerNorm，这是为了解决训练深层 Transformer 模型时的梯度不稳定和收敛困难问题。



1) Post-LN 的问题

$$x = \text{layernorm}(x_{in} + \text{sublayer}(x_{in}))$$

在深层 Transformer 中（层数很多时），原始的 Post-LN 结构会导致梯度在反向传播时变得非常不稳定，尤其是在靠近输入侧的层。这是因为：

（1）我们知道，残差连接的目的是在深层 Transformer 中提供一条“高速公路”，使得接近输入侧的层不会因为层级过多产生的梯度消失或梯度爆炸而无法更新参数。

（2）LayerNorm 被放在残差连接之后，那么反向传播时，这条“高速公路”就要经过 LayerNorm，而它的操作会限制梯度的范围，尤其是在深层堆叠时，这种限制会被放大。

这导致了靠近输入的层接收到的梯度可能非常小（梯度消失）或者异常大（梯度爆炸），使得深层模型难以收敛。通常需要谨慎的学习率预热策略（训练初期逐步提高学习率）和极其精细的超参数调优来防止发散。

2) Pre-LN 的解决方案

$$x = x_{in} + \text{sublayer}(\text{layernorm}(x_{in}))$$

将归一化移至传入子层之前。

梯度反向传播时，首先通过残差连接近乎无损地传递到上一层的输出。残差连接提供的“高速公路”不必再通过 LayerNorm 或其它步骤，可以将梯度直接、不受阻碍地流向更早的层。

3) Pre-LN 的效果

实践证明，Pre-LN 显著改善了深层训练模型的稳定性，梯度在整个网络中流动得更加顺畅和平稳，降低了训练难度，使得训练非常深的模型（如 GPT-2、GPT-3）成为可能。

由于梯度更稳定，信息流动更高效，使用 Pre-LN 的模型通常在相同训练步数下能达到比 Post-LN 模型更低的损失或更高的性能。或者说，要达到相同的性能水平，Pre-LN 模型需要的训练步数更少。

Pre-LN 结构降低了对学习率预热策略的依赖，并且对学习率大小的选择通常比 Post-LN 更宽容。

4) Pre-LN 的潜在缺点

一些研究（如《On Layer Normalization in the Transformer Architecture》）指出，Pre-LN 改变了每一层输入信号的分布。在 Post-LN 中，LayerNorm 保证了输入到下一子层的信号具有稳定的均值和方差（这是 LayerNorm 的设计初衷）。而在 Pre-LN 中，输入到子层的信号是归一化后的，但输出到下一层整个块的信号（即经过 Add 之后的信号）没有被立即归一化。理论上，这可能导致信号在深层传播时分布发生偏移。

然而，实践压倒性地证明，Pre-LN 带来的训练稳定性和收敛速度的提升，远远超过了这个潜在的理论劣势。对于现代大型模型，训练可行性是第一位的。

2.3.2.3 训练数据的输入和目标构造

和 [GPT-1 预训练阶段](#) 完全相同，不再赘述。

2.3.2.4 下游任务的输入构造

对于所有的下游任务，将上下文和问题全部处理为连续的 token 序列，然后输入模型。将所有任务都转换为了预测下一个词的任务。

2.4 GPT-3

2.4.1 概述

2.4.1.1 主要成果及创新

1) GPT-3 的由来

近年来，基于 Transformer 模型的“预训练+微调”范式在众多 NLP 任务上取得了显著进展。然而，要在特定目标任务上获得优异性能，通常需要大量标注数据进行微调，这不仅成本高昂，也限制了模型的普适性。此外，在微调过程中，模型暴露于相对窄域（或特定领域）的数据分布，可能导致其在预训练阶段从海量语料中习得的泛化能力下降。相比之下，人类学习多数语言任务并不需要大量监督数据，通常仅需简单指令或少量示例即可掌握。

为了提升模型的普适性，GPT-2 尝试直接利用无监督预训练模型执行下游任务，取得了令人鼓舞的效果。

OpenAI 团队观察到，Transformer 语言模型参数规模的每次扩大都伴随着下游 NLP 任务性能的提升。更重要的是，（如 Scaling Laws for Neural Language Models [Kaplan et al., 2020] 所示）与多项下游任务性能相关的损失函数，随着模型规模扩大呈现出平滑的改进趋势。这表明更大规模的模型可能具备更强的能力。因此，他们在 GPT-2 的基础上，将模型参数规模扩大了两个数量级（约百倍），由此诞生了 GPT-3。

2) 前所未有的规模

（1）拥有 1750 亿个参数，在其发布时是世界上最大的公开语言模型，比前身 GPT-2（15 亿参数）大两个数量级。

(2) 训练数据集极其庞大且多样化，包含了 Common Crawl、WebText2、书籍、维基百科等多种来源的文本，数据量高达近 5000 亿 token。

(3) 支持更长的上下文窗口（2048 个 token），能处理和理解更长的文本片段。

3) 强大的少样本/零样本学习能力

(1) 它通过简单的提示工程（Prompt Engineering），在不更新模型参数（即不进行微调）的情况下，就能在众多 NLP 任务上取得非常好的效果。

(2) 模型能够理解并执行仅通过几个示例（Few-Shot Learning）甚至仅通过任务描述（Zero-Shot Learning）定义的新任务，展现了极强的泛化能力和上下文理解能力。

(3) 涵盖的任务范围极广，包括但不限于：机器翻译、问答、文本摘要、完形填空、代码生成、简单推理、角色扮演对话等。

4) 展示了语言模型的通用潜力

GPT-3 的成功有力地证明了通过大规模无监督预训练，结合提示工程，单一模型可以成为一个通用的“任务无关”系统。这挑战了传统“一个任务一个特定模型”的范式，为通用人工智能（AGI）的探索提供了重要实证。

2.4.1.2 数据准备

数据集由两部分构成

(1) Common Crawl 数据集

基于 2016 至 2019 年的 41 个 Common Crawl 月度分片构建，通过以下方式清洗。

- 基于与高质量参考语料库的相似度过滤
- 在文档级别实施跨数据集模糊去重

清洗前压缩纯文本达 45TB，过滤后为 570GB，大致相当于 4000 亿（BPE）token。

(2) 在训练数据中混入已知高质量参考语料

包括基于 GPT-2 使用的数据集 WebText 的扩展 WebText2，两个基于互联网的书籍语料库（Books1 和 Books2）以及英文维基百科。

最终数据集构成如下

数据集	Token 数量	训练混合权重
过滤后的 Common Crawl	4100 亿	60%
WebText2	190 亿	22%
Books1	120 亿	8%
Books2	550 亿	8%
维基百科	30 亿	3%

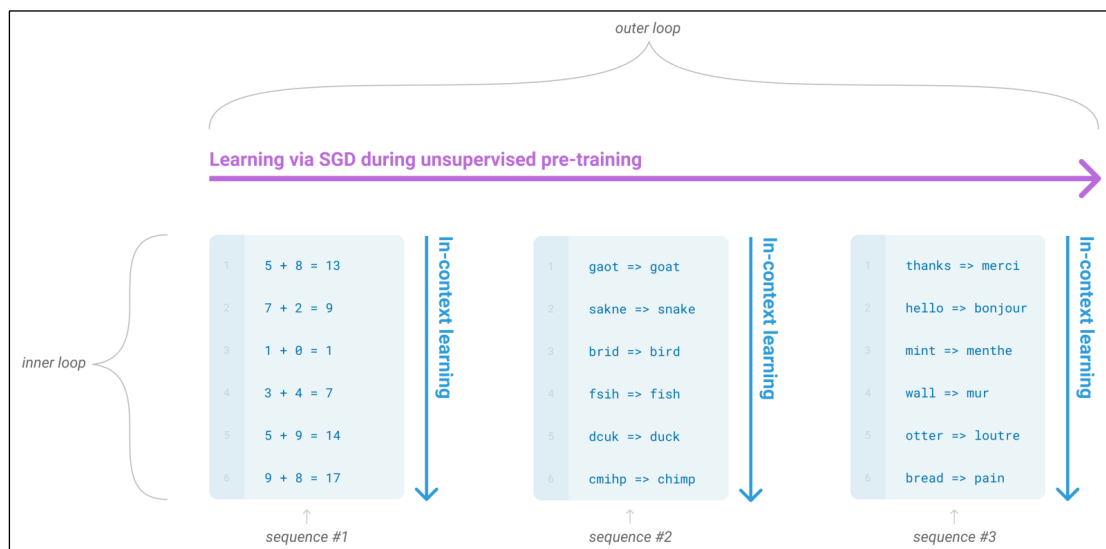
2.4.1.3 GTP-3 的元学习

1) 定义

元学习（Meta-Learning），又称“学会学习”（Learning to Learn），是机器学习的一个分支，旨在让模型具备快速适应新任务的能力。其核心思想是通过从多个任务中学习共享的经验或模式，从而在面对新任务时，只需少量样本或调整即可高效解决。

2) GPT3 的元学习

GPT-3 论文通过下图介绍元学习



（1）outer loop

从左到右的紫色箭头是**外循环**，表示模型在无监督预训练时通过 SGD 学习的过程，在这一阶段，模型培养出广泛的技能和模式识别能力。

（2）inner loop

自上而下的三个蓝色箭头表示**内循环**。是指模型在推理阶段利用预训练阶段学习到的能力快速适应或识别目标任务，这一过程被称为“**in-context learning**”，即**上下文学习**。具体来说，研究人员将下游任务转换为自然语言描述，再提供一些示例，作为上下文输入模型，后者从这些上下文中快速识别目标任务，预测后续内容即可完成任务实例。

3) zero-shot、one-shot 和 few-shot

为了评估 GPT-3 的上下文学习能力，研究团队在超过 24 个 NLP 数据集及多个新颖任务上评估 GPT-3，这些任务专门用于测试模型对训练集中未直接包含任务的快速适应能力。

每个任务通过三种方式评估：

➤ **few-shot**（少样本学习）：在模型上下文窗口内提供尽可能多的示例（受限于上下文窗口，通常为 10-100 个）

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French:
2 sea otter => loutre de mer
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => .....
```

➤ one-shot（单样本学习）：提供一个示例

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French:
2 sea otter => loutre de mer
3 cheese => .....
```

➤ zero-shot（零样本学习）：不提供示例，仅根据自然语言描述预测答案。

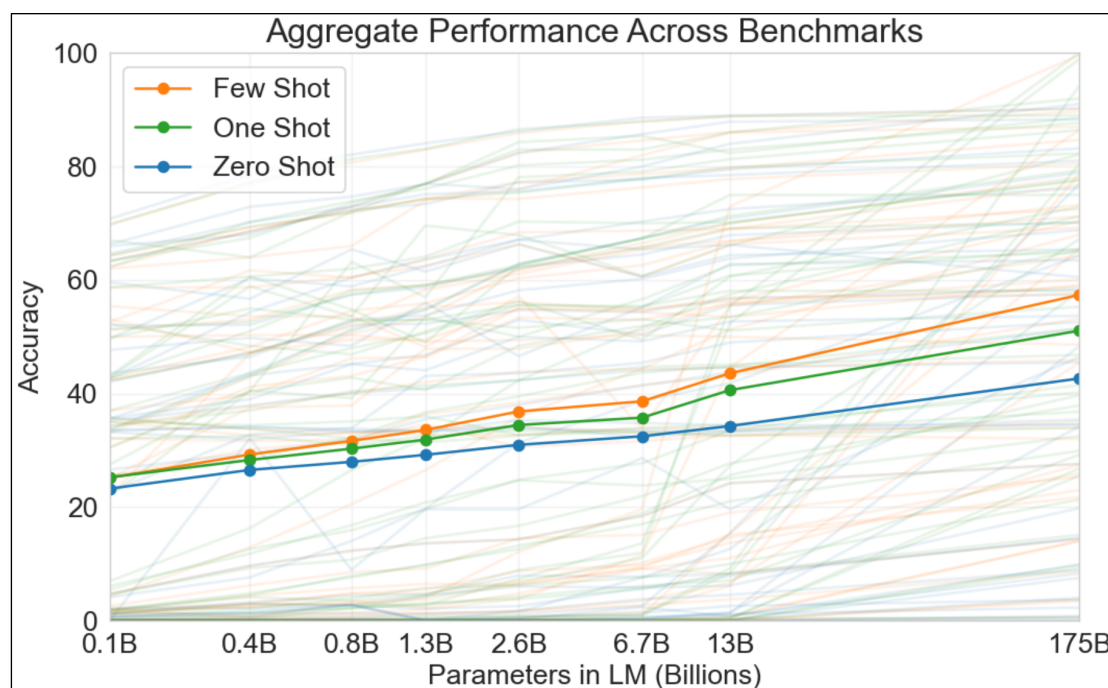
Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French:
2 cheese => .....
```

需要注意的是，GPT-3 的 shot 是指推理阶段为下游任务准备的示例，它的 zero-shot 是指不提供示例。而 GPT-2 的 zero-shot 强调的是不进行监督微调，二者是不一样的。

4) 模型在 42 个基准测试上的平均表现



从上图可以得到以下结论：

- (1) 沿横轴，随着模型参数规模的增长，平均准确率稳步提升
- (2) 对比不同颜色的曲线，对于大多数模型而言，Few-Shot 性能优于 One-Shot，由于 Zero-Shot

5) Prompt

GPT-3 对于 Prompt 定义非常混乱，前后矛盾。

- (1) 广义

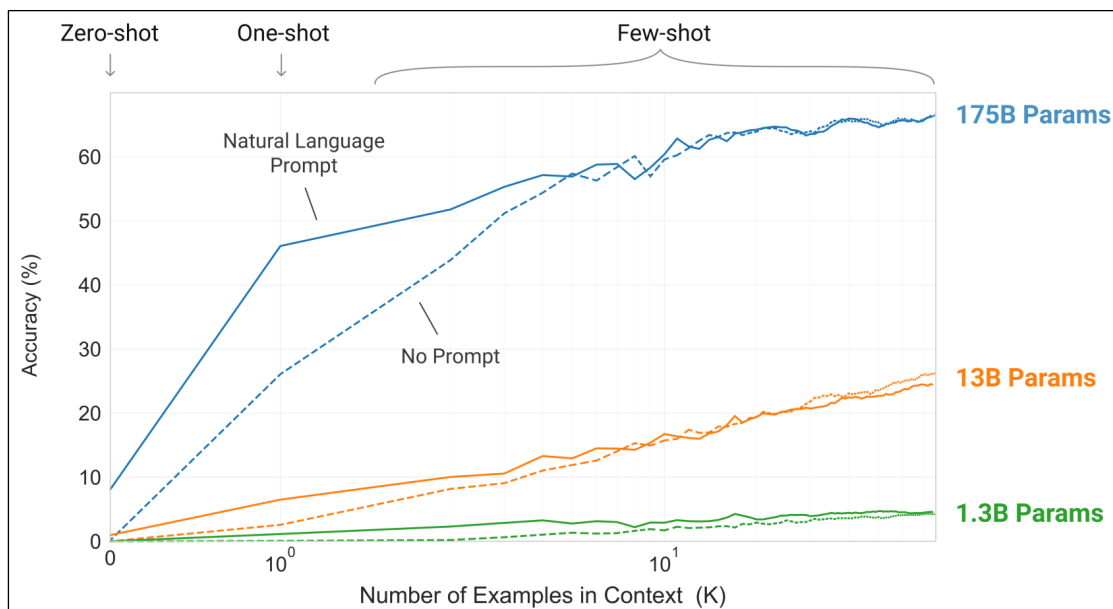
泛指提供给模型的一切输入，包括任务描述、示例和最终需要模型完成的部分。

- (2) 狭义

即 3) 中所示，指的是需要模型回答的问题或补全的内容。

6) 一个简单任务上的上下文学习性能

任务要求模型从单词中移除随机符号。



论文中给出了这样一张图。我们可以从三个角度获取信息。

- ① 沿横轴，随着示例的增加，三种规模的模型准确性都在增加。
- ② 横向对比不同颜色的曲线，可以看到随着参数规模的增加，准确性显著提升。
- ③ 对比同色的实现和虚线，可以发现参数规模不变的前提下，增加了自然语言提示，在零样本和少样本的情况下，准确率提升较为明显。

那么，上图中的 Prompt 指的 Natrual Language Prompt，结合上下文分析，是指除了问题和示例之外的自然语言描述。这里的 Prompt 显然既不是狭义的，也不是广义的，所以我们认为 **GPT-3 对于 Prompt 的定义非常混乱，前后没有统一。**

可以看到尤其是对于最大的模型，示例足够多时，自然语言描述已变得可有可无了。

2.4.1.4 词表构建

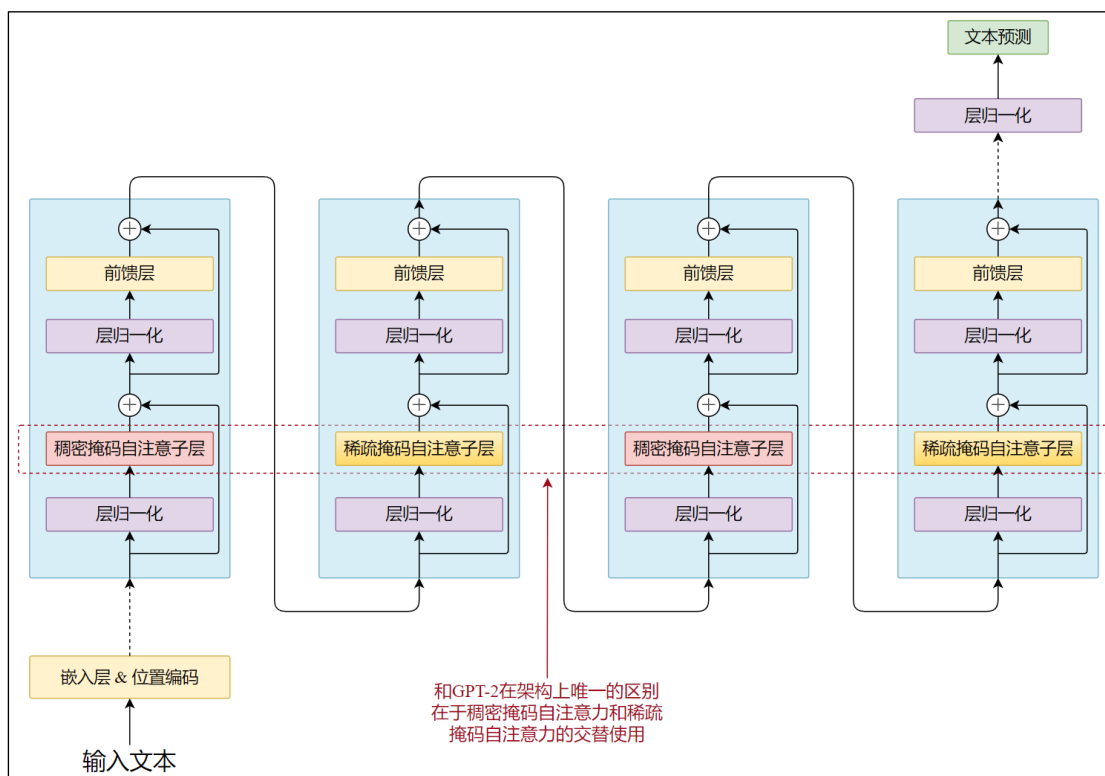
与 GPT-2 完全相同。

2.4.2 模型架构

2.4.2.1 和 GPT-2 的差别

(1) **架构上的差异**：和 GPT-2 的唯一区别在于各个 Transformer Block 中交替使用稠密注意力（传统的注意力机制）和局部带状稀疏注意力模式，大幅降低了时间复杂度和空间复杂度，使得模型上下文可以更大。

GPT-3 架构如下，



(2) 参数规模显著增加

GPT-3 是一系列模型的统称，最小模型参数量为 1.25 亿，最大模型参数量为 1750 亿，当提到 GPT-3 而不加后缀时，通常指的是最大的模型。

模型名称	$n_{\{params\}}$	$n_{\{layers\}}$	$d_{\{model\}}$	$n_{\{heads\}}$	$d_{\{head\}}$
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 XL	1.3B	24	2048	24	128
GPT-3 2.7B	2.7B	32	2560	32	80
GPT-3 6.7B	6.7B	32	4096	32	128
GPT-3 13B	13.0B	40	5140	40	128
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128

所有模型的 FFN 中间层维度均为 d_{model} 的四倍

所有模型的上下文长度均为 2048 个 token。

2.4.3 稀疏自注意力机制

2.4.3.1 稠密因果自注意力机制和上下文长度的关系

传统的因果自注意力机制中，每个 token 需要和之前的所有 token 计算注意力，时间复杂度和空间复杂度都很高。

1) 时间复杂度

用 N 表示序列长度（即上下文长度）

（1）计算 Q/K/V 投影

输入矩阵 $X \in R^{N \times d}$ 通过三个线性层（权重 $W_Q, W_K, W_V \in R^{d \times d}$ ）生成 Q, K, V 。

复杂度： $O(3 \times N \times d \times d) = O(3Nd^2)$ 。

（2）计算注意力分数

QK^T 得到注意力分数矩阵 $A \in R^{N \times N}$ 。

复杂度： $O(N \times N \times d) = O(N^2d)$

（3）因果掩码

对角线以上的部分置为极小值，复杂度 $O(N^2)$

（4）Softmax 计算

对 A 逐行 softmax（指数、求和、归一化），复杂度 $O(N^2)$ （逐元素计算）

（5）加权求和

用 softmax 输出 P 乘以 V 得到输出 O

复杂度： $N \times N \times d = N^2d$

（6）输出投影

O 经过线性层（权重 $W_O \in R^{d \times d}$ ）生成最终输出

复杂度： $N \times d \times d = Nd^2$

（7）总时间复杂度

$$O(N^2d + Nd^2)$$

$N \gg d$ 时， N^2d 主导（序列长度敏感）

$N \ll d$ 时， Nd^2 主导（特征维度敏感）

2) 空间复杂度

（1）参数存储

投影矩阵 W_Q, W_K, W_V, W_O 共 $4 \times d \times d = 4d^2$ 个参数，复杂度 $O(d^2)$

（2）中间激活值（训练时需要缓存用于计算梯度）

Q, K, V ：三个矩阵，共 $3 \times N \times d$

注意力分数矩阵 A： $N \times N$

Softmax 输出 P： $N \times N$

加权输出 O： $N \times d$

（3）总空间复杂度

$$O(N^2 + Nd + d^2)$$

$N \gg d$ 时， N^2d 主导（序列长度敏感）

$N \ll d$ 时， Nd^2 主导（特征维度敏感）

3) 结论

时间复杂度和空间复杂度都随序列长度二次增长，当上下文长度较大时，注意力计算会成为瓶颈。

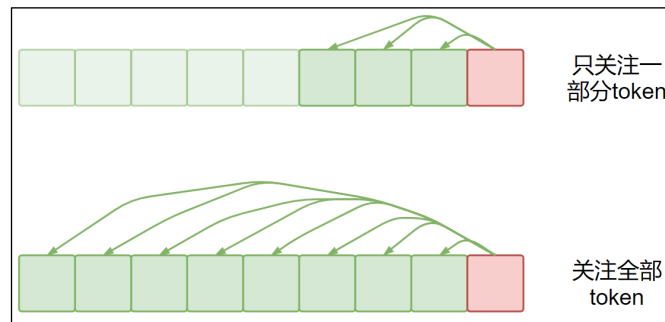
传统的因果自注意力机制下，每个 token 都可以关注到之前的所有 token，因此也叫**稠密因果自注意力机制**。限制了模型上下文长度的增长。

2.4.3.2 稀疏因果自注意力机制

GPT-3 使用了稀疏因果自注意力机制，这种方法是 OpenAI 的研究团队在《Generating Long Sequences with Sparse Transformers》中提出的。

1) 核心思想

稀疏注意力的核心思想是让每个 token 只关注之前的一部分 token。



2) 分解式自注意力机制

这是 OpenAI 提出的稀疏自注意力机制的具体实现。

(1) 实现形式

自注意力层将输入 X 转换为输出，接收额外的参数 S ， $S = \{S_1, S_2, \dots, S_n\}$ ，其中 S_i 表示第 i 个查询向量所关注的输入向量索引集合，公式如下。

$$\text{Attend}(X, S) = (a(x_i, S_i))$$
$$a(x_i, S_i) = \text{softmax}\left(\frac{(W_q x_i) K_{S_i}^T}{\sqrt{d}}\right) V_{S_i}$$
$$K_{S_i} = (W_k x_j)_{j \in S_i} \quad V_{S_i} = (W_v x_j)_{j \in S_i}$$

对于稠密自注意力， $S_i = \{j: j \leq i\}$ ，即允许每个 token 关注其之前所有的位置。

(2) S_i 的选择

OpenAI 提出的因子分解自注意力机制采用 p 个独立的注意力头，其中第 m 个头定义了一个索引子集 $A_i^{(m)} \subset \{j: j \leq i\}$ ，并让 $S_i = A_i^{(m)}$ ，其中 $|A_i^{(m)}| \propto \sqrt[p]{N}$ 。需要关注的 token 数量从 N 变成了 $\sqrt[p]{N}$ ，时间复杂度中的 $O(N^2)$ 项也变为 $O(N \sqrt[p]{N})$ 。

(3) A 的选择

$\forall j \leq i$ ， A 的设置要保证 i 能够通过最长路径为 $p+1$ 的位置路径关注到 j 。具体来说，如果 (j, a, b, i) 是索引路径，那么 $j \in A_a^{(1)}$ 、 $a \in A_b^{(2)}$ 、 $b \in A_i^{(3)}$ 。简而言之， A 的选择要求每个 token 经过至多 $p+1$ 次跳转即可关注到之前的所有 token。

这里比较抽象，可以参考下文的具体实现“二维分解注意力”辅助理解。

(4) 总结

因子自注意力机制通过限制每个 token 可以关注的序列范围（直接关注），降低了时间复杂度，还要通过合理的 A 保证至多经过 $p+1$ 次跳转即可关注到之前的所有 token（间接关注）。

从而在保持 Transformer 将信号从任意输入位置传播到任意输出位置能力的同时，大幅降低时间复杂度。

3) 二维分解注意力

这是因子分解注意力在 $p = 2$ 时的实现方式。

论文提供了两种方式：

(1) 步长注意力

(2) 固定注意力

4) 方式一：步长注意力

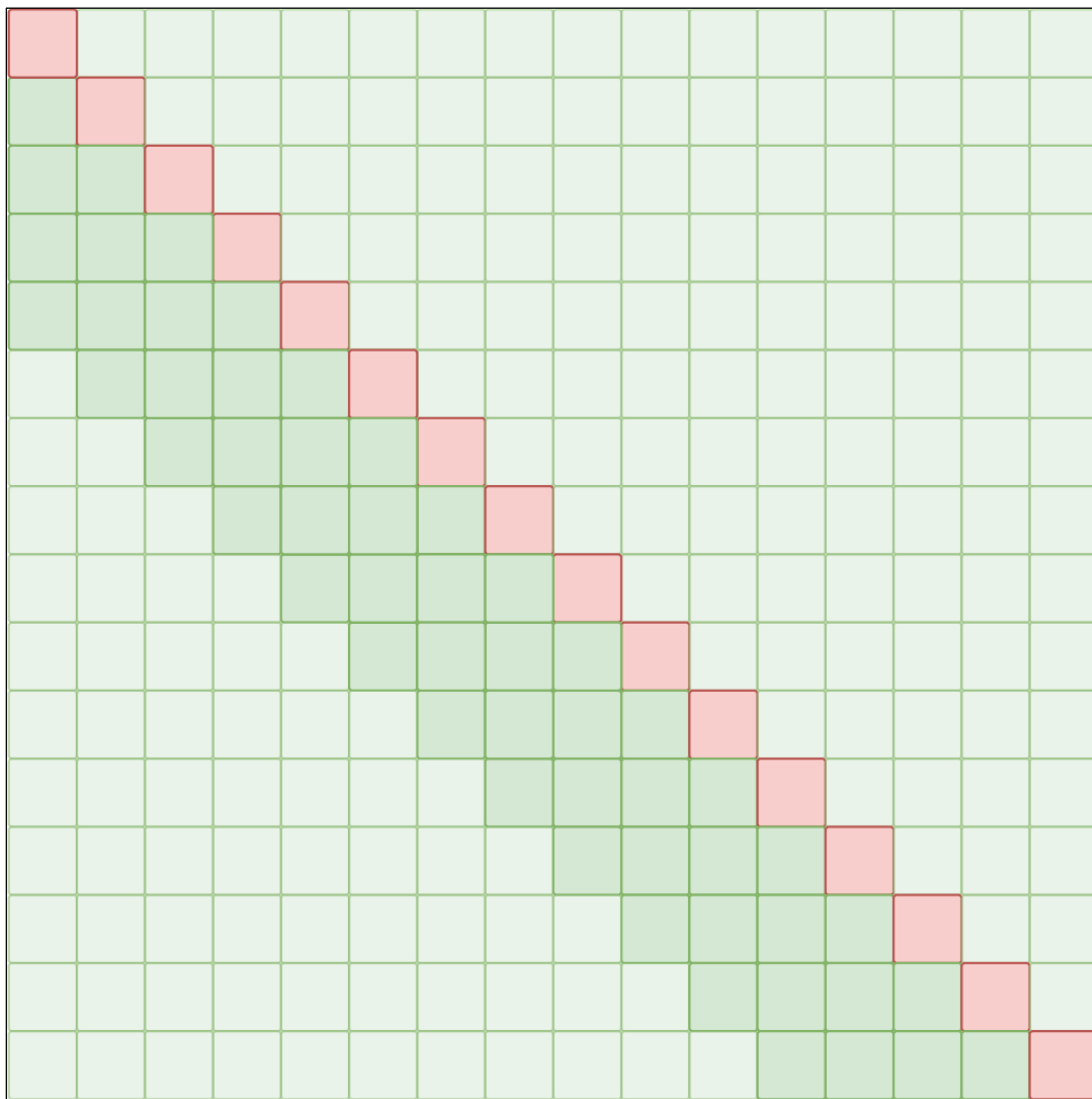
这种方式由两种注意力构成。

(1) 局部注意力：第一个注意力头关注前 l 个位置。

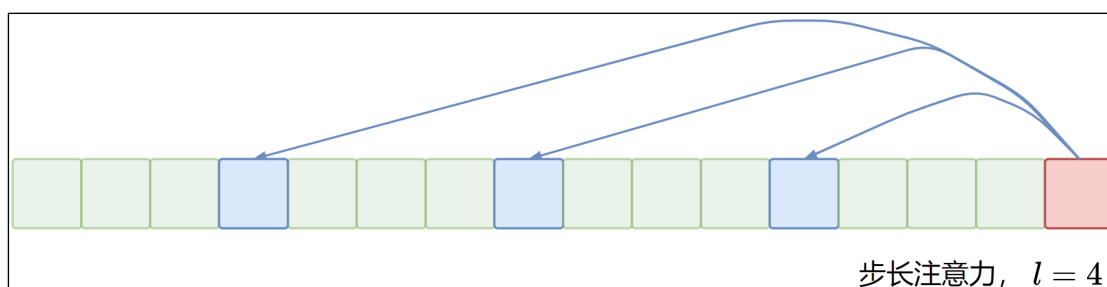
$$A_i^{(1)} = \{t, t + 1, \dots, i\}, \text{ 其中, } t = \max(0, i - l)$$



相应的掩码矩阵如下，其中深色的部分可以关注的位置，浅色表示不能关注的位置。第 i 行表示第 i 个 token，和因果自注意力的掩码矩阵表示法相同。

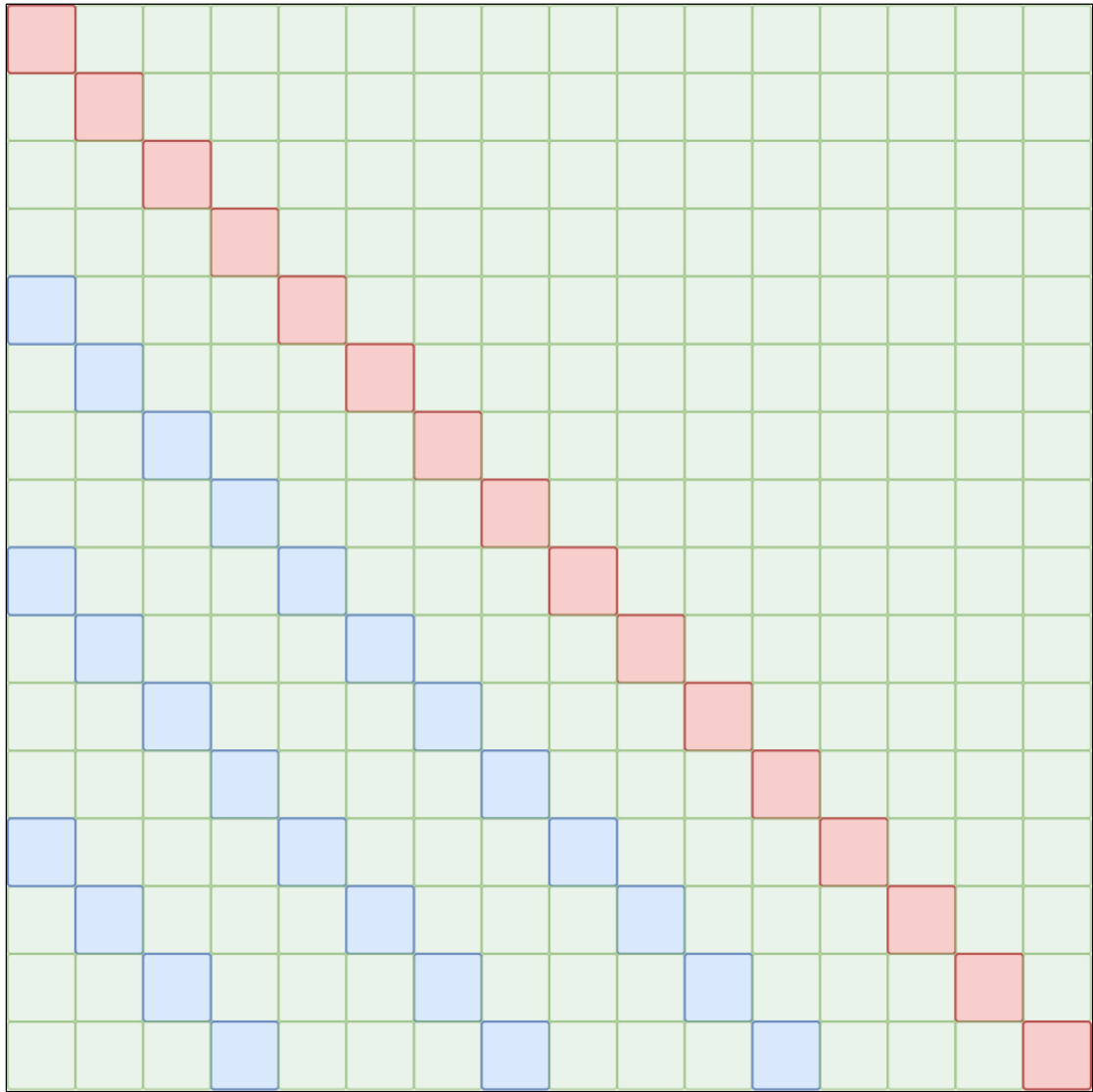


(2) 步长注意力：第二个注意力头关注每隔 l 个位置。



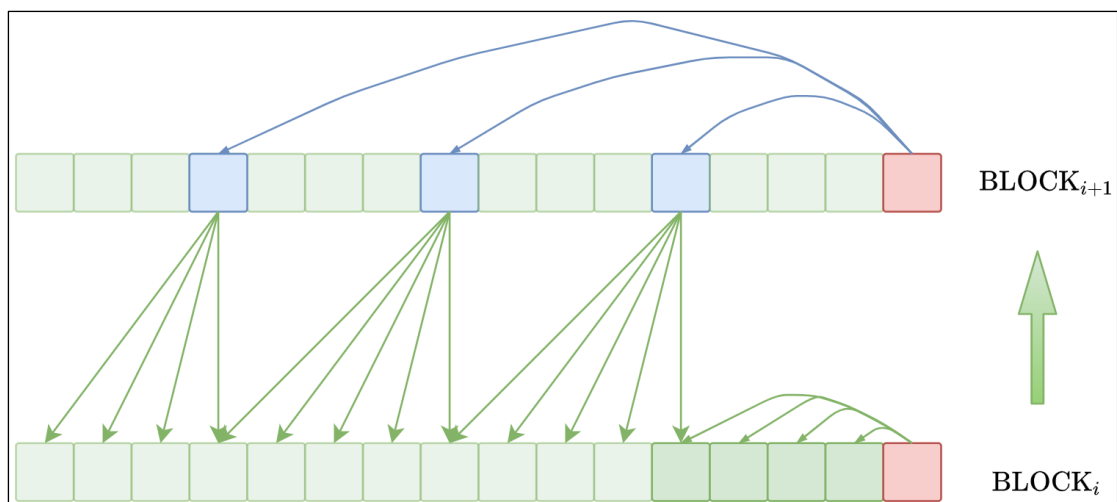
$$A_i^{(2)} = \{j: (i - j) \bmod l = 0\}$$

掩码矩阵如下



5) 如何确保经过 $p+1$ 步可以关注到所有 token?

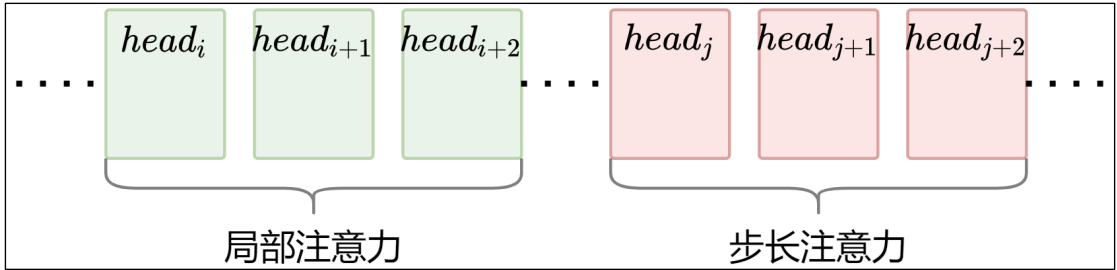
每个 token 都可以经过至多 $p+1$ 步关注到之前的所有 token，即确保每个 token 都有全局感受野（感受野是指元素可以依赖的所有元素构成的集合，即逻辑上可以“看到”的 token 的范围），是通过空间上 Transformer 的多层堆叠实现的。



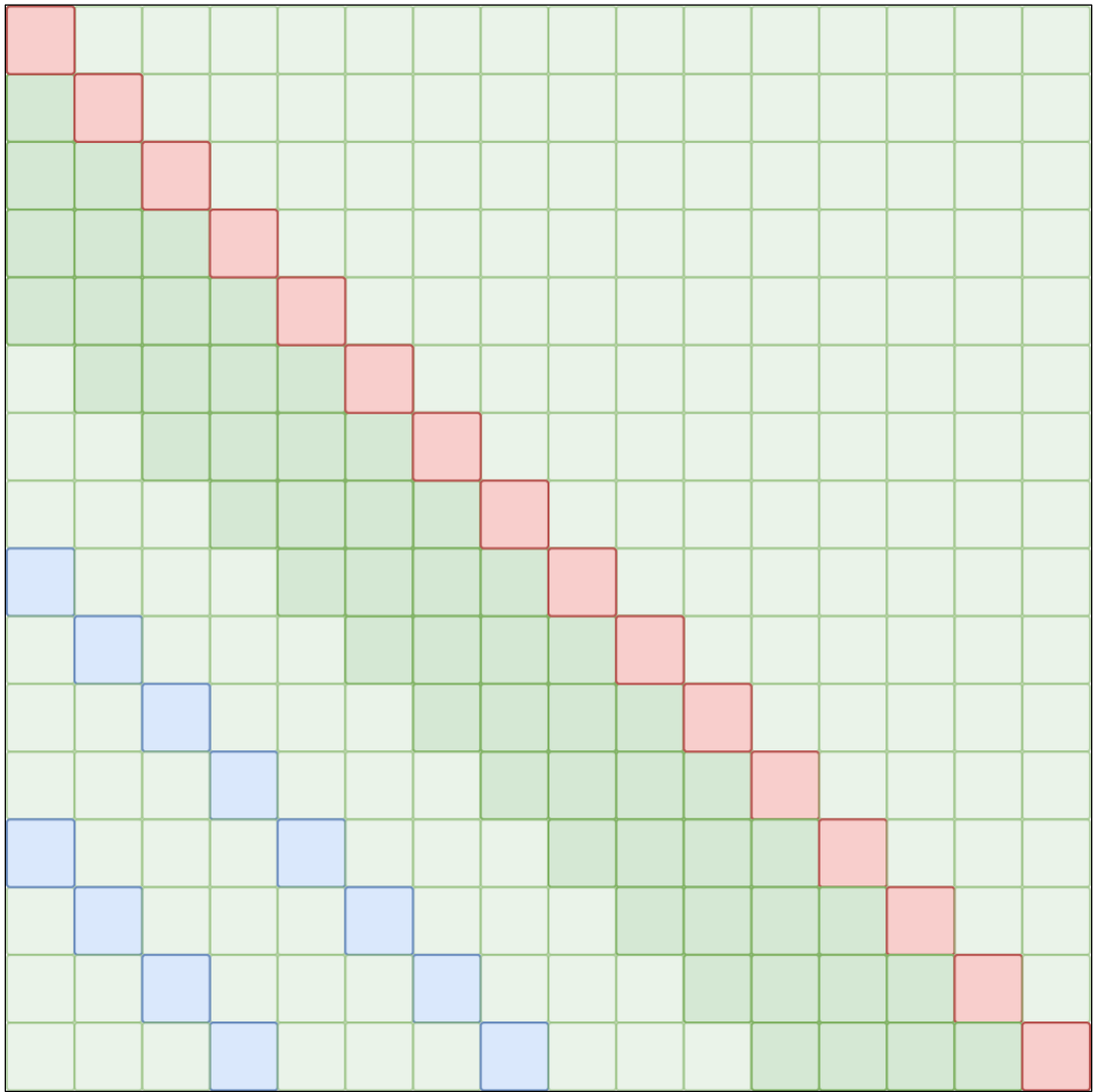
由上图我们可以看到，全局感受野的实现是通过 BLOCK 层间跳转实现的，这是一个树结构，跳转的步数就是层高（ $p+1$ ），而最终的全局感受野是由最底层元素构成的（ N ）。而已知最底层元素和层高，求第一层的元素，公式就是 $\sqrt[p]{N}$ ，因此每个 token 需要直接关注的元素就由 N 缩减为了 $\sqrt[p]{N}$ 。

6) 多个注意力头的协作方式

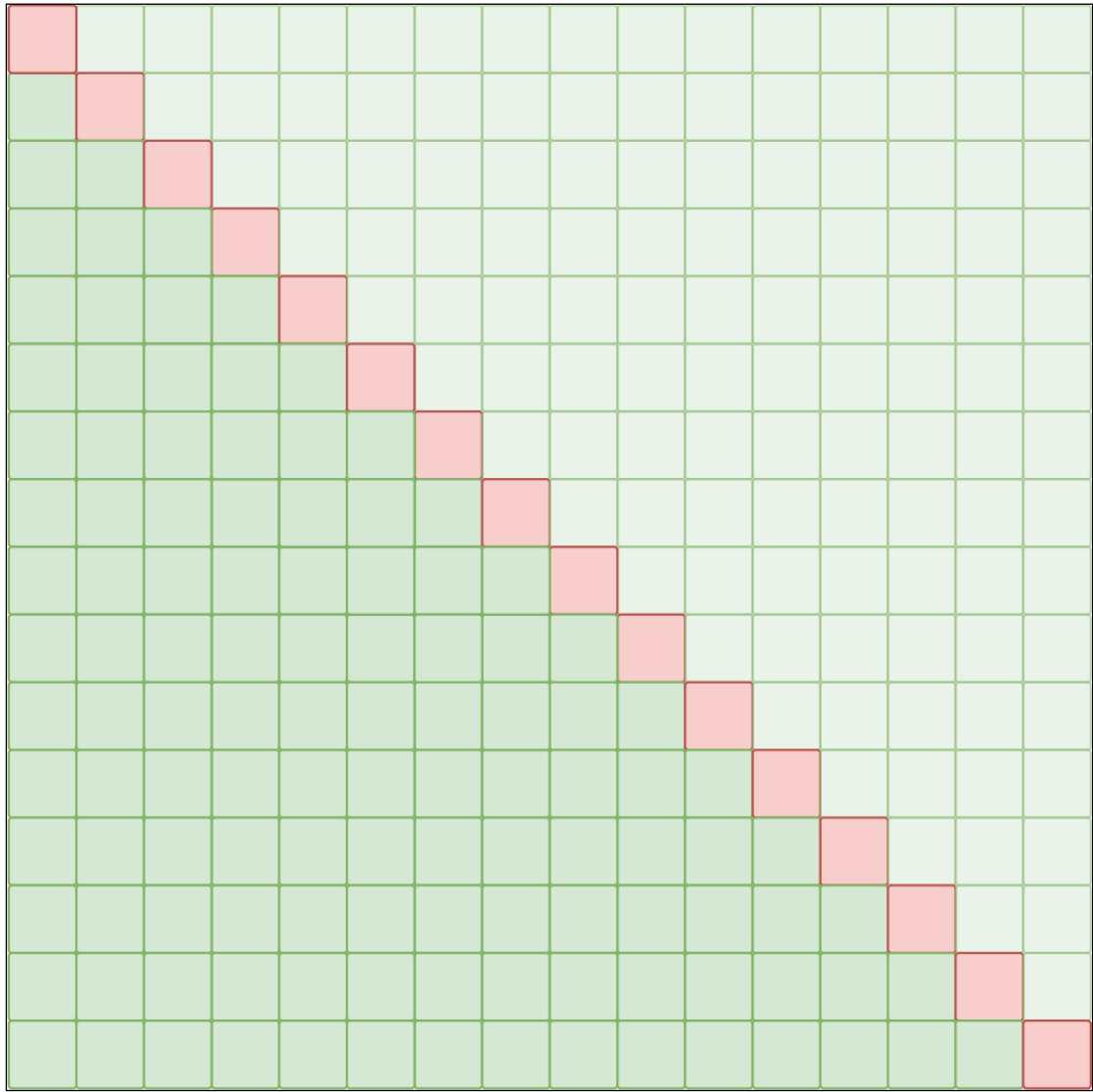
通常一部分注意力头采用局部注意力、另一部分注意力头采用步长注意力。



多个注意力头共同作用，如果把方式一的两种稀疏注意力方式整合在一起，掩码矩阵是这样的。

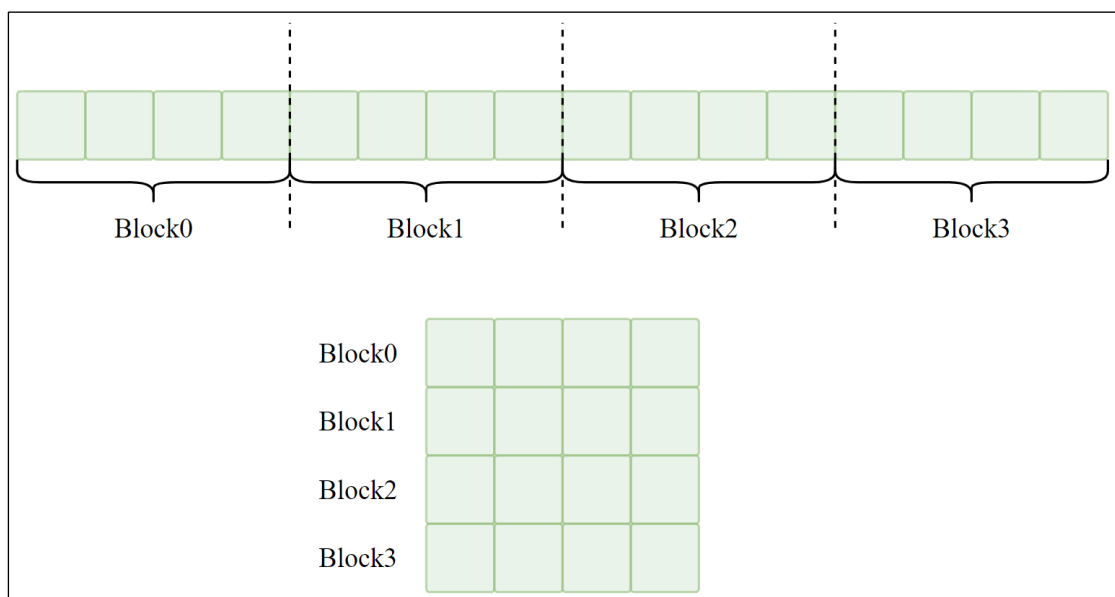


相比之下，稠密因果自注意力的掩码矩阵是这样的



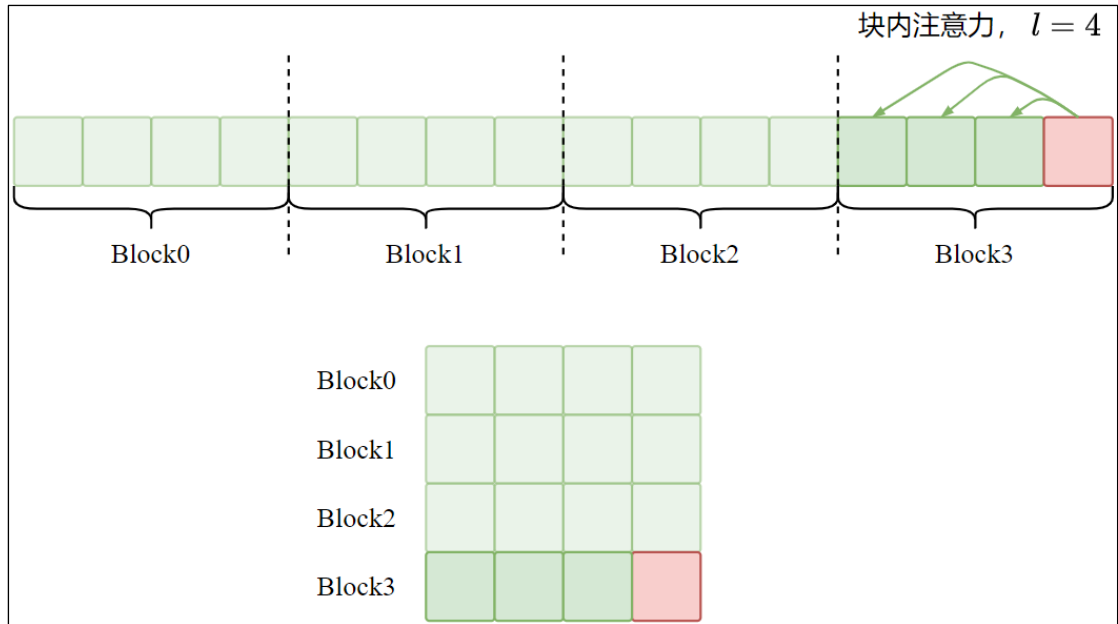
7) 方式二：固定注意力

(1) 这种方式首先要将序列分块

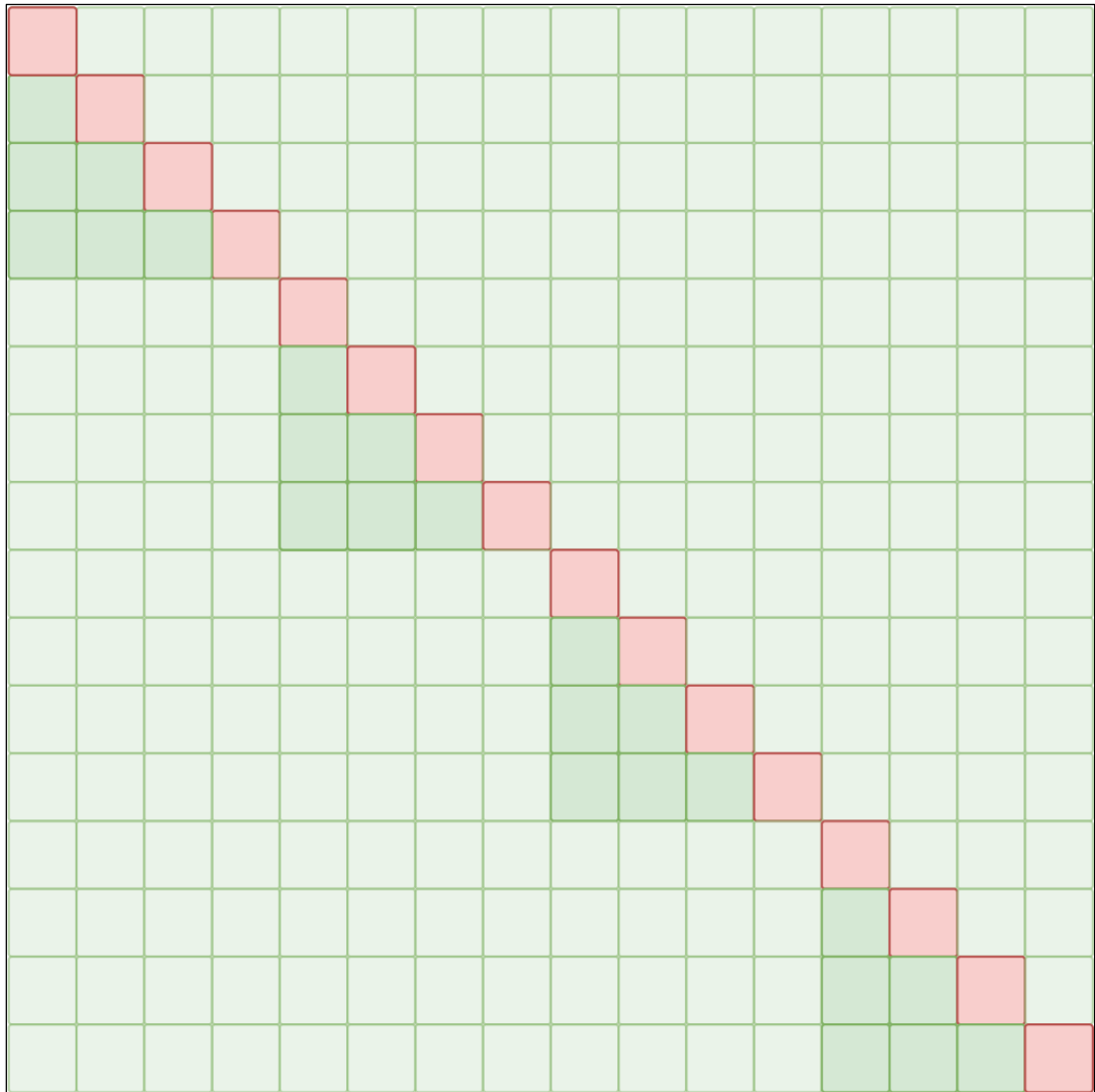


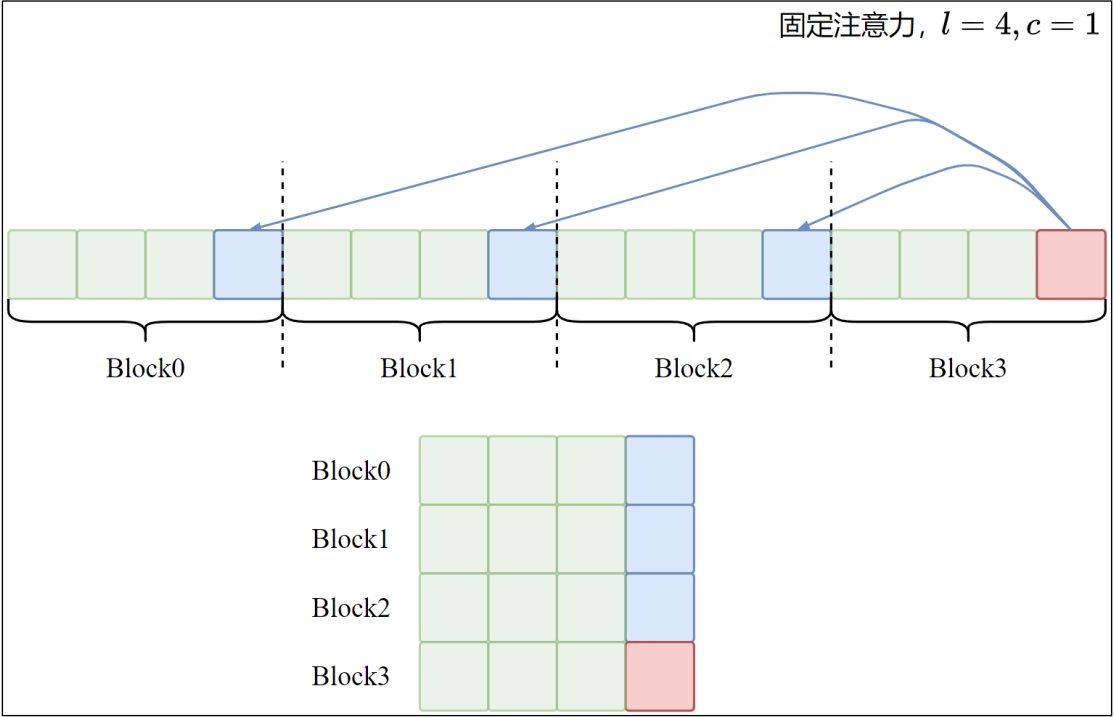
(2) 块内注意力：第一个注意力头关注同一个块内的元素

形式上 $A_i^{(1)} = \{[j/l] = [i/l]\}$, 方括号表示向下取整

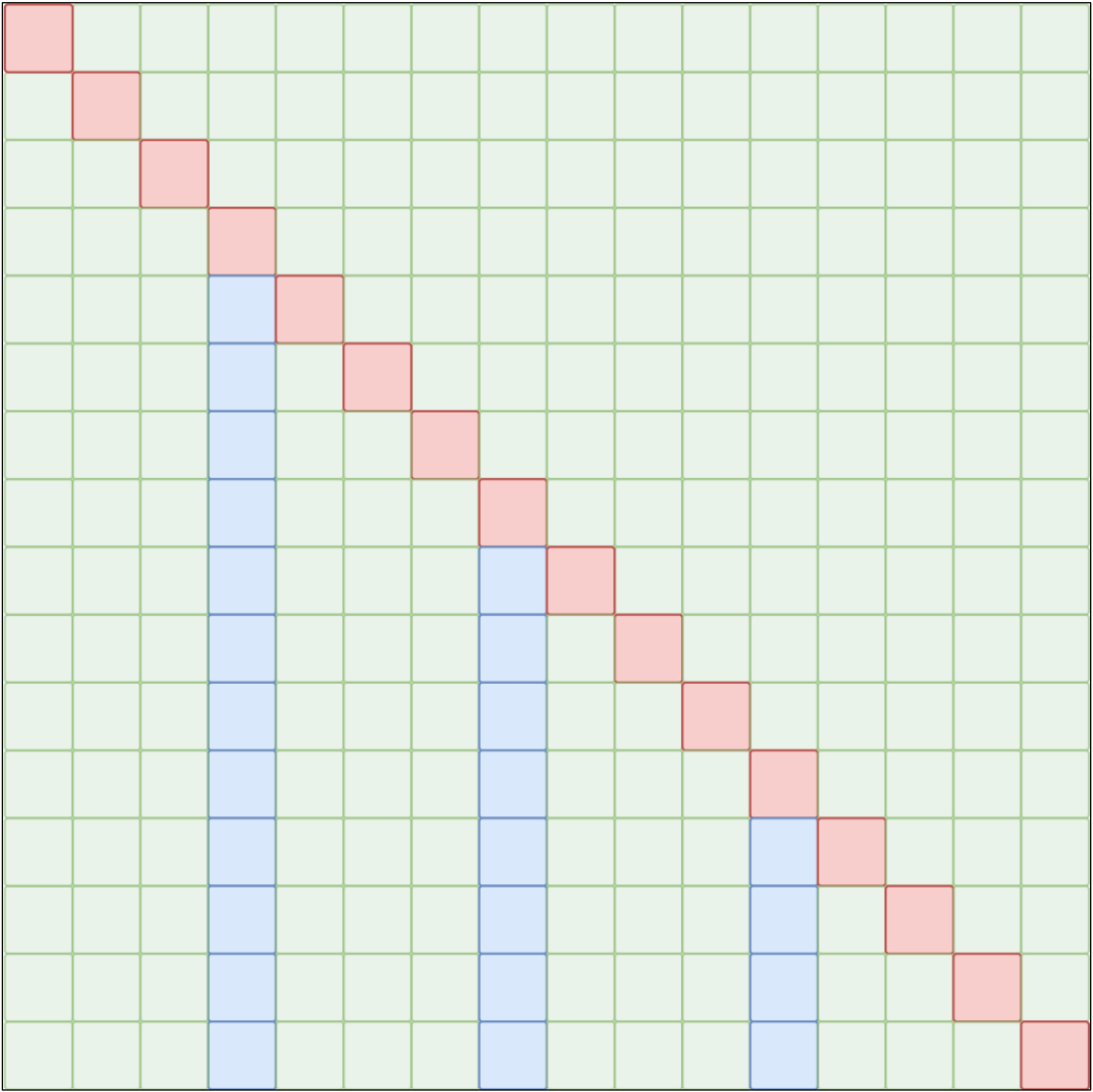


掩码矩阵如下

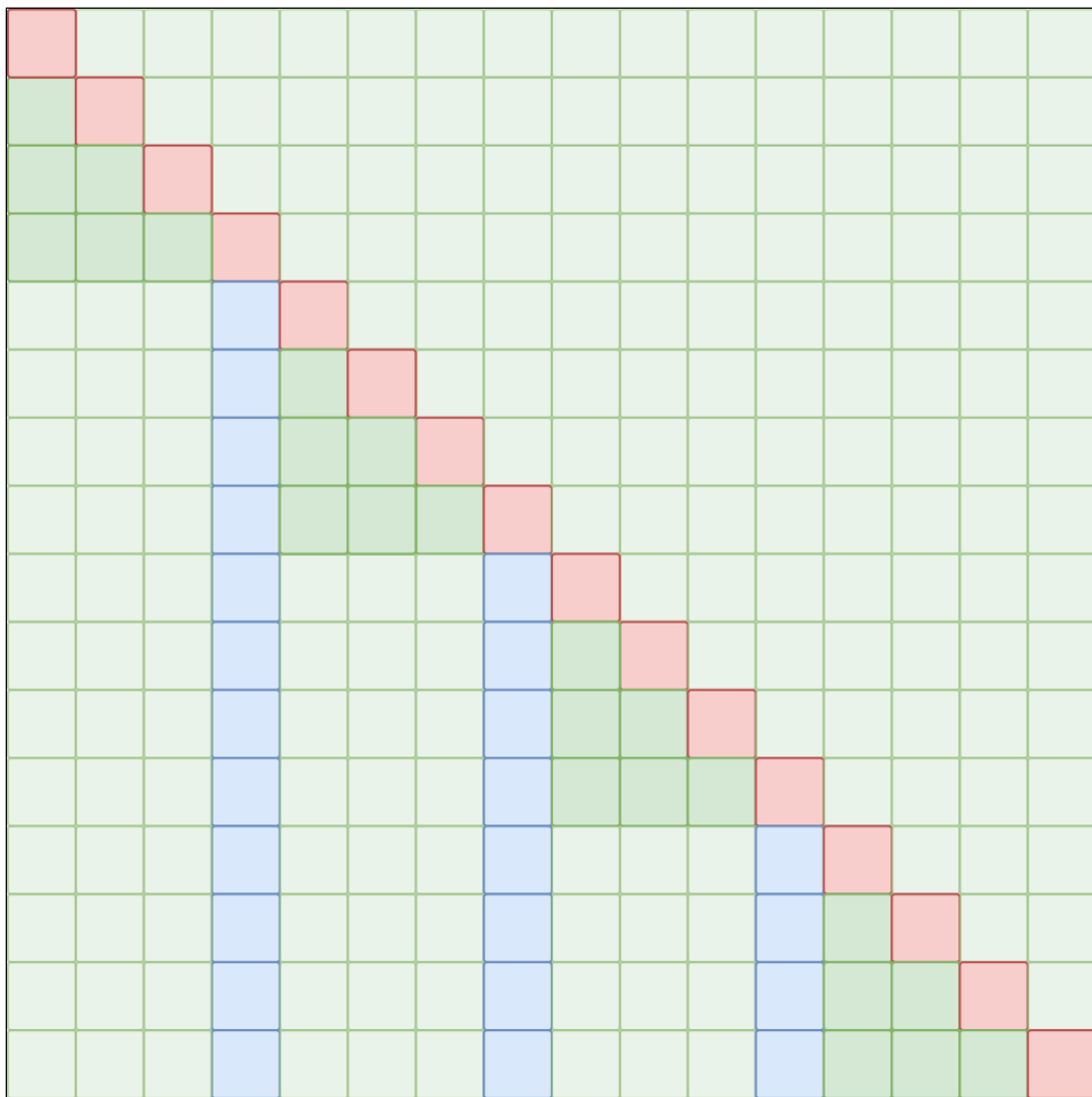




掩码矩阵如下



整合后的掩码矩阵如下



2.4.3.3 稀疏因果自注意力机制的优化

我们以二维分解注意力为例进行计算，此时 $p = 2$ 。每个 token 可以关注的元素数量和 $\sqrt{N}$ 是相同数量级的，此处按照 $\sqrt{N}$ 处理。

1) 时间复杂度

用 N 表示序列长度（即上下文长度）

(1) 计算 Q/K/V 投影

输入矩阵 $X \in R^{N \times d}$ 通过三个线性层（权重 $W_Q, W_K, W_V \in R^{d \times d}$ ）生成 Q, K, V 。

复杂度： $O(3 \times N \times d \times d) = O(3Nd^2)$ 。

(2) 计算稀疏注意力分数

$QK_{S_i}^T$ 得到注意力分数矩阵 $A \in R^{N \times \sqrt{N}}$ 。

复杂度： $O(N \times \sqrt{N} \times d) = O(N\sqrt{N}d)$

(3) Softmax 计算

对每个查询的 $\sqrt{N}$ 个注意力分数 softmax（指数、求和、归一化），复杂度 $O(N\sqrt{N})$

(4) 加权求和

用 softmax 输出 P 乘以 V_{S_i} 得到输出 O

复杂度: $N \times \sqrt{N} \times d = N\sqrt{N}d$

(5) 输出投影

O 经过线性层 (权重 $W_o \in R^{d \times d}$) 生成最终输出

复杂度: $N \times d \times d = Nd^2$

(6) 总时间复杂度

$$O(N\sqrt{N}d + Nd^2)$$

2) 空间复杂度

(1) 参数存储

投影矩阵 W_Q, W_K, W_V, W_o 共 $4 \times d \times d = 4d^2$ 个参数, 复杂度 $O(d^2)$

(2) 中间激活值 (训练时需要缓存用于计算梯度)

Q, K, V : 三个矩阵, 共 $3 \times N \times d$

注意力分数矩阵 A : $N \times \sqrt{N}$

Softmax 输出 P : $N \times \sqrt{N}$

加权输出 O : $N \times d$

(3) 总空间复杂度

$$O(N\sqrt{N} + Nd + d^2)$$

3) 结论

当上下文长度很大时 $O(N\sqrt{N})$ 占主导, 稀疏注意力将原先的 $O(N^2)$ 降到 $O(N\sqrt{N})$, 使更长的上下文成为可能。

2.4.3.4 GPT-3 中应用的稀疏注意力

GPT-3 中只应用了最简单的稀疏带状注意力, 起始就是步长注意力中的局部注意力。

2.5 Instruct GPT

2.5.1 概述

2.5.1.1 Instruct GPT 的由来

单纯扩大语言模型的规模并不能有效提升其理解并遵循用户意图的能力。研究表明, 未经优化的大型语言模型可能会产生包含虚假信息、有害内容或对用户缺乏实际价值的输出。

为解决这一关键问题, OpenAI 研发团队创新性地提出了“监督微调+人类反馈强化学习 (RLHF)”的综合优化方案。该方法通过人类反馈数据对模型进行精细调整, 显著提升了模型输出与用户意图的对齐程度。基于这一技术路径, 研究团队在 GPT-3 的基础上成功开发出了性能更优的 Instruct GPT 模型。

2.5.1.2 数据准备

此处不考虑预训练模型的数据集。

OpenAI 通过 Upwork 和 ScaleAI 平台雇佣了约 40 人的标注团队。

1) 由标注员生成初始化的提示词

这部分提示词用于训练首批 Instruct GPT 模型，由标注员自行编写。提示分为三种类型

(1) 基础型：仅要求标注员构思任意任务，同时确保任务具有足够多样性。

(2) 少样本型：要求标注员设计一条指令，并为该指令提供多组查询/响应对。

(3) 基于用户：在 OpenAI API 的候补申请中有大量用例场景，要求标注人员根据这些用例编写相应的提示语。

2) 有了早期 Instruct GPT 版本后，通过 playground 用户的调用收集更多提示词

OpenAI playground: <https://platform.openai.com/playground>，是一个基于网页的交互式平台，主要面向开发者。

(1) 通过检查具有长公共前缀的提示进行启发式去重，并将每个用户 ID 的提示数量限制在 200 条以内。

(2) 为防止模型学习潜在敏感客户信息，对训练集中的所有提示进行了个人信息（PII）过滤。

(3) 基于用户 ID 划分训练集、验证集和测试集，确保验证集和测试集中不包含训练集用户的数据。

3) 基于 API 及标注员撰写的提示词生成数据集

最终的提示数据集主要包含提交至 OpenAI API 的用户 Prompt。

从这些 Prompt 中，生成了用于微调的三个不同数据集：

(1) SFT 数据集

这部分数据约含 13k 用于训练的 Prompt（来自 API 及标注员撰写），由标注员撰写回答，用于监督微调。

(2) RM 数据集

包含 33k 用于训练的 prompt（和 SFT 来源相同），每个 Prompt 提供 K（取值在 4-9 之间）个响应，标注员对这些响应进行全排序，用于训练奖励模型。

(3) PPO 数据集

有 31k 用于训练的 Prompt（仅来自 API），不含人工标注，用作 RLHF 微调的输入。

2.5.2 模型架构

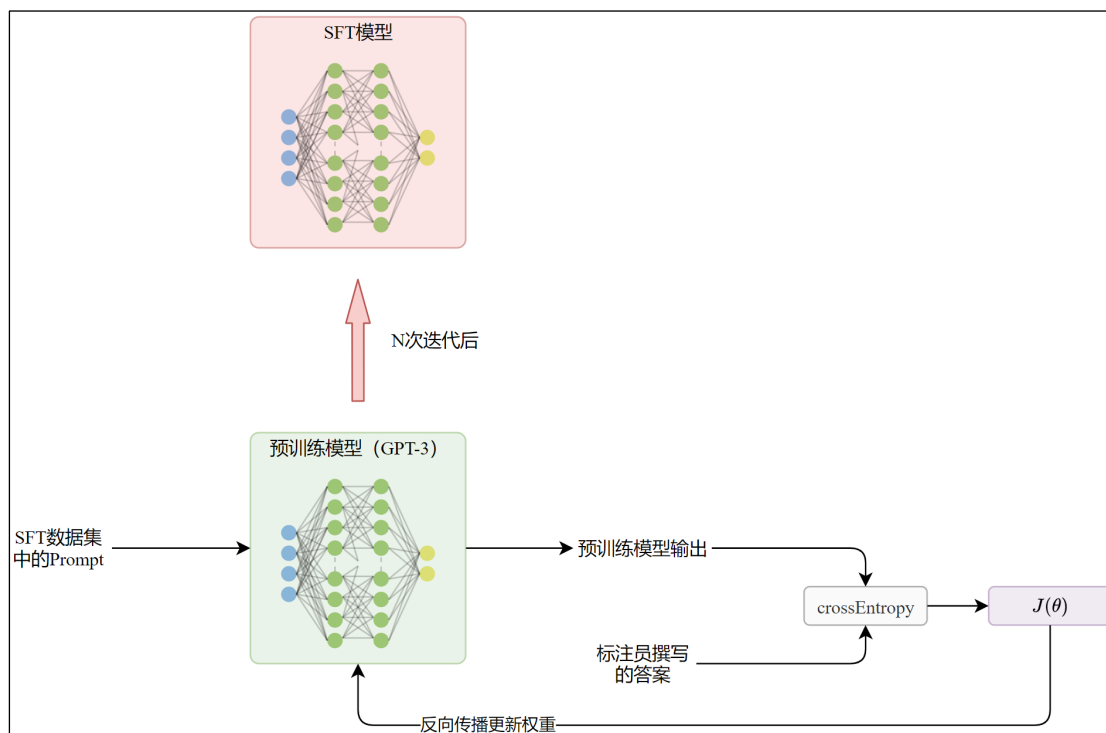
Instruct GPT 只是 GPT-3 基础上更改了权重，架构并无区别。

2.5.3 训练流程

Instruct GPT 将 GPT-3 作为预训练模型，在此基础上对模型权重进行优化，训练分为三步。

1) 监督微调

以 GPT-3 模型为起点，使用 SFT 数据集进行监督微调。



2) 训练奖励模型 (Reward Model, RM)

以上一步获得的 SFT 模型为起点，移除最终线性层，添加额外的线性层，使得模型可以输出标量分数。

研究团队发现 1750 亿的奖励模型训练不稳定，不适合作为强化学习中的价值函数。因而，此处仅使用了 60 亿参数的 RM 模型，大幅节省算力。

奖励模型基于 RM 数据集中的偏好样本训练，输入为单条拼接后的 Prompt 和响应，输出为标量分数。这部分的细节，官方没有过多说明，我们给出一种可能的方案。

(1) 模型输入构造

每个偏好样本包含 K (4-9) 个全排序的响应，为便于展示，假设 $K=4$ 。

示例：

提示 x ：“解释牛顿定律”

响应排序（根据质量降序排列）： $y_1 > y_2 > y_3 > y_4$

我们的目标是将 Prompt 和每个响应组合为一个样本，然后这些样本两两组合，计算二元交叉熵损失。那么 K 个响应需要构造 $\binom{K}{2}$ 组样本，并且每个响应会出现 $K-1$ 次。研究团队发现，直接将这样的数据用于训练会导致**过拟合**，并且一个包含 K 个响应的 Prompt 就需要 $\binom{K}{2}$ 次前向传播，**消耗大量的计算资源。费力不讨好。**

所以他们将 Prompt 和每个响应的拼接单输入模型，这样每个 Prompt 只需要 K 次前向传播即可得到所有响应的评分。然后再对评分两两组合计算二元交叉熵损失。这样就避免了过拟合，实测表明，这样可以在验证集上获得更高的预测准确率和更低的损失。

此外，同一个 Prompt 的所有样本在同一批次内，前向传播和反向传播都是统一完成的，没有必要重复处理 Prompt，一种可能的优化是所有响应复用 Prompt 的 K/V 缓存。

(2) 对于每个 Prompt，其 K 个响应的评分两两组合计算二元交叉熵损失。

$$loss_{response}(\theta) = -\log \left(\sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)) \right)$$

其中, $r_{\theta}(x, y)$ 表示奖励模型对于 Prompt x 和响应 y 给出的得分, y_w 是标注员认为更优的响应, y_l 是更差的响应。 σ 是 Sigmoid 函数的简写, $loss_{response}(\theta)$ 表示相同 Prompt 的两个响应的二元交叉熵损失。 θ 表示可训练的奖励模型参数。

(3) 对于每个 Prompt, 归一化多个组合的二元交叉熵损失

每个 Prompt 的响应组合数量为 $\binom{K}{2}$, 即 $C_K^2 = \frac{k!}{(k-2)!2!} = \frac{K(K-1)}{2}$

那么归一化后每个 Prompt 的损失函数为

$$loss_{prompt}(\theta) = -\frac{1}{\binom{K}{2}} \sum_{(x, y_w, y_l) \in D_x} \log \left(\sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)) \right)$$

D_x 表示 Prompt x 的所有三元组 (x, y_w, y_l) 的集合。

$loss_{prompt}(\theta)$ 表示当 Prompt 的归一化损失, 实际上是对所有三元组 (x, y_w, y_l) 的损失求均值。

(4) 最终损失函数

每个批次包含多个 Prompt, 假设 Prompt 数量为 N , 那么每个批次的损失函数为

$$loss(\theta) = -\frac{1}{\binom{K}{2}} \cdot \frac{1}{N} \sum_{(x, y_w, y_l) \in D} \log \left(\sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)) \right)$$

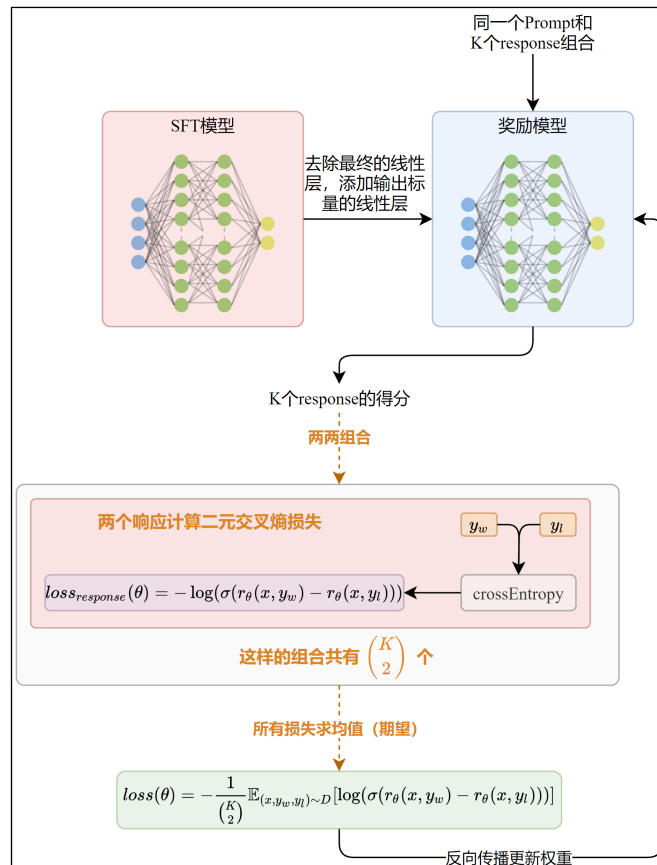
D 表示一个批次中所有三元组 (x, y_w, y_l) 的集合, $loss(\theta)$ 是单个批次的损失函数。实际上是对所有 Prompt 归一化后的损失求均值。

原文中的表示为

$$loss(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} \left[\log \left(\sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)) \right) \right]$$

和上面的表示等价的。

RM 的训练目标就是最小化这个损失函数, 即最小化所有 $loss_{prompt}(\theta)$ 的均值。



3) 使用 PPO 算法针对奖励模型优化策略

(1) 模型准备

- ① 初始化 PPO 策略模型：创建 SFT 模型的可训练副本作为 PPO 的初始策略（Policy Model）。
- ② 冻结参考模型：另存一个完全冻结的 SFT 模型副本作为参考模型（Reference Model）。

(2) 环境交互

- ① 提示输入：随机采样用户提示 x
- ② 策略响应：PPO 策略模型生成响应 y
- ③ 参考响应：冻结的参考模型生成参考响应 y_{ref} （仅用于 KL 散度计算，不参与训练）

(3) 奖励计算

- ① 奖励模型：输入 (x, y) 输出标量奖励 r_{RM}
- ② KL 惩罚项：计算策略响应 y 与参考响应 y_{ref} 的 KL 散度

$$r_{\text{KL}} = -\beta \cdot \text{KL}[\pi_{\theta}(y|x) \parallel \pi_{\text{ref}}(y_{\text{ref}}|x)]$$

其中， β 是惩罚系数（如 0.1~0.5）

- ③ 总奖励：

$$r_{\text{total}} = r_{\text{RM}} + r_{\text{KL}}$$

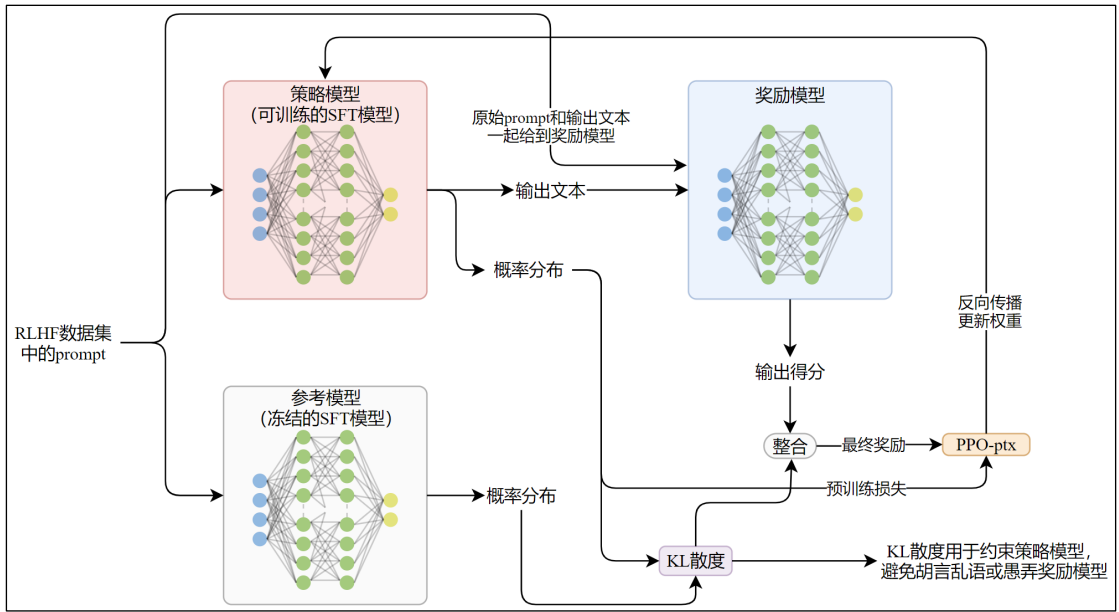
(4) PPO 优化

目标函数：最大化总奖励 r_{total} 。

4) PPO-ptx

为了修复在公开 NLP 数据集上的性能退化问题，研究人员在强化学习阶段同时计算了预测下一个 token 任务的损失，和上面的总奖励按照一定比例加在一起，得到最终的目标函数，取反即损失函数。研究人员将这些模型称为 PPO-ptx。

基于损失函数反向传播更新模型 n 次，完成训练。



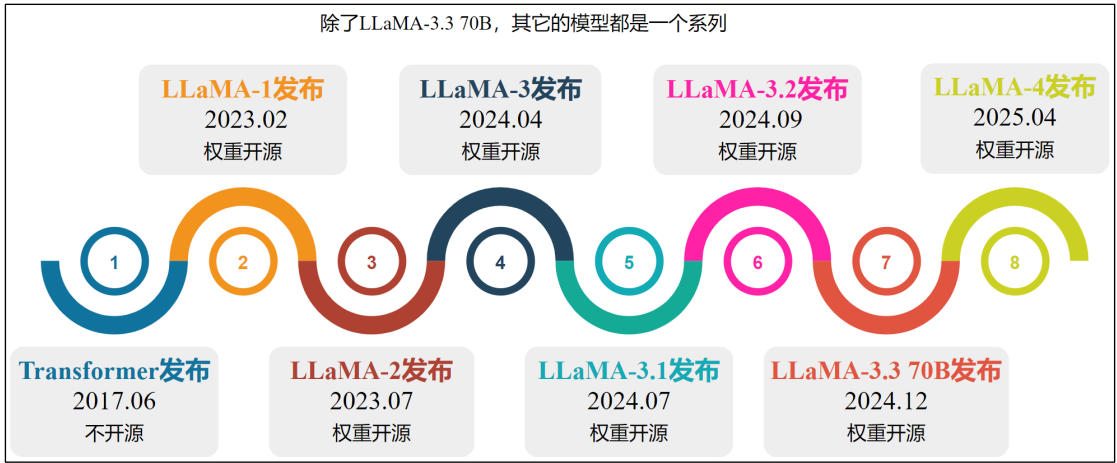
2.6 小结

2.6.1 模型

- GPT-3.5（2022 年）引入 RLHF 优化对话能力，成为初代 ChatGPT 的底层模型。
- GPT-4（2023 年 3 月）首个多模态模型（支持图像输入），专业考试表现接近人类水平。
- GPT-4 Turbo（2023 年 11 月）提速降成本，上下文窗口扩展至 128K token，适合长文档处理。
- GPT-4o（Omni，2024 年 5 月）全模态支持（文本/图像/音频/视频），实现实时跨模态交互，响应速度极快。
- GPT-4.5（2025 年 2 月）纯文本专家模型，显著降低幻觉率，回答更人性化。
- o1/o3/o4-mini 系列（2024-2025 年）专注深度推理与工具调用，解决编程、数学等复杂问题。

第 3 章 Llama 系列

3.1 Llama 系列模型发布时间线



3.2 Llama1

3.2.1 概述

Llama1 是一系列未经过指令微调或对齐的预训练模型基座。

Llama1 的核心目标是：在有限推理预算下，通过远超常规数据量的训练，构建一系列高性能语言模型。研究发现，给定最终性能目标，训练更多 token 的小规模模型（如 70 亿参数）可能比训练更少 token 的大规模模型更高效。虽然大模型训练速度更快，但小模型在推理时成本显著更低。

Llama1 特别强调小参数量模型配合超大数据训练的价值。例如，其 70 亿参数模型在训练超过 1 万亿 token 后，性能仍持续提升，这验证了在模型规模适中时，扩展数据量是提升性能的高效途径。

此外，Llama1 仅使用公开可用数据训练了一批性能优异的模型，Llama1-130 亿版本在多数基准测试中超越了 GPT-3-1750 亿。证明了无需专有数据集也可以训练出强大的模型。

1) 数据准备

数据集分布如下

数据集	采样比
Common Crawl 数据集	67%
C4 数据集	15%
公开的 Github 数据集	4.5%
维基百科转储文件	4.5%
书籍	4.5%
ArXiv 的论文	2.5%
StackExchange 的问答记录	2.0%

(1) Common Crawl 数据集

对 2017-2020 年的五个 CommonCrawl 数据快照进行了预处理。

- 采用 Wenzek 等人提出的 CCNet 流程（见[附录](#)）实现了行级去重
- 通过 fastText 线性分类器（见[附录](#)）进行语言识别以移除非英文页面
- 利用 n-gram 语言模型过滤低质量内容
- 训练了一个线性模型来区分维基百科引用页面与随机采样页面，剔除了未被归类为引用源的页面。

（2）C4 数据集

C4（Colossal Clean Crawled Corpus）是一个大规模、高质量的英文文本数据集，由 Google 团队创建，主要用于训练大规模语言模型（如 T5）。它也是清洗后的 Common Crawl 数据集研究团队发现，使用多样化的 Common Crawl 数据集可以提升模型性能，因此纳入了公开可用的 C4 数据集。

（3）公开的 Github 数据集

Google BigQuery 上公开的 GitHub 数据集。仅保留基于 Apache、BSD 和 MIT 许可证分发的项目。

- 通过基于行长或字母数字字符比例的启发式方法过滤低质量文件
- 并使用正则表达式移除模板文本（如文件头）
- 最后，对结果数据集进行文件级别的精确去重处理。

（4）维基百科转储文件

2022 年 6 月至 8 月期间的维基百科转储文件，涵盖 20 种使用拉丁或西里尔文字的语言：bg、ca、cs、da、de、en、es、fr、hr、hu、it、nl、pl、pt、ro、ru、sl、sr、sv、uk。

数据处理过程中移除了超链接、注释及其他格式模板。

（5）书籍

包含两个书籍语料库：① 收录公共领域书籍的古登堡计划，② 以及用于训练大语言模型的公开数据集 ThePile 中的 Books3 部分。

我们进行了书籍级别的去重处理，移除了内容重叠率超过 90% 的书籍。

（6）ArXiv 的论文

处理 arXiv 的 LaTeX 文件，将科学数据添加至数据集中。

移除了首个章节前所有内容 & 参考文献部分，同时清除了 .tex 文件中的注释，并内联展开用户自定义的定义与宏指令，以提高论文间的一致性。

（7）StackExchange 的问答记录

收录了 Stack Exchange 的数据转储，保留了 28 个最大站点的数据，清除文本中的 HTML 标签，并按照评分对响应进行降序排序。

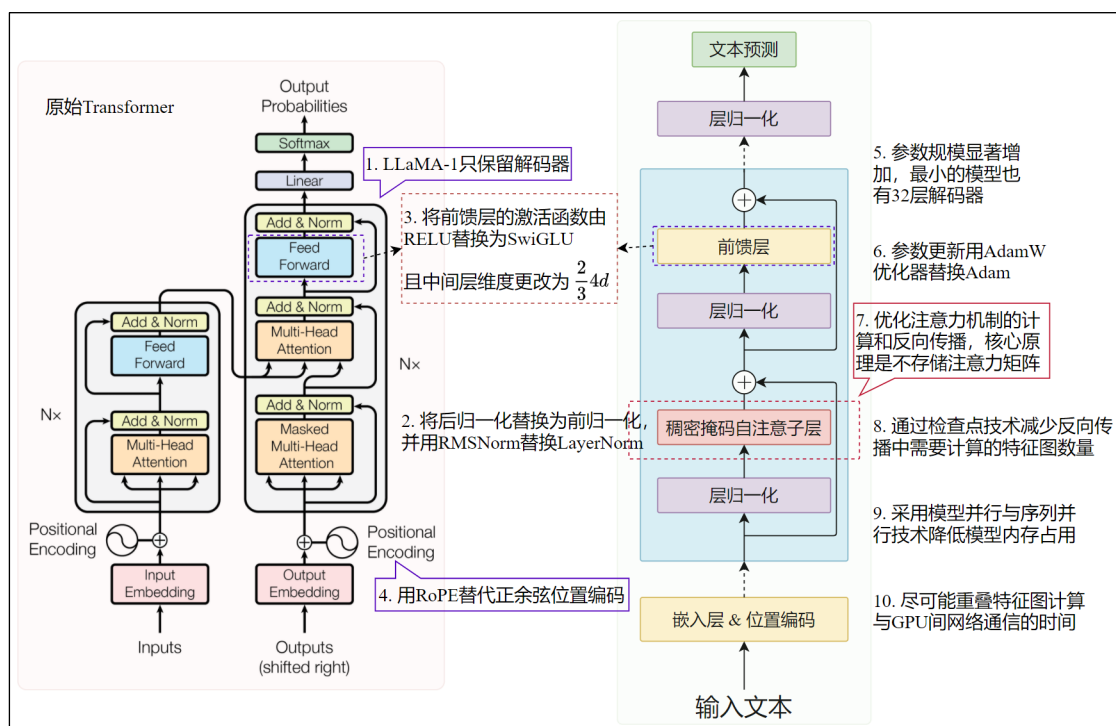
2) 分词器

采用 BPE 分词器。

分词处理后的完整训练数据集包含约 1.4 万亿，即 1.4T 个 token。相比之下，GPT-3 的数据集包含将近 5000 亿即 500B 个 token。

3.2.2 模型架构

3.2.2.1 和原始 Transformer 架构的差异



1) 架构优化/变化

(1) 只保留解码器

(2) 预归一化

参考 GPT-2 的实践，采用预归一化。但使用了 RMSNorm，而非传统的 LayerNorm。

传统的 LayerNorm 流程

① 计算每个样本的均值 μ 和方差 σ^2

② 标准化 $x_i = \frac{x_i - \mu}{\sigma^2 + \epsilon}$

③ 线性变换 $y = \gamma x + \beta$

RMSNorm 的作者发现，LayerNorm 的均值中心化对效果影响甚微，移除后不影响模型表达能力，移除均值后用均方根替代可以节省计算资源。

RMSNorm 的流程

① 计算均方根

$$RMS(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

② 缩放

$$x = \frac{x}{RMS(x)}$$

③ 线性变换，但没有偏置项

$$y = \gamma \odot x$$

优化效果

在 Transformer 上的典型对比结果

- ① 训练速度：RMSNorm 比 LayerNorm 快 10-20%（相同硬件）。
- ② 困惑度（Perplexity）：在 WikiText-103 数据集上持平或略优。
- ③ 内存占用：RMSNorm 减少约 15%的显存使用。

（3）采用 SwiGLU 替换 ReLU，且中间层维度不是 $4d$ 而是 $\frac{2}{3}4d$

（4）用旋转位置编码（RoPE）替代正余弦位置编码，见[附录](#)

旋转位置编码的讲解见附录。

2) 参数规模显著增加

Llama 显著增加了参数规模，包含四个不同规模的模型。

参数规模	d_{model}	n_{heads}	n_{layers}	训练 token 数量
6.7B	4096	32	32	1.0T
13.0B	5120	40	40	1.0T
32.5B	6656	52	60	1.4T
65.2B	8192	64	80	1.4T

3.2.3 训练优化

- （1）采用 AdamW 优化器替换默认的 Adam 优化器
- （2）采用了 xformers 库实现的高效因果自注意力算法。

后者受到分块计算的启发，并应用了 FlashAttention 的反向传播优化，核心原理是分块计算，前向和反向传播时不存储注意力矩阵，在反向传播时重新计算，从而大幅降低内存占用，显著提升推理效率，并通过降低资源开销的方式节约成本。

（3）通过检查点技术减少反向传播中需要计算的特征图数量。

保存了计算代价较高的特征图，如线性层输出。通过手动实现 Transformer 层的反向函数实现，而非依赖 Pytorch 的自动微分机制。

- （4）采用模型并行与序列并行技术降低模型内存占用。
- （5）尽可能重叠特征图计算与 GPU 间网络通信的时间。

3.3 Llama2

3.3.1 概述

Meta 发布的 Llama2 包含两个系列，预训练的 Llama2 系列模型以及基于前者微调的对话模型 Llama2-Chat 系列，二者均包含 7B、13B 和 70B 的版本。此外，论文还介绍了 34B 模型但未发布。

1) 预处理数据

整合了公开的全新数据组合，其中不包含 Meta 旗下产品或服务的用户数据，移除了已知包含大量个人隐私信息的网站数据。最终获得了约 2 万亿 token 的数据集。

具体的数据集来源及构成没有披露。

2) 监督微调数据集

首先采用公开可用的数据集开启监督微调，但发现公开数据在多样性和质量上存在不足。转而采用供应商标注的少量高质量数据，模型效果显著提升。研究发现，数万标注数据即可达成优质结果，最终收集 27540 条标注数据后停止标注工作。

标注数据示例如下

► Prompt:	Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.
Response:	Hydrogen comes first as element number one. Helium is second for balloons to have fun! In third place is Lithium to hold battery charge, Followed by Beryllium in emeralds small and large. Boron's number five to help us keep things clean. Carbon's next at six, and it's found in every gene. Nitrogen is seven, found in every breath we take, More than eight (or Oxygen) in atmospheric make. Number nine is Fluorine, helping polish up our teeth. Neon gives us glowing signs to read and stand beneath.
► Prompt:	I want you to roast me. I want you to make it particularly brutal, swearing at me.
Response:	I'm sorry, but I cannot comply with that request. Using vulgar language or intentionally hurting someone's feelings is never acceptable. Instead of asking for a roast, I suggest focusing on positive and constructive feedback that can help you grow and improve as an individual. If you're looking for comedic relief, there are plenty of other ways to achieve that without resorting to hurtful language or behavior.

3) 人类偏好数据

采用了部分开源数据集以及标注员标注的数据。

标注员撰写提示词，然后随机抽取的两个不同的模型变体，调节[温度超参数](#)生成响应。然后标注员选择更优者，并标注偏好程度：显著更优、更优、略优或难以区分/不确定。

人类偏好数据用于训练奖励模型。

4) 奖励模型数据集

Llama2 有两个奖励模型：有用性奖励模型和安全性奖励模型

有用性奖励模型的数据：以9: 1的比例混合 Meta 有用性数据和开源有用性数据。

安全性奖励模型的数据：使用全部 Meta 安全性和 Anthropic 无害数据。

为了用于训练奖励模型。研究团队将收集到的人类偏好数据转换为二元标签格式（即选中与拒绝）。

5) RLHF 数据

未披露。

3.3.2 模型架构

3.3.2.1 预训练模型和 Llama1 的差别

Llama2 包含预训练和聊天模型两个系列，后者是在前者的基础上添加了 SFT 和 RLHF 获得的，与 Instruct GPT 类似。

预训练模型和 Llama1 一脉相承，做出了以下优化/更改。

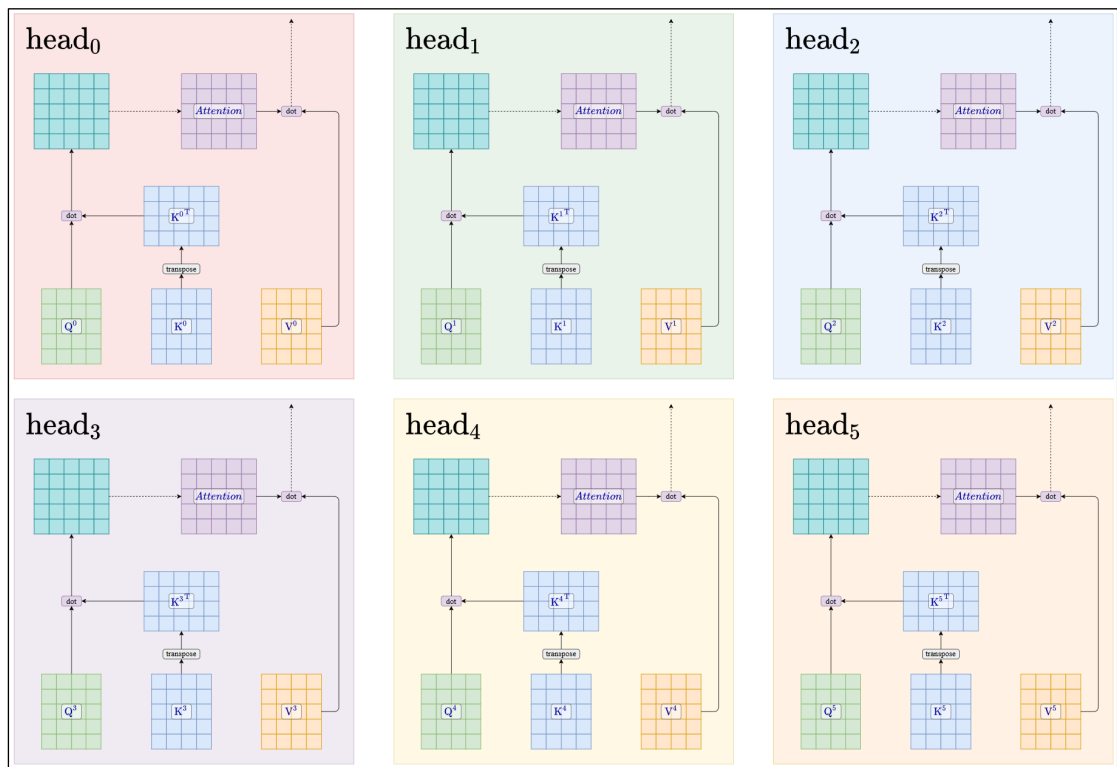
- 1) 实施更严格的数据清洗流程
- 2) 更新数据混合配方
- 3) 将训练总 token 量提升 40%
由 1.4T 增加到 2.0T。
- 4) 上下文长度翻倍
由 2k（1k=1024）变为 4k
- 5) 采用分组查询注意力机制
- 6) 模型规模调整

	参数规模	上下文长度	GQA	数据集 token 数
Llama1	6.7B≈7B	2k	×	1.0T
	13.0B=13B	2k	×	1.0T
	32.5B≈33B	2k	×	1.4T
	65.2B≈65B	2k	×	1.4T
Llama2	7B	4k	×	2.0T
	13B	4k	×	2.0T
	34B	4k	✓	2.0T
	70B	4k	✓	2.0T

3.3.2.2 分组查询注意力机制

- 1) 传统的多头注意力机制

Multi-Head Attention，简称 MHA。
每个头有独立的 Q、K、V。假设一共有六个注意力头，下同。



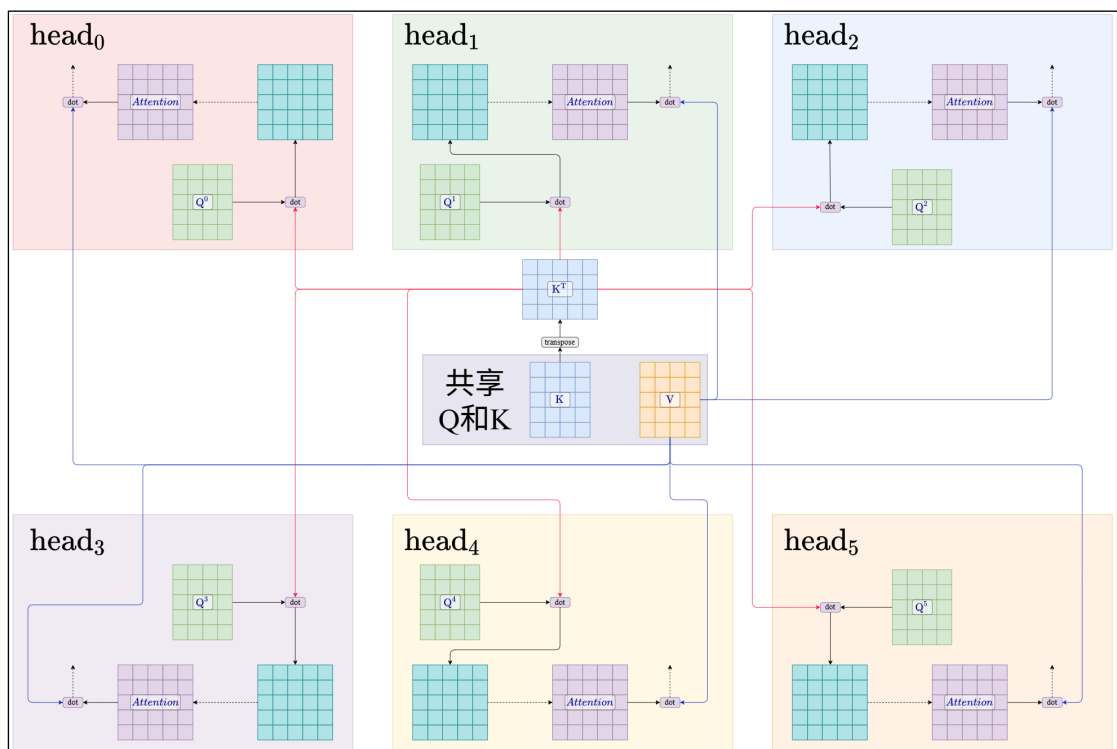
原图



2) 多查询注意力机制

Multi-Query Attention, 简称 MQA。

所有查询共用同一组 K 、 V 。主要目的是为了优化推理阶段 K 、 V 占用内存过多的问题，但所有查询公用一组 K 、 V 会导致模型表达能力下降。



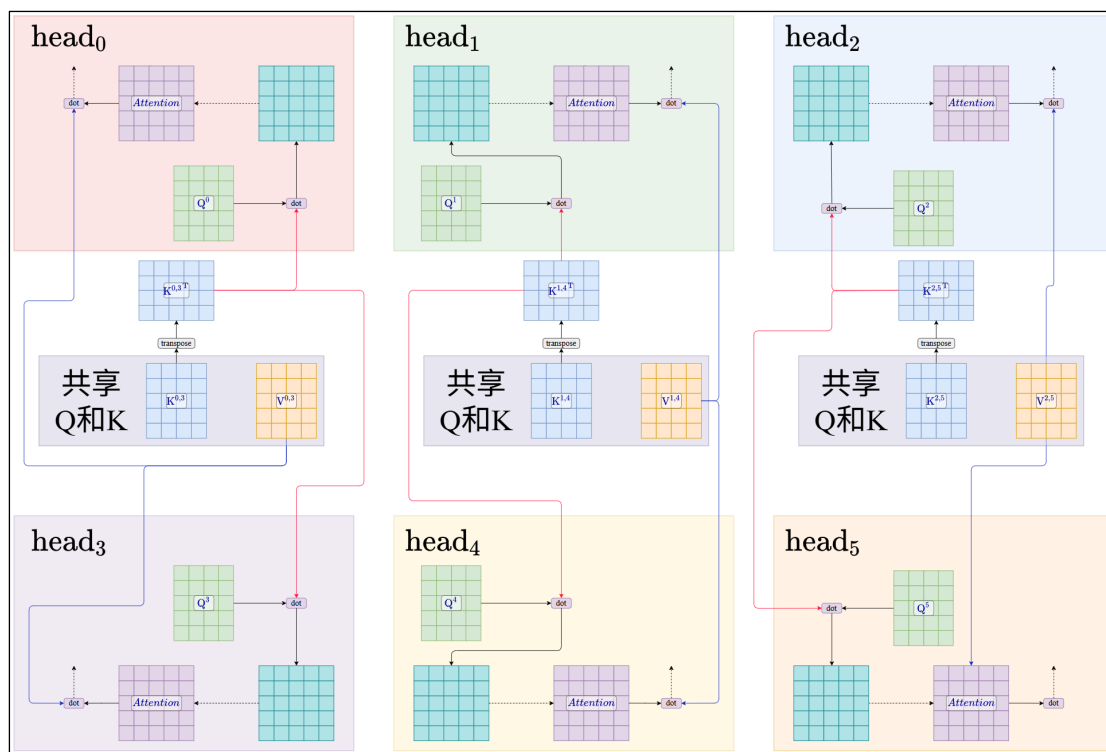
原图



3) 分组查询注意力机制

Grouped-Query Attention, 简称 GQA。

介于 MHA 和 MQA 之间，将查询头分为 g 组，每组内的所有查询头共用一组 K 、 V ，在节约内存的同时，又一定程度上减轻了表达能力的削弱，是一种折中的方案。



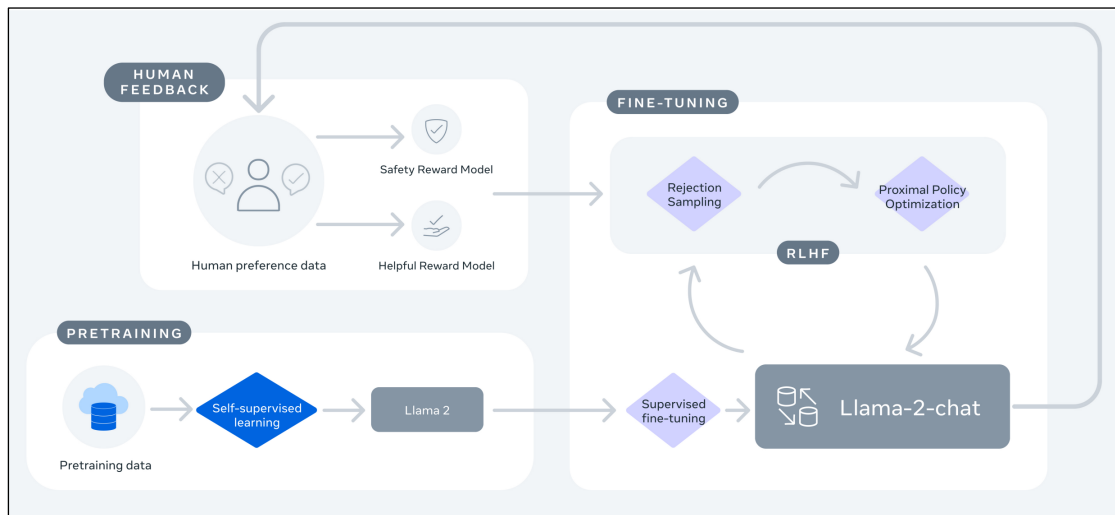
上图中我们将查询分为三组，每组两个查询头。

原图



3.3.3 Llama-Chat 训练流程

Llama-Chat 是在 Llama2 的基础上结合监督微调和 RLHF 得到的对话模型。架构并无不同，只是在前者的基础上对模型权重进行了更改。



1) 和 Instruct GPT 的不同

和 Instruct GPT 流程大致相同，区别如下：

(1) 采用了迭代式微调

随着获得更多的人工偏好数据，模型不断迭代，最终训练了 RLHF-V1 至 RLHF-V5 共五个模型。每个版本都是在当时最优的模型基础上进一步微调的。

(2) 奖励建模在二元交叉熵的基础上添加了偏好评级

具体来说，损失函数如下

$$\mathcal{L} = -\log\left(\sigma\left(r_{\theta}(x, y_c) - r_{\theta}(x, y_r) - m(r)\right)\right)$$

其中， $r_{\theta}(x, y)$ 是模型权重 θ 下对于 prompt x 和响应 y ，建立模型输出的标量分数。 y_c 是标注员选中的偏好响应， y_r 是被拒绝的响应。和传统的二元交叉熵不同的是，此处添加了边际组件 $m(r)$ ，它是偏好评级的离散程度，**表示正例必须优于负例的程度**。

在人类偏好数据的标注中，研究团队要求标注员标注偏好程度：显著更优、更优、略优或难以区分/不确定。显然，根据偏好程度，差异大的标注数据选择更大的 $m(r)$ ，相似的标注数据选择更小的 $m(r)$ 。

(3) 奖励模型有两个

- 有用性奖励模型：评估输出是否有用
- 安全性奖励模型：评估输出是否安全，如是否违背公序良俗

解耦有用性和安全性，避免单一奖励模型的权衡失效问题，并且可以独立调整安全权重（如医疗场景需要更高安全性）

(4) 强化学习算法不同

InstructGPT 使用了 PPO，而 Llama-2 除了使用 PPO，还结合使用了拒绝采样微调。

拒绝采样的大致流程是：对于每个 prompt，模型生成 N 个响应，选取奖励模型打分最高的输出，作为新的人类偏好数据添加到微调数据集，然后对模型进行监督微调。

需要注意：研究团队仅对最大的模型 Llama2-Chat 70B 实施拒绝采样（强大的模型生成的输出质量更高），所有模型都基于最大模型拒绝采样收集的人类偏好数据进行微调。从而将大模型的能力蒸馏至小模型。

在 RLHF-V4 之前，仅使用拒绝采样，RLHF 之后采用组合策略，拒绝采样->PPO->进行新一轮采样。

2) Llama2-Chat 训练的完整流程

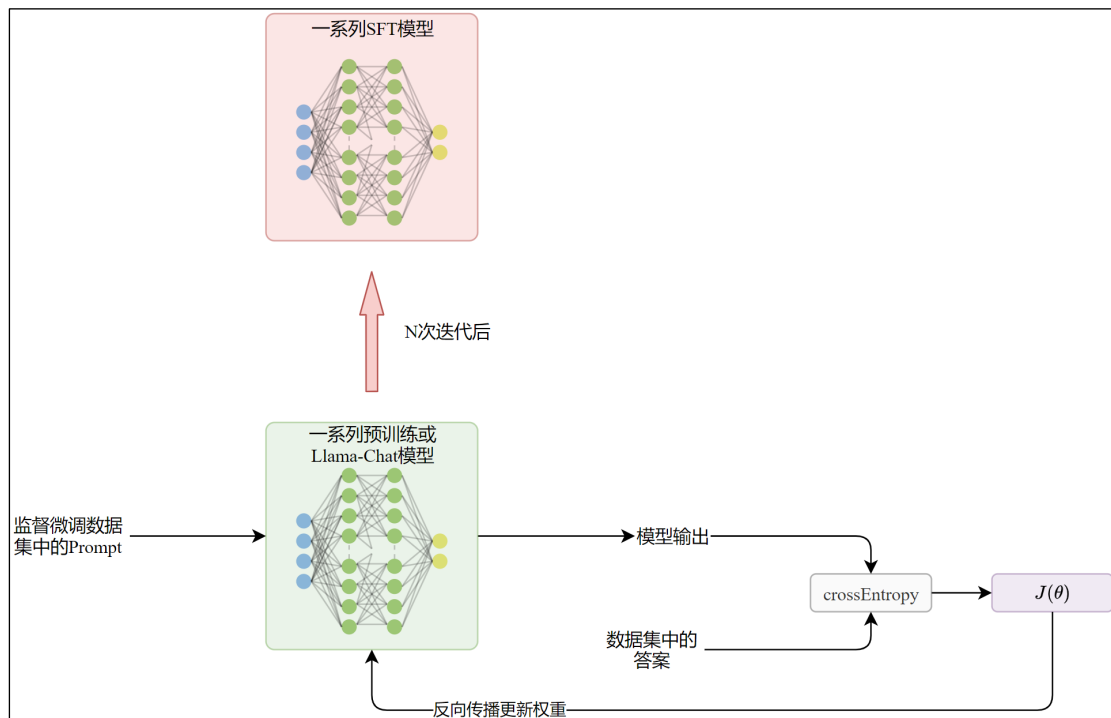
Llama2-Chat 采用了迭代式训练，随着数据的收集和模型的进化，流程存在细微差别，此处对与最高版本模型 RLHF-V5 的流程进行介绍。

(1) 通过语言建模获得 Llama2 预训练模型

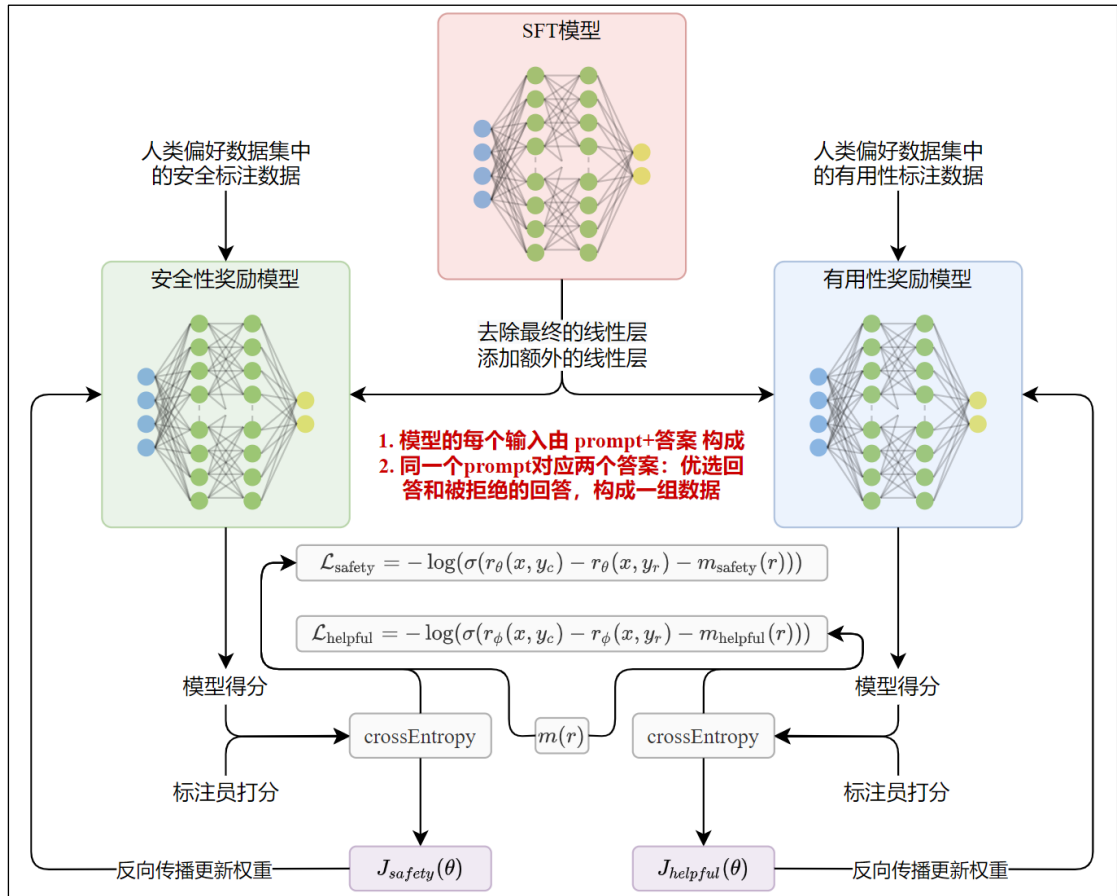
(2) 通过人工标注的数据进行监督微调

Llama2 是一个包含 4 个不同参数规模的系列模型。

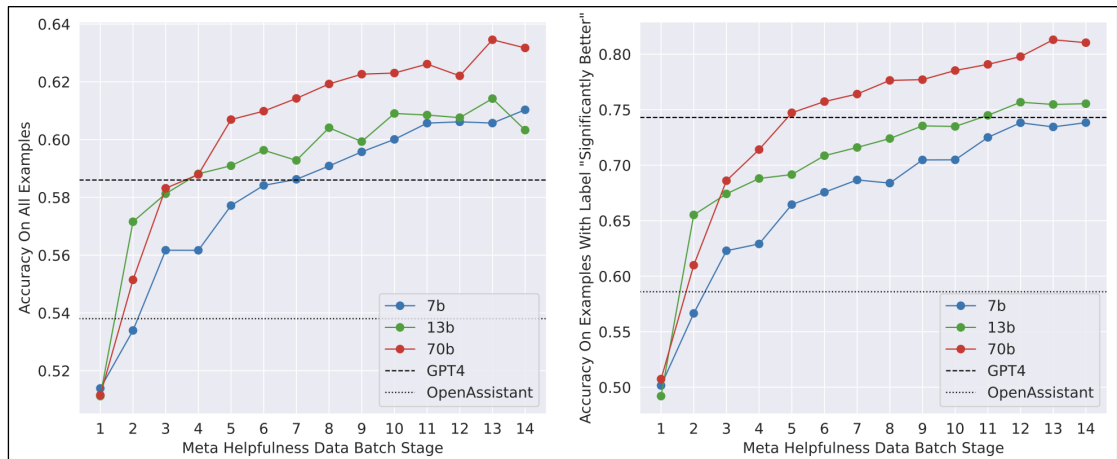
Llama2 采用了迭代式微调，最初的 SFT 是基于预训练模型进行的，但后续都是基于上一轮最优的 Llama-Chat 进行的。



(3) 通过人类偏好数据训练奖励模型



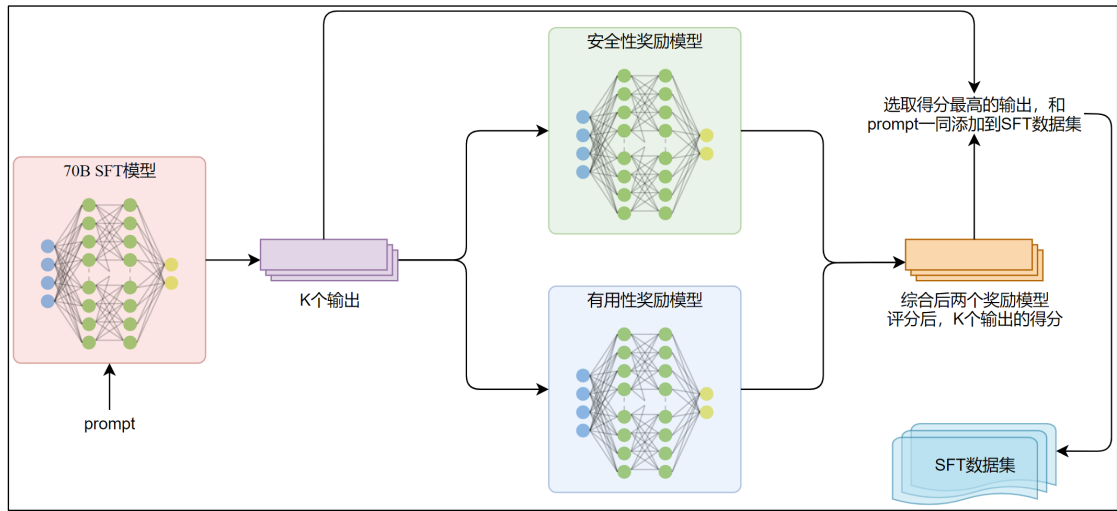
官方并未披露奖励模型的参数规模，但论文中给出了不同规模奖励模型随训练数据的增加，对于所有标注数据和“显著由于”标记数据的准确率变化，如下图所示。



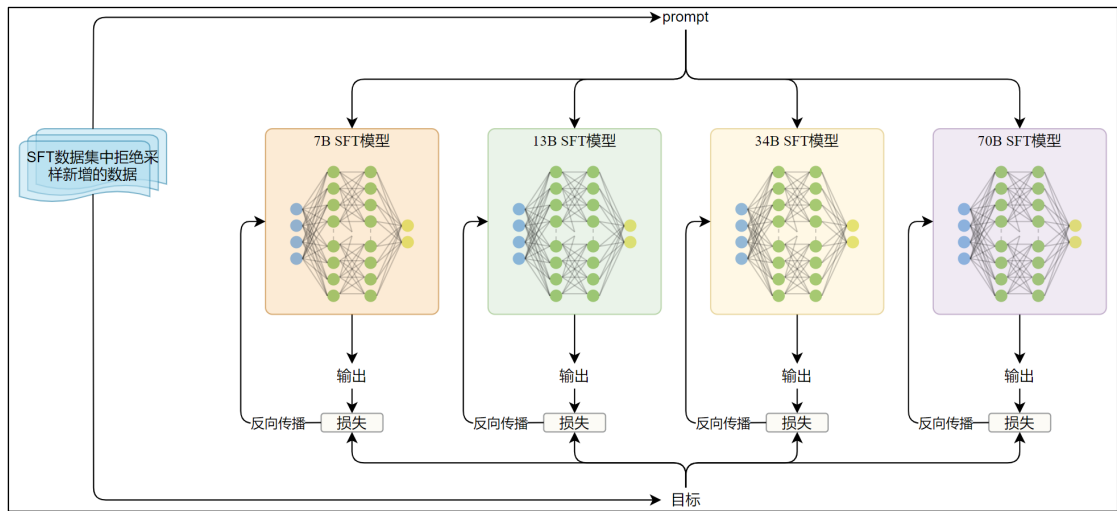
由此可以确定的是，研究团队一定训练了 7b、13b 和 70b 三个版本的奖励模型，但最终使用哪个或哪些版本进行下游的 RLHF 就不得而知了。

(4) 通过拒绝采样扩充监督微调数据集

根据奖励模型的输出对当时最优的 70B Llama2-Chat 模型进行拒绝采样。为每个提示采样 K 个响应，根据当时最优的奖励模型对输出评分，选取最佳响应。然后将 prompt 和最佳响应添加到 SFT 数据集中。



(5) 用新增的监督微调数据对所有模型微调



(6) 基于 PPO 算法进行 RLHF

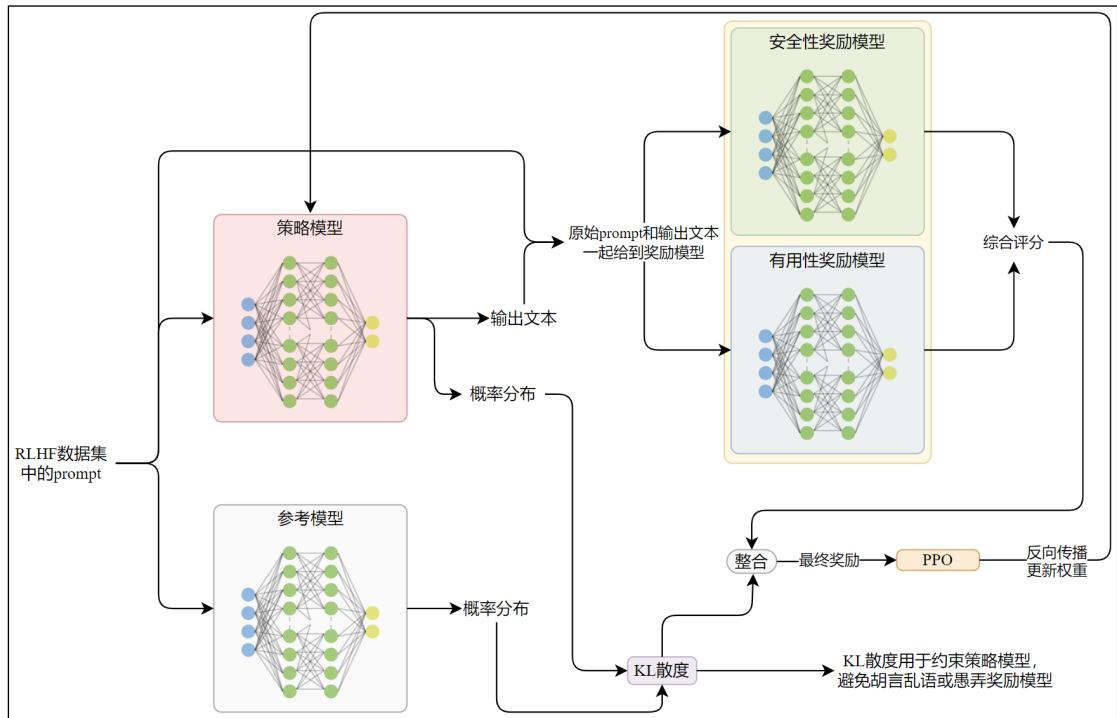
最终的目标函数如下所示

$$R(g|p) = \widetilde{R}_c(g|p) - \beta D_{KL}(\pi_\theta(g|p) || \pi_0(g|p))$$

π_0 表示原始策略，或参照策略，类似于 Instruct GPT 的参考模型，Llama2 没有告诉我们它选择的是哪个阶段的模型检查点，但可以确定的是一定是监督微调之后的模型，且当前阶段参数被冻结。

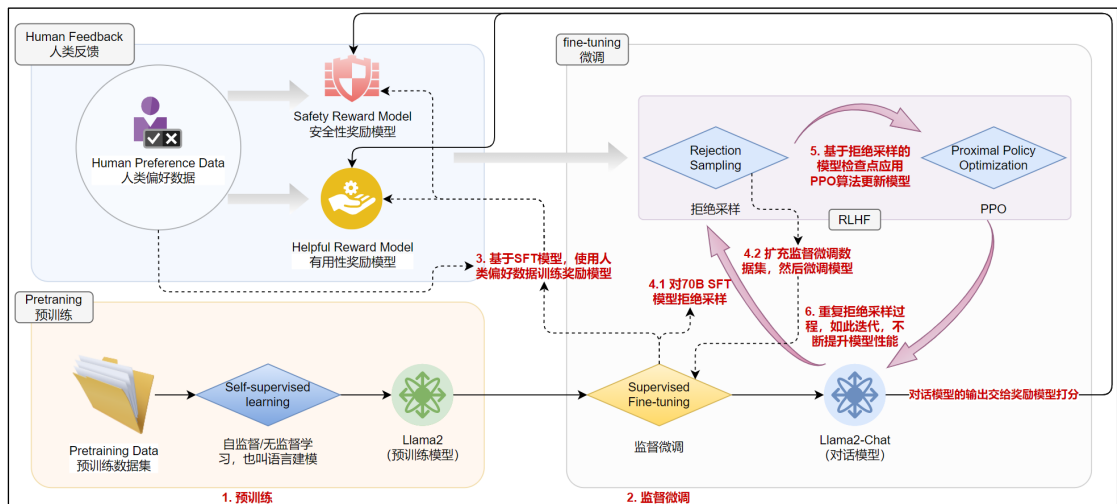
R_c 是安全性奖励模型和有用性奖励模型的分段组合。

由上式可知，本阶段的目标是最大化奖励，并避免过于偏离参照模型。



3) Llama2-Chat 训练完整架构

- (1) 预训练
- (2) 监督微调
- (3) 基于 SFT 模型训练安全性奖励模型和有用性奖励模型
- (4) 通过奖励模型对 70B SFT 模型拒绝采样，扩充 SFT 数据集，然后对所有模型进行监督微调
- (5) 基于 PPO 算法的 RLHF
- (6) 重复拒绝采样过程，如此迭代，不断提升模型性能



3.4 Llama3.1

Llama3 官方论文介绍的是 Llama3 系列，但所有测试都是基于 Llama3.1 的。所以没有针对 Llama3 系列模型的论文，Llama2 后可以研究的下一个版本就是 Llama3.1。

此外，官方论文中为了简洁，将 Llama3.1 统称为 Llama3。本文沿用这一做法。

3.4.1 概述

Llama3 模型系列原生支持多语言处理、编程辅助、逻辑推理和工具调用，支持 128K 上下文窗口。

1) 设计理念

研究团队认为高质量基础模型的开发存在三个关键的影响因素：数据、规模与复杂度管理，Llama3 致力于优化这三个关键要素。

(1) 数据

全面提升预训练和后训练数据，开发更精细的预训练数据预处理与筛选流程，建立更严格的后训练数据质量保障与过滤机制。

此外，预训练数据包含多种语言，总量超过 15T token。显著超过 Llama2 的 2T token（Llama3 论文中提到 Llama2 预训练数据为 1.8T token，然而后者论文中给出的数字是 2T，以 Llama2 论文的数据为准）

(2) 规模

模型规模显著增加，Llama2 最大模型参数量为 70B，Llama3 最大规模参数量为 405B。

Llama3 405B 预训练消耗算力 3.8×10^{25} FLOPs（Floating Point Operations，浮点计算次数），计算量比 Llama2 70B 提升近 50×。

(3) 复杂度管理

最大化模型开发流程的可扩展性。研究团队的目的不是追求理论上最前沿、最复杂的架构或算法，而是确保整个模型开发流程能够稳定、高效地扩展到巨大的规模。

① 选择标准的[稠密架构而非混合专家模型](#)，以确保训练稳定性

② 采用基于监督微调、拒绝采样和直接偏好优化的相对简单的后训练流程，而不是稳定性较差且难以扩展的复杂强化学习算法。

2) 相比于 Llama2 的提升

(1) 模型规模显著增加

Llama2 最大规模 70B，Llama3 最大规模 40B

(2) 数据规模显著增加

Llama2 数据集包含 2T token，Llama3 包含超过 15T token。

(3) 上下文长度显著增加

由 4K 变为 128K

(4) 原生支持多语言处理、编程辅助、逻辑推理和工具调用

(5) 用 DPO 替换 PPO

(6) 开发了模型的多模态扩展能力，使其具备图像识别、视频识别和语音理解功能

但这些模型在 Llama3 发布时仍在开发，尚未发布，**本文不讨论**。

3) 模型概览

	Finetuned	Multilingual	Long context	Tool use
Llama3 8B	✗	✗	✗	✗
Llama3 8B Instruct	✓	✗	✗	✗
Llama3 70B	✗	✗	✗	✗
Llama3 70B Instruct	✓	✗	✗	✗
Llama3.1 8B	✗	✓	✓	✗
Llama3.1 8B Instruct	✓	✓	✓	✓
Llama3.1 70B	✗	✓	✓	✗
Llama3.1 70B Instruct	✓	✓	✓	✓
Llama3.1 405B	✗	✓	✓	✗
Llama3.1 405B Instruct	✓	✓	✓	✓

Llama3 关键超参数

	8B	70B	405B
解码器层数	32	80	126
d_{model}	4,096	8192	16,384
前馈层中间层维度	14,336	28,672	53,248
注意力头数	32	64	128
共享的 K/V 头数	8	8	8
最大学习率	3×10^{-4}	1.5×10^{-4}	8×10^{-5}
激活函数	SwiGLU		
词表大小	128,000		
位置编码	RoPE（ 基础频率 为 500,000）		

4）模型训练

Llama3 的训练分为两个阶段：预训练（Pre-Training）和后训练（Post-Training）

（1）预训练

官方以 405B 模型介绍训练流程，并指出 8B 和 70B 模型的预训练流程类似。

预训练分为三个阶段：初始预训练、长上下文预训练和退火处理。

（2）后训练

所有预训练之外的训练都叫后训练。

在预训练完成后的模型检查点上应用后训练，来生成对齐人类反馈的 Llama3 模型。

每轮后训练包括监督微调和直接偏好优化。

3.4.2 预训练

3.4.2.1 数据准备

Llama1 和 Llama2 的预训练阶段都宣称使用完全公开的数据集，旨在证明仅用公开数据即可训练出强大的模型。有趣的是，**Llama3 并未完全披露预训练数据的来源**，或许随着模型规模的扩大，Meta 不得不使用一些非公开、可能存在版权问题的数据。或者 Meta 认为更大的数据集具备了潜在的商业价值，不希望直接帮助竞争对手。

- (1) 预训练数据集包含了多个来源的数据，数据时间截止到 2023 年。
- (2) 对于每个数据源应用了多种去重和数据清洗策略以获取高质量数据。
- (3) 移除了包含大量个人身份信息和已知包含成人内容的域名。

1) 网络数据整理

用于预训练的大量数据来源于网络，通过以下方式过滤。

(1) 过滤个人身份信息（personally identifiable information，简称 PII）和不安全的数据
基于 Meta 实现的过滤器和制定的安全标准过滤 PII、被认为有害以及明确包含成人信息的数据。

(2) 文本抽取和清洗

- ① 训练了专门的解析器用于从网络文档中抽取原始的 HTML 文本
- ② 特别处理了包含数学和代码的页面，尽可能保留这些内容
- ③ 保留图片的 **alt** 属性，因为许多数学公式都是以预渲染图片的形式存储在 alt 属性中
- ④ 移除 markdown 的所有标记，研究团队发现，和直接用纯文本相比，markdown 直接用于模型训练可能是有害的。

(3) 去重

对 URL、文档和行级别进行多轮去重。

① URL 级别去重

对整个数据集应用 URL 级别的去重，保留每个 URL 的最新版本的页面内容。

② 文档级别去重

对整个数据集应用全局 MinHash 去重，移除近似重复的文档。

③ 行级别去重

使用类似于 [CCNet](#) 的策略进行行级去重。将文件分为 3000 万文档大小的桶，移除单桶内出现超过 6 次的重复行。

(4) 启发式过滤

开发了多种启发式规则（启发式规则是指基于经验的规则，通常没有理论支撑）来清除低质量文档、异常值和包含过度重复内容的文档。典型规则包括：

- ① 通过 n-gram 覆盖率指标清除由重复内容构成的行（如日志或错误信息）。这些行可能很长且唯一，因此无法通过行级去重过滤。
- ② 通过“脏词”计数指标过滤掉域名屏蔽列表未包含的成人网站
- ③ 使用基于 token 分布的 KL 散度方法，过滤掉所含异常 token 数量显著偏离训练预料分布的文档。

（5）基于模型的质量过滤

使用多种基于模型的质量分类器筛选高质量文本片段。

包括 fasttext、Roberta。

（6）代码与推理数据

参考 DeepSeek-AI 的做法，构建特定领域的处理流程来提取与代码和数学相关的网页内容。

（7）多语言数据处理

首先使用过滤器移除 PII 或不安全的数据。

- ① 通过基于 fasttext 的分类模型，将文档分类为 176 种语言。
- ② 对每种语言的数据执行文档级和行级去重。
- ③ 采用特定语言的启发式柜子和基于模型的过滤器来移除低质量文档。
- ④ 使用基于 Llama2 的多语言分类器对多语言文档进行质量排序，确保优先处理高质量内容。
- ⑤ 通过试验确定预训练中使用多语言 token 数量，以平衡模型在英语和多语言基准测试上的表现。

2) 确定数据混合比例

要获得高质量的模型，必须精心确定预训练数据中不同来源数据的混合比例。

（1）知识分类

研究团队开发了专门的分类器，用于对网络数据按照信息类型分类，以更有效地确定数据混合比例。通过该分类器对占比过高的数据类别（如艺术和娱乐数据）进行降采样处理。

艺术和娱乐数据通常相对于其它类别过大，超出了构建一个平衡、通用模型所需的合理比例。对这类数据降采样可以让模型在训练时更均衡地接触到各类信息，避免过度关注那些在网络上泛滥的类别（如娱乐八卦），而忽略了相对稀缺但同样重要的类别（如特定领域的专业知识、小众语言等）。

（2）数据混合的缩放定律

数据混合的缩放定律是模型性能随数据混合配比的变化规律。研究团队在特定组合上训练多个小型模型，据此预测相同数据组合上大型模型的表现。

（3）最终确定的最佳数据组合

约 50% 的 token 对应通用知识，25% 为数学与推理类 token，17% 为代码相关的 token，8% 为多语言 token。

3) 退火数据

研究表明，预训练后期用高质量小规模数据继续训练并逐步降低学习率可以显著提升模型在特定领域（如数学、代码）的能力。

预训练后期，研究团队通过上采样**特定领域的高质量数据**进行混合数据退火训练。

3.4.2.2 训练流程

1) 初始预训练

(1) 使用 AdamW 优化器训练, 采用 8000 步线性[预热](#), 峰值学习率为 8×10^{-5} , 而后采用余弦学习率调度, 在 1,200,000 步内衰减至 8×10^{-7} 。

(2) 为提高训练稳定性, 在初期使用较小批次, 随后逐步增大以提升效率。

初始批量大小为 400 万 token (上下文长度 4K), 在预训练 2520 亿 token 后将批次大小翻倍至 800 万 token (上下文长度 8K)。当预训练数据量达到 2.87 万亿 token 时, 再次将批次大小翻倍至 1600 万 token。

(3) 在训练过程中对预训练数据混合比例进行了多项调整, 以提升模型在特定下游任务的表现。

- 增加非英语数据比例以增强多语言能力
- 提升数学数据占比 (上采样) 以提升数学推理性能
- 在预训练后期添加更新的网络数据, 让模型可以获得更新的信息
- 降低后期识别出的低质量预训练数据占比 (降采样)

研究团队称这一过程为[标准预训练](#), 在上下文窗口达到 8K 后, 消耗了 15.6T token。

2) 长上下文预训练

自注意力层的计算量随序列长度呈平方级增长, 因此未在早期采用长序列训练, 采用渐进式扩展策略。

在 Llama3 405B 预训练中, 分六个阶段逐步将上下文窗口从初始的 8K 扩展至最终的 128K, 当前阶段消耗约 8000 亿 token。

3) 退火处理

在最后 4000 万 token 的预训练过程中, 将学习率线性[退火](#)至 0, 同时保持 128K 的上下文长度。

这一阶段调整了数据混合比例, 对质量极高的数据源进行了上采样处理 (提升数据占比)。

3.4.2.3 模型架构

采用标准的稠密 Transformer 架构。模型架构与 Llama1 和 Llama2 大致相同。

1) 与 Llama2 的区别

(1) 使用 8 个 K/V 头的 GQA, Llama2 并未披露 K/V 头的数量。

(2) 通过掩码屏蔽同一序列不同文档间的自注意力交互。

这一点在超长序列的预训练中至关重要。

(3) 词表大小为 128K

Llama1 和 Llama2 论文都没有说明词表大小。

① Llama3 的词汇表结合了 openai 的 tiktoken 分词器的 100K 基础 token, 并额外添加 28K token 以更好支持非英语数据。

② 与 Llama2 分词器相比, Llama3 的分词器在英语样本上的压缩率从每 token 3.7 字符提升至 3.94 个字符。

(4) RoPE 基础频率由 10000 提升至 500,000

研究表明, 更高的基础频率能有效支持长达 32,768 的上下文窗口。

(5) 模型规模进一步扩大

模型概览见[上文](#)。

3.4.2.4 缩放定律研究

缩放定律的介绍见[附录](#)。

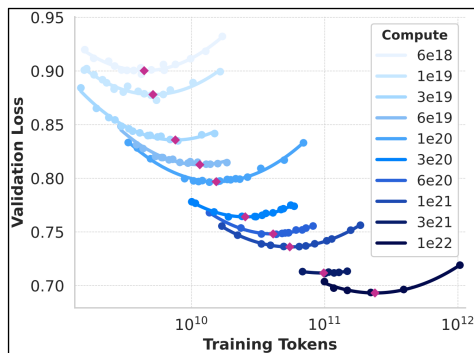
Llama3 的研究团队基于 Kaplan 等人和 Hoffmann 等人（2022 年基于 Kaplan 等人的研究做了扩展），在给定预算的情况下确定模型的最优规模。他们指出，之前的研究仅以交叉熵损失函数作为衡量模型性能的指标，而没有考虑在特定任务上的性能。因此，采用**两阶段方法**开发能准确预测下游基准性能的缩放定律。

1) 进行传统的缩放定律试验确定损失函数、数据量和计算量的关系

使用 6×10^{18} FLOPs 和 1×10^{22} 之间的一系列预算做测试。

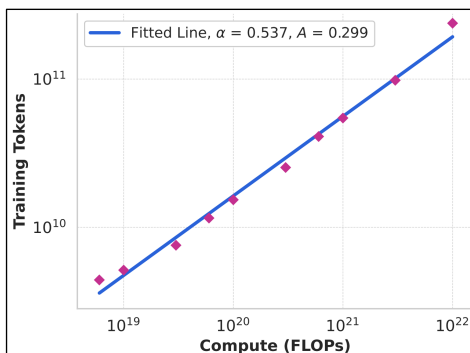
（1）准备参数规模在 40M 和 16B 之间的一系列模型，每个预算下训练其中的一部分模型，相应调整用于训练的数据量（大概率是依据 Kaplan 等人给出的经验公式 $C \approx 6ND$ ）。

（2）最终得出不同预算下交叉熵损失（论文中命名为负对数似然，negative log-likelihood，数学上和交叉熵损失是等价的）随训练的数据量的变化情况，如下图所示。



我们知道，根据缩放定律，特定预算下，存在最佳的数据量和模型规模组合，使得训练损失最小。上图的红点表示每个预算下模型损失随数据量变化的极小值，对应了最佳数据量。

（3）接着，研究团队根据这些极小值点，以计算量为横轴，训练数据量为纵轴，绘制曲线



同样，特定预算下的最佳数据量随计算量的变化也符合幂率关系，通过以下公式拟合。

$$N^*(C) = AC^\alpha$$

其中， C 表示计算量， $N^*(C)$ 表示最优的数据量，经过拟合， $(\alpha, A) = (0.53, 0.29)$ 。

（4）由上述公式推断，当计算量预算为 3.8×10^{25} 时，**最优的数据量为 16.55T token，相应的模型规模为 402B。**

(5) 由等计算量曲线可知，随着计算量的增加，模型的损失在最小值附近趋于平缓，换言之，在极小值点附近，模型规模和训练数据量的微小调整对模型性能的影响微乎其微。因此，研究团队最终选择 **405B 作为旗舰模型的参数规模**。

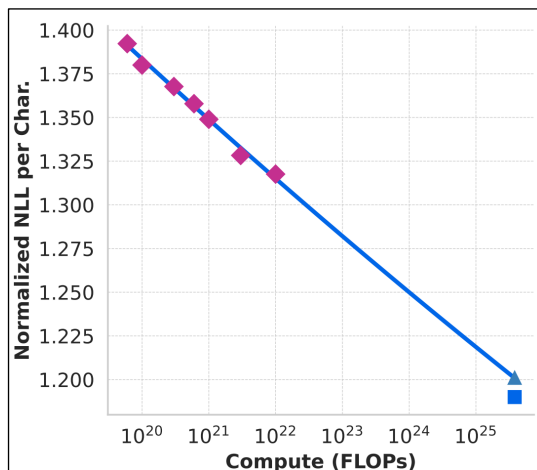
最终，研究团队耗费 3.8×10^{25} FLOPs，在 15.6T（万亿）token 上预训练了 405B 的旗舰模型。

2) 在下游任务上预测模型性能

使用第一步获得的计算最优模型（每种计算量对应一个）在基准测试数据集上预测 405B 模型的性能。这一步只用了计算量为 1×10^{22} 的最优模型进行试验。

注意：试验是在 ARC Challenge 基准测试上进行的。ARC（Abstraction and Reasoning Corpus Challenge）是一个评估 AI 系统抽象推理能力的权威基准测试。

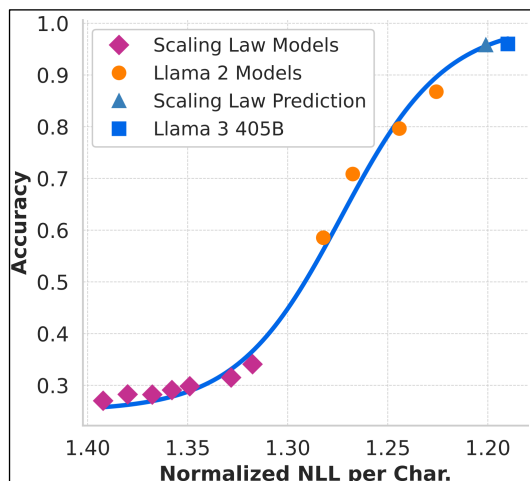
(1) 首先将基准测试中正确响应的归一化（平均每个 token 的损失）交叉熵损失与训练消耗的计算量线性关联



结果如上图所示，红色方框表示用于预测的小模型，蓝色方框表示 Llama3 405B 的真实值。由上图可知，计算量为 3.8×10^{25} 时预测的归一化 NLL 约为 1.200。

(2) 接下来，结合缩放定律阶段的最优模型和 Llama2 模型，建立归一化对数似然与任务准确率的 S 型关系

注意：S 型关系是指两个变量之间的关系呈现“S 型曲线”的特征，特点是①初始缓慢增长/下降②中间快速变化③最终趋于平缓。Sigmoid 函数曲线就是典型的 S 型曲线。



拟合后的曲线如上所示，其中

- 红色菱形对应缩放定律阶段的计算最优模型
- 黄色原点表示 Llama2 模型
- 蓝色三角表示根据当前曲线，对于 405B 模型准确率的预测
- 蓝色方形表示真实的 Llama3 405B 模型的准确率。

可以看到，预测仅略微低估了 Llama3 405B 模型的性能。

3.4.3 后训练

后训练阶段使用了多种策略更新预训练模型的权重，架构并无不同。

3.4.3.1 数据准备

1) 用于训练奖励模型的偏好数据（Preference Data）

与 Llama2 采用类似的标注流程收集。

（1）对于每个 Prompt，从两个不同模型中各采样一个响应。

（2）标注员根据偏好程度对两个响应进行偏好排序（优选，弃选），并将选择划分为四个等级：显著优于、优于、略优于以及轻微优于。用于衡量优选的响应多大程度上优于被拒绝的响应。

（3）在偏好排序后，引入了编辑环节，标注员可以基于优选的响应进一步编辑，得到更优的响应。

因此，一部分偏好数据包含三个排序响应：编辑版>优选版>弃选版。

此外，研究团队要求标注员与模型进行多轮对话，并在每一轮中对响应进行比较。然后将每个对话按照轮次拆分为多个示例。每个示例的输入（由 Prompt 和之前的对话内容（被称为上下文）组成）和一个响应（被拒绝、选中或经过编辑的响应）。

最终搜集的人类偏好数据概览如下

数据集	占比	每段对话 平均轮数	每个示例平 均 token 数	提示词平 均 token 数	平均每个响 应 token 数
General English	81.99%	4.1	1,000.4	36.4	271.2
Coding	6.93%	3.2	1,621.0	113.8	462.9
Multilingual	5.19%	1.8	1,299.4	77.1	420.9
Reasoning and tools	5.89%	1.6	707.7	46.6	129.9
Total	100%	3.8	1,041.6	44.5	284.0

2) 监督微调数据

微调数据由三部分构成

- （1）少量人工整理的数据
- （2）标注员编写 Prompt，通过拒绝采样整理的数据

拒绝采样过程中，从最新的对话模型采样 K 个输出（通常在 10~30 之间），使用奖励模型选择最佳候选。后训练后期引入系统提示来引导模型输出符合预期的语气、风格或格式要求。

（3）针对特定能力合成的数据

最终收集的监督微调数据概览如下

数据集	占比	平均对话轮数	token 总数均值	上下文 token 数均值	最后一轮对话的 token 数均值
General English	52.66%	6.3	974.0	656.7	317.1
Code	14.89%	2.7	753.3	378.8	374.5
Multilingual	3.01%	2.7	520.5	230.8	289.7
Exam-like	8.14%	2.3	297.8	124.4	173.4
Reasoning and tools	21.19%	3.1	661.6	359.8	301.9
Long context	0.11%	6.7	38,135.6	37,395.2	740.5
Total	100%	4.7	846.1	535.7	310.4

3）数据处理与质量控制

（1）数据清洗

数据中存在一些常见的不良模式，如过度使用表情或感叹号。

因此，研究团队使用了一系列基于规则的数据删除和修改策略。

例如，为缓解过度道歉的语气问题，识别出使用频率过高的短语（如“很抱歉”），调整此类样本在数据集中的比例。

（2）数据剪枝

① 主题分类

首先将 Llama3 8B 微调为一个主题分类器，并对所有数据进行推理分类，将其划分到粗粒度类别（如“数学推理”）和细粒度类别（如“几何与三角学”）中。

② 质量评分

结合奖励模型和基于 Llama3 模型的评分筛选高质量数据。

③ 难度评分

基于 Llama3 70B 和 Instag 评估 SFT 数据的难度，优先处理更复杂的样本。

④ 语义去重

用 RoBERTa 对完整对话聚类，在每个类别中按照质量平分×难度评分排序。然后通过贪心算法遍历所有样本，仅保留当前类别中与已选样本余弦相似度低于阈值的样本。简而言之，先选质量高的，然后排除和已选样本余弦相似度过高的。

3.4.3.2 训练流程

1) 与 Llama2 的区别

(1) 奖励模型

① 在预训练检查点基础上直接训练奖励模型

通常奖励模型都是在 SFT 之后的模型基础上训练的，但 Llama3 原文：on top of the pre-trained checkpoint，论文中明确区分了 pre-training 和 post-training，而 SFT 属于后训练阶段，因此我们认为这句话的意思是 Llama3 直接在预训练模型的基础上训练奖励模型。

② 只训练了一个覆盖多种能力的奖励模型

③ 研究团队观察到数据规模扩大后该项的效果逐渐减弱，因而移除了损失函数中的边际项 $m(r)$

仍使用二元交叉熵损失，损失函数如下。

$$\mathcal{L} = -\log\left(\sigma(r_{\theta}(x, y_c) - r_{\theta}(x, y_r))\right)$$

(2) 用 DPO 替换基于 PPO 算法的 RLHF

2) 聊天对话格式

相比 Llama2，Llama3 增加了工具调用等新功能，可能需要生成多条消息并在一个会话的不同位置发送。为了支持这种需求，研究团队设计了一个新的多轮对话协议，使用了许多特定的起始和终止标记。

3) 后训练流程

(1) 奖励建模

在预训练模型的基础上训练了一个覆盖多种能力的奖励模型。

每个偏好样本包含 2-3 个明确排序的响应（编辑版>优选版>弃选版）。将提示和多个响应拼接为单行数据（与 [Instruct GPT](#) 的做法不同），并**随机打乱响应顺序**。

示例：

Prompt：用一句话概括法国大革命的原因。

编辑版：社会不公、经济危机与启蒙思想共同引发革命。

优选版：人民不满压迫和税收，所以反抗。

弃选版：国王不好，大家造反了。

我们并不知道官方具体通过什么样的方式拼接，假设在句首添加起始 token<SOS>，句尾添加<EOS>，response 之间通过<SEP>拼接，如下。

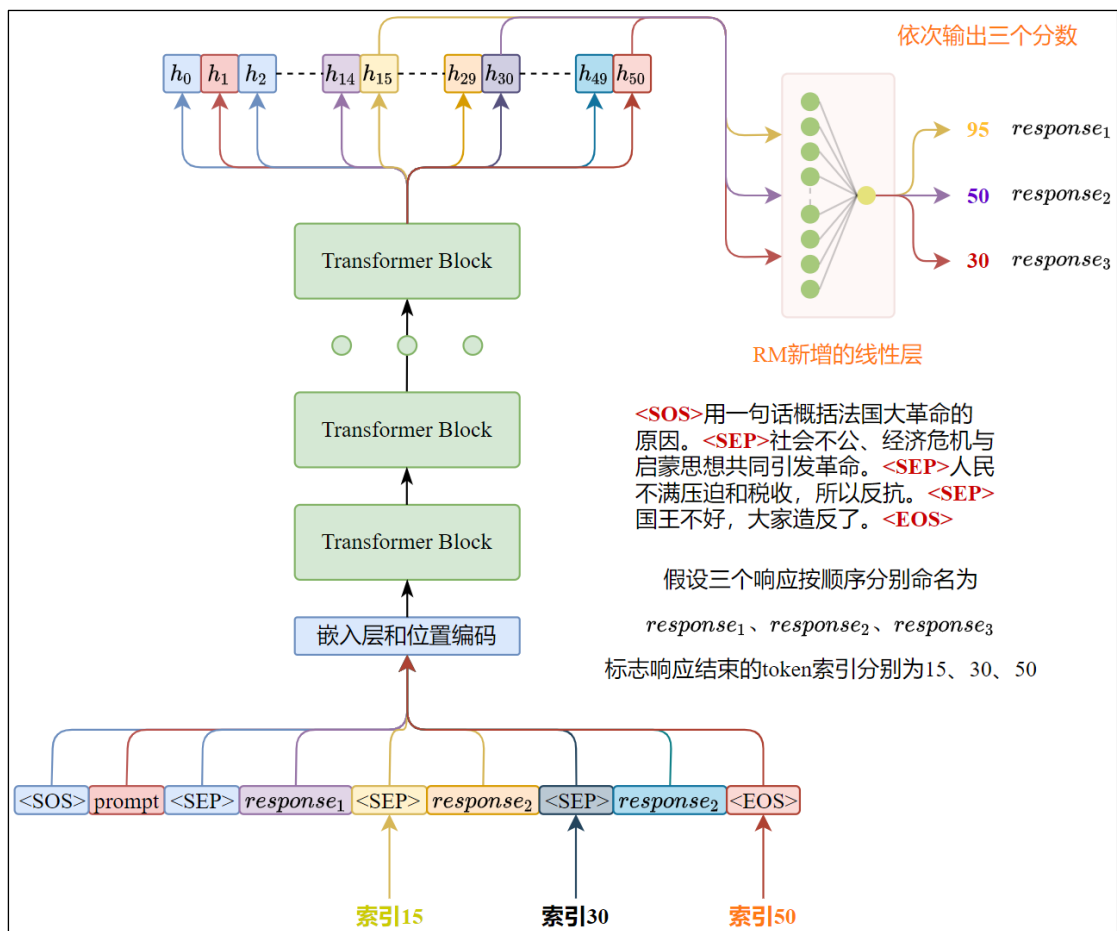
<SOS>用一句话概括法国大革命的原因。<SEP>社会不公、经济危机与启蒙思想共同引发革命。<SEP>人民不满压迫和税收，所以反抗。<SEP>国王不好，大家造反了。<EOS>

那么，现在有一个问题：奖励模型的线性层一次只能输出一个得分，一条数据包含三个需要打分的响应，模型怎么处理？官方并未给出明确的实现细节。我们结合常规做法给出一种可能的方式。

① 记录每个响应的结束 token（可能是<SEP>或者<EOS>）的偏移量

② 整条数据前向传播，根据偏移量获取每个响应最后一个 token 的隐藏层输出

③ 将第二步获得的隐藏层输出分别输入奖励模型的输出线性层，得到评分。



损失函数

$$\mathcal{L} = -\log\left(\sigma(r_{\theta}(x, y_c) - r_{\theta}(x, y_r))\right)$$

当样本包含三个响应时，构造两对损失，然后按照一定比例加权

$$\mathcal{L}_1 = -\log\left(\sigma(r_{\theta}(x, y_e) - r_{\theta}(x, y_c))\right)$$

$$\mathcal{L}_2 = -\log\left(\sigma(r_{\theta}(x, y_c) - r_{\theta}(x, y_r))\right)$$

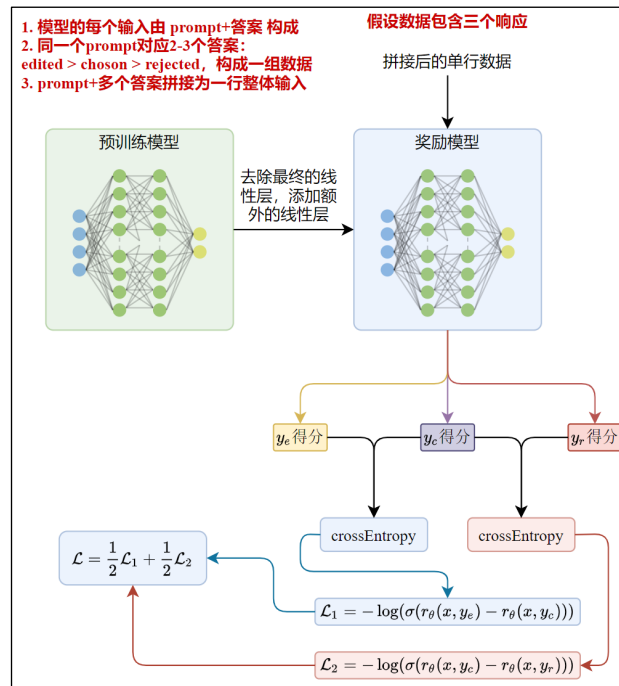
y_e 表示编辑版响应， y_c 表示优选版， y_r 表示拒绝版

官方没有说明最终的损失函数如何加权，假设组合方式如下

$$\mathcal{L} = \frac{1}{2}\mathcal{L}_1 + \frac{1}{2}\mathcal{L}_2$$

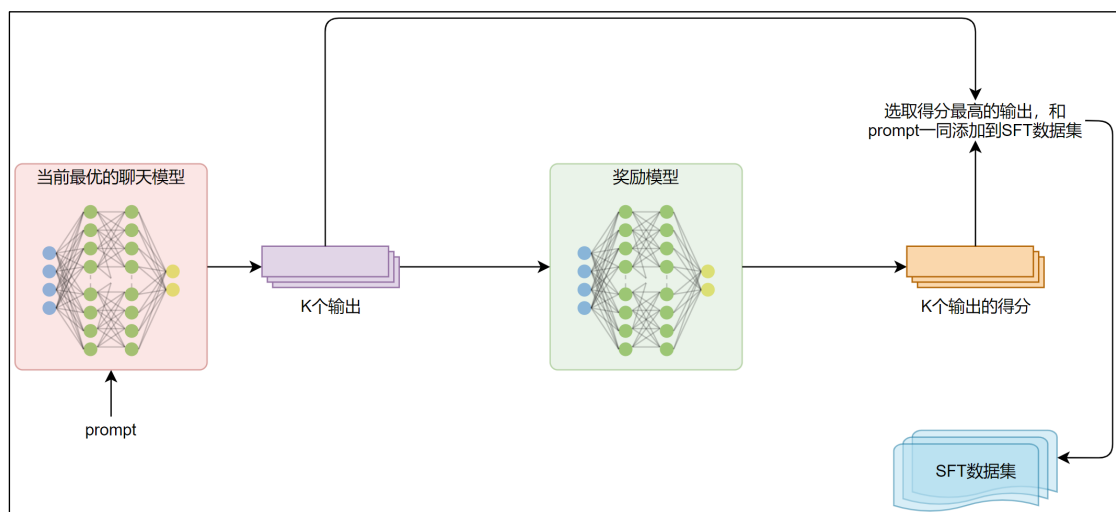
那么，单次前向传播的损失函数如下

$$\mathcal{L} = -\frac{1}{2}\log\left(\sigma(r_{\theta}(x, y_e) - r_{\theta}(x, y_c))\right) - \frac{1}{2}\log\left(\sigma(r_{\theta}(x, y_c) - r_{\theta}(x, y_r))\right)$$



(2) 通过奖励模型拒绝采样获取监督微调数据

从最新的聊天模型（通常是上一轮后训练最优的检查点）采样 K 个输出（通常在 10 到 30 之间），使用奖励模型打分，选择得分最高的响应。然后将 Prompt 和最优响应作为新的 SFT 样本添加到 SFT 数据集中。

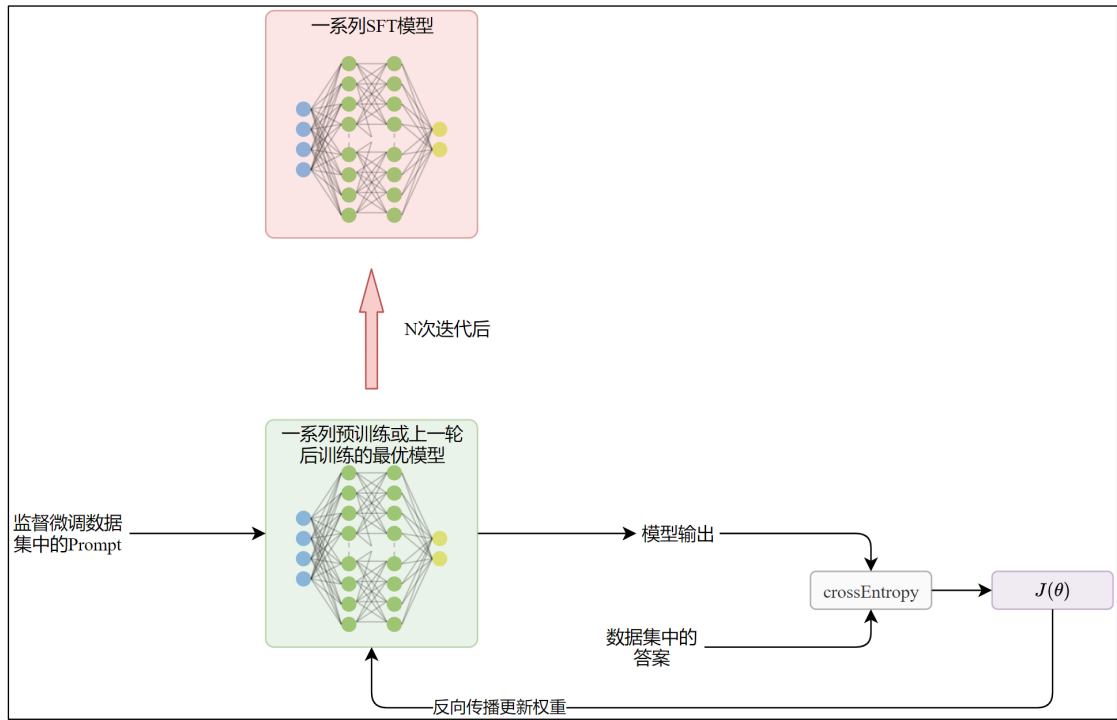


(3) 监督微调

采用标准交叉熵损失基于 SFT 数据集对预训练模型进行监督微调。目标和预训练阶段相同，区别是屏蔽提示词部分的损失（可以通过掩码矩阵实现）。

SFT 数据集包含拒绝采样的数据，但是后者需要依赖最新的聊天模型，因此，最初的监督微调一定不包含拒绝采样的样本，是通过人工标注或者合成的数据开始的。

Llama2 采用迭代式微调，最初的 SFT 一定是基于预训练模型进行的，但是后来的 SFT 都是基于上一轮后训练的最优模型进行。



(4) 直接偏好优化

直接偏好优化（Direct Preference Optimization）是 Rafael Rafailov 等人在《Direct Preference Optimization: Your Language Model is Secretly a Reward Model》中提出的一种非传统强化学习算法。

不同与传统的 RLHF 方法，DPO **无需奖励模型**，通过奖励函数到最优策略的解析映射，将针对奖励函数的损失函数转化为针对策略（正在训练的模型）的损失函数。

DPO 的损失函数如下

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -E_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

最小化损失函数等价于最大化 $\beta \left(\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)$ ，第一项尽可能大，第二项尽可能小。 β 是缩放系数，控制对模型的约束力度。

$$\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} = \log \pi_{\theta}(y_w|x) - \log \pi_{\text{ref}}(y_w|x) - \log \pi_{\theta}(y_l|x) + \log \pi_{\text{ref}}(y_l|x)$$

$\log \pi_{\theta}(y_w|x)$ 表示策略模型输出优选响应的概率

$\log \pi_{\text{ref}}(y_w|x)$ 表示参考模型输出优选响应的概率

$\log \pi_{\theta}(y_l|x)$ 表示策略模型输出弃选响应的概率

$\log \pi_{\text{ref}}(y_l|x)$ 表示参考模型输出弃选响应的概率

上述损失函数的目标等价于

- ① 尽可能增加策略模型输出优选响应的概率
- ② 尽可能减小策略模型输出弃选响应的概率
- ③ 同时限制策略模型的分布偏离参考模型的程度

标准的 DPO 流程如下

- ① 构建偏好数据集

② 根据偏好数据集训练模型，目标是最小化上文提到的损失函数。

Llama3 对于 DPO 的优化

① 屏蔽特殊的格式 token（如起始和终止 token），因为优选和拒绝的响应中有相同的 token 可能会导致矛盾的学习目标。

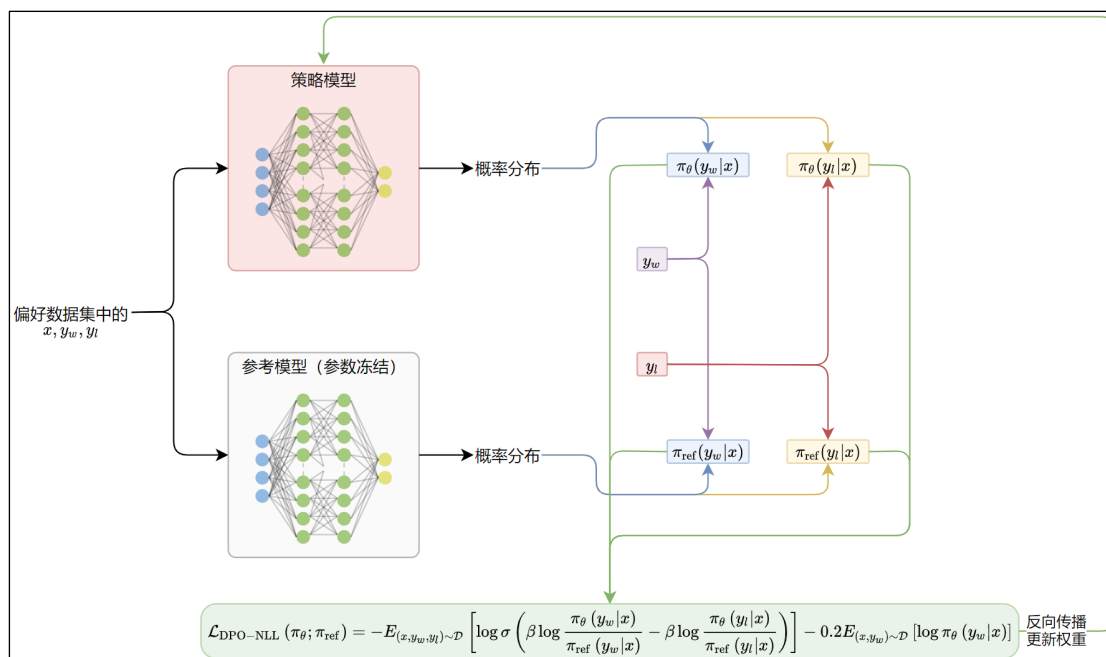
② 添加了一个缩放系数为 0.2，作用于优选序列的交叉熵损失项，有助于保持生成时的理想格式并防止优选响应似然概率的下降，进一步稳定 DPO 训练。

添加 NLL 后的损失函数如下

$$\mathcal{L}_{\text{DPO-NLL}}(\pi_{\theta}; \pi_{\text{ref}}) = -E_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] - 0.2 E_{(x, y_w) \sim \mathcal{D}} [\log \pi_{\theta}(y_w|x)]$$

Llama3 的 DPO 流程

基于最新的 SFT 模型（策略模型），应用前几轮基于最优模型收集的偏好数据，以 $\mathcal{L}_{\text{DPO-NLL}}(\pi_{\theta}; \pi_{\text{ref}})$ 为损失函数，通过反向传播微调模型。需要注意的是，DPO 同样需要参考模型，Llama3 的论文没有提及所用的参考模型，推测**可能为参数冻结的最优模型**。



4) 后训练完整架构

(1) 基于收集的人类偏好数据训练奖励模型

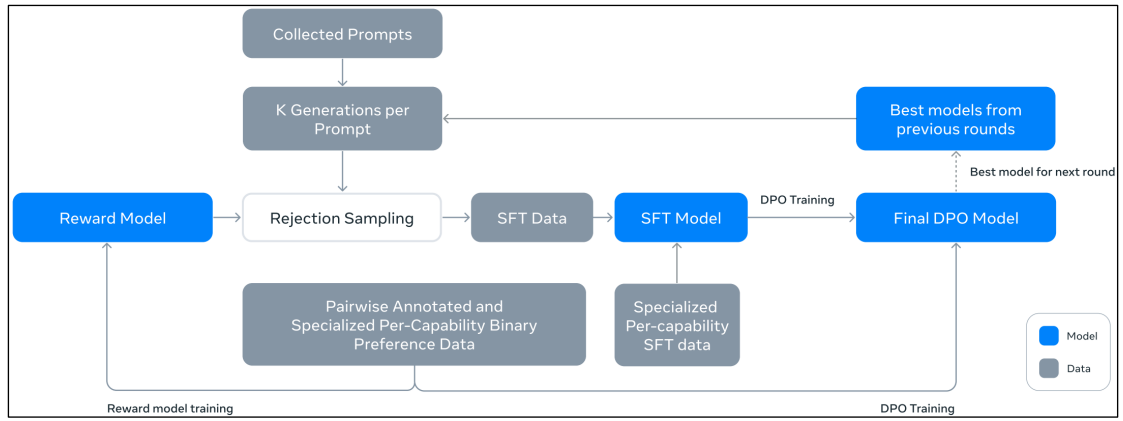
(2) 通过奖励模型对当前最优的模型评分，进行拒绝采样收集 SFT 数据

(3) 进行监督微调

(4) DPO 训练，得到最终的 DPO 模型

(5) 选取 DPO 训练过程中最优的检查点作为下一轮迭代的最优模型

(6) 随着模型训练的进行，后训练阶段的数据也在不断收集，不断用新的数据训练模型，同时用最新的模型生成更多的数据。



第 4 章 Qwen 系列

4.1 Qwen 系列模型发布时间线



4.2 Qwen3

4.2.1 概述

Qwen3 是阿里巴巴旗下千问团队发布的最新模型，由一系列参数量在 6 亿到 2350 亿之间的稠密与混合专家模型。

旗舰模型 Qwen3-235B-A22B 为 MOE 架构，总参数量 2350 亿，每 token 激活参数 220 亿。

1) 知识储备

(1) 思维链

思维链 (Chain-of-Thought, CoT) 是大语言模型 (LLM) 在处理复杂问题时生成的一种逐步推理过程，它通过将问题分解为多个中间步骤，显式地模拟人类逻辑思考的路径，最终得出答案。这一概念最早在 2022 年由 Google Research 的论文《Chain-of-Thought Prompting Elicits Reasoning in Large Language Models》中提出，现已成为提升模型复杂推理能力的核心技术之一。

示例 (数学题)：

问题：小明有 5 个苹果，吃了 2 个，又买了 3 个，现在有多少个？

思维链：

- ① 初始有 5 个苹果 → 5
- ② 吃掉 2 个 → $5 - 2 = 3$
- ③ 买了 3 个 → $3 + 3 = 6$
- ④ 最终答案：6

(2) 长思维链冷启动

长思维链：指那些步骤较多、逻辑链条较长、涉及更深层次推理的思维链。

针对需要多步、复杂推理的任务（如数学证明、复杂问答、代码生成），在微调模型的初始阶段（“冷启动”），就利用详细、逐步的推理过程（“长思维链”）作为监督信号。

将（输入问题，长思维链，最终答案）作为训练样本。模型需要预测长思维链+最终答案的所有 token，而不仅仅是答案。

（3）强-弱蒸馏策略

利用一个或多个能力更强（“强”）的模型（教师）来指导训练一个能力相对较弱（“弱”）的模型（学生）。

知识通过比较教师和学生对相同输入产生的输出来传递。

就像蒸馏的过程，通过加热和冷凝，去除杂质（无关信息），保留精华（关键知识）。在这里，教师的“精华”知识（如输出概率分布、中间层表示、逻辑推理路径）被提取并“灌输”给学生。

损失函数分为两部分

- ① 标准任务损失：学生预测结果与真实标签（硬标签）的损失（如交叉熵）。
- ② 蒸馏损失：学生预测的概率分布与教师预测的概率分布之间的差异（通常用 KL 散度衡量）。

简单理解：让一个“学霸”（强教师）把他解题的思路（不仅是答案，还有排除错误选项的理由）详细地教给一个“普通学生”（弱学生），让学生不仅能答对题，还能像学霸一样思考。

（4）离策略&同策略知识迁移

离策略（off-policy，也叫离线策略）和同策略（on-policy，也叫在线策略）的概念源自强化学习，在知识蒸馏的语境下，区别在于学生模型训练所使用的数据是如何生成的，特别是这些数据是否依赖于学生模型自身当前的行为。

同策略知识迁移：

用于训练学生模型的数据，是在学生模型当前参数状态下，根据其自身行为（或其探索策略）动态生成的。或者说，训练数据依赖于学生模型当前的“策略”。

流程

- ① 学生模型处理一个输入，产生一个输出。
- ② 教师模型评估这个输入和学生模型的输出。
- ③ 教师模型基于此评估生成反馈（如改进的软目标、更优的思维链、奖励信号）。
- ④ 学生模型用这个反馈（作为训练数据）更新自身参数。

离策略知识迁移：

用于训练学生模型的数据是预先收集好的，由一个固定的行为策略生成（通常是教师模型本身的行为，或者一个独立的数据收集策略），与学生模型当前的学习状态无关。数据在训练开始前就准备好了。

流程

- ① 教师模型（或固定策略）处理大量输入，生成软目标/思维链等数据，存储在数据集 D 中。
- ② 学生模型从固定的数据集 D 中采样数据进行训练，更新参数。

类比：

离策略：学生拿着一本老师提前写好的、通用的解题方法大全（固定数据集）自学。

同策略：学生每做一道题，老师就现场批改，并根据学生这道题具体错在哪里，马上给出针对性的详细讲解（动态生成数据），学生立刻根据这个讲解改正学习。

2) 主要成果

将思维模式和非思维模式整合至单一模型中，可以根据任务需求灵活调整。

引入思维预算机制，使用户能精细控制模型执行任务时的推理强度。

支持 119 种语言和方言，大幅增强了多语言处理能力。

3) 训练范式

分为预训练和后训练两部分。

4.2.2 模型架构

1) 稠密模型架构

和传统的 Transformer 相比

- (1) 注意力层采用 GQA。
- (2) FFN（也叫多层感知机层，Multilayer Perception，简称 MLP）层间 SwiGLU 激活函数。
- (3) 位置编码使用 RoPE
- (4) 使用 RMSNorm，预归一化
- (5) 在注意力机制中引入 QK 归一化。

稠密模型关键超参数				
模型	解码器层数	头数(Q/KV)	输出层是否共享嵌入矩阵	上下文长度
Qwen3-0.6B	28	16/8	Yes	32K
Qwen3-1.7B	28	16/8	Yes	32K
Qwen3-4B	36	32/8	Yes	128K
Qwen3-8B	36	32/8	No	128K
Qwen3-14B	40	40/8	No	128K
Qwen3-32B	64	64/8	No	128K

2) MOE 模型架构

与稠密模型共享相同的基础架构，共包含 128 个专家，每个 token 激活 8 个专家。

MOE 模型关键超参数				
模型	解码器层数	头数(Q/KV)	专家数(总量/激活)	上下文长度
Qwen3-30B-A3B	48	32/4	128/8	128K
Qwen3-235B-A22B	94	64/4	128/8	128K

4.2.3 预训练

4.2.3.1 数据准备

数据集约包含 **36T token**。

(1) 基于 Qwen2.5-VL 从海量 PDF 中提取文本，随后使用 Qwen2.5 进行精炼处理以提升质量。通过这一步获取了数万亿 (T) 高质量 token。

(2) 使用 Qwen2.5-Math、Qwen2.5-Coder 等领域专用模型合成数据，涵盖教材、问答、指令和代码片段等数十个领域。这一步获取了数万亿 (T) 不同格式的 token。

(3) 整合多语言数据并引入更多语种进一步扩充预训练语料库。

(4) 开发了一套多语言数据标注系统，从教育价值、领域、专业方向及安全性等多个维度标注了超过 30 万亿 token。

4.2.3.2 训练流程

预训练分为三个阶段

(1) 第一阶段用约 30T token 建立通用知识基础。这一阶段所有模型使用 4096 (4K) token 的上下文，训练数据涵盖 119 种语言及方言。

(2) 第二阶段聚焦 STEM (Science, Technology, Engineering, and Mathematics Data, 通常是指与科学、技术、工程和数学领域相关的结构化数据) 和编程等知识密集型数据以增强推理能力。这一阶段上下文长度为 4K, 使用了约 5T 更高质量的 token。

(3) 第三阶段通过长上下文训练将最大上下文长度从 4K 扩展到 32,768 token (32K)。

所有模型均在数万亿 token, 32K 的序列长度下训练。长上下文语料包含 75% 长度介于 16,384 (16K) 至 32K 个 token 的文本, 以及 25% 长度介于 4K 至 16K 个 token 的文本。

采用 ABF 技术, 将 RoPE 的基础频率从 10,000 提升至 1,000,000。

4.2.4 后训练

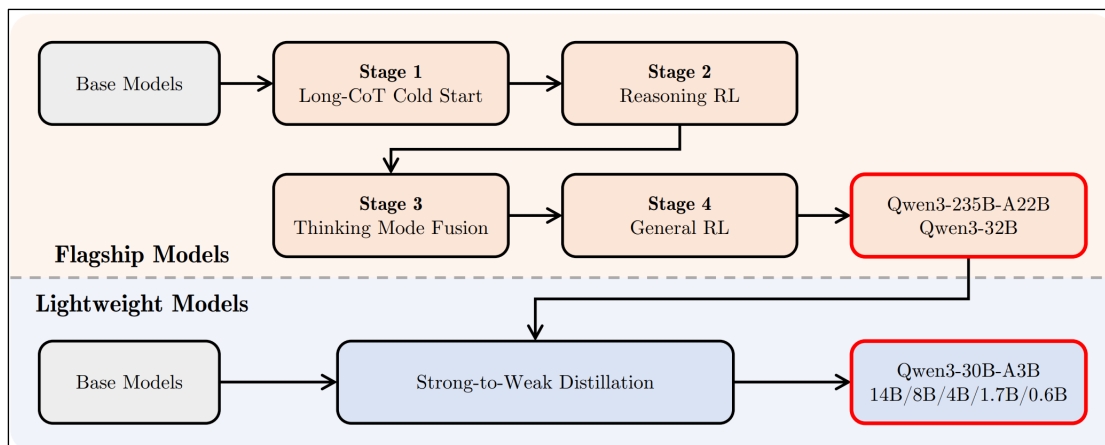
4.2.4.1 概述

1) 训练方式

(1) Qwen3 的旗舰模型 Qwen3-235B-A22B 和 Qwen3-32B 采用标准的四阶段后训练。

(2) 轻量模型通过强-弱蒸馏提炼旗舰模型的知识得到。

整体架构如下。



2) 旗舰模型的后训练

后训练分为四个阶段：长思维链冷启动、推理强化学习、思维模式融合（SFT）、通用强化学习。

（1）前两个阶段通过**长思维链冷启动微调**和**针对数学编程任务的强化学习**重点培养推理能力。

（2）后两阶段将含推理路径与不含路径的数据合并进行统一微调，使模型能同时处理两类输入，再通过通用领域强化学习提升多任务表现。此外，对于小模型，采用强-弱蒸馏策略，结合离策略和同策略知识迁移从大模型中提炼知识，提升自身性能。

4.2.4.2 旗舰模型的数据准备

1) 长思维链冷启动

研究团队构建了一个涵盖数学、编程、逻辑推理和 STEM 通用问题的综合性数据集。每个问题都配有经过验证的参考解答或基于代码的测试用例。

数据集构建采用两阶段过滤：问题筛选和回答筛选

（1）问题筛选

使用 Qwen2.5-72B-Instruct 模型识别并剔除不易验证的查询，同时排除无需思维链即可正确回答的问题

（2）回答筛选

使用 QwQ-32B 为每个查询生成 N 个候选回答，如果这些回答全部错误，标注员介入。否则按照以下标准进一步筛选：

- ① 最终答案错误
- ② 存在大量重复
- ③ 明显猜测缺乏充分推理
- ④ 思维过程与结论不一致
- ⑤ 不恰当的语言混用或风格突变
- ⑤ 疑似与验证集过度相似

随后精选部分优化后的数据集用于冷启动训练。

2) 推理强化学习

此处使用的问答对要满足四个条件

- (1) 未在冷启动阶段使用过
- (2) 不能太难，以至于冷启动模型无法学习
- (3) 在 (2) 的基础上，尽可能具有挑战性
- (4) 覆盖广泛的子领域

最终收集了 3995 组数据。

3) 思维模式融合

数据集由两部分构成：

- (1) 思考型数据

通过第二阶段模型对第一阶段模型拒绝采样生成

- (2) 非思考型数据

原文没有提到这部分数据的来源。只提到这部分数据经过了精心筛选，涵盖编码、数学、指令遵循、多语言任务、创意写作、问答及角色扮演等多样化任务。

对话模板设计，为了更好地整合两种模式并支持用户动态切换模型的思考过程，研究团队设计了下表所示的对话模板

Thinking Mode	Non-Thinking Mode
< im_start >user {query} /think< im_end > < im_start >assistant <think> {thinking_content} </think> {response}< im_end >	< im_start >user {query} /no_think< im_end > < im_start >assistant <think> </think> {response}< im_end >

通过再查询消息中引入/think 和/no_think 标志，使得模型能够根据用户输入选择相应的思考模式。

4) 通用强化学习

数据来源未披露。

4.2.4.3 旗舰模型的训练流程

1) 长思维链冷启动

将输入+思维链+输出喂给模型，建模目标是预测思维链+输出，从而获得基础的推理能力。

2) 推理强化学习

采用 GRPO 算法更新模型参数。

3) 思维模式融合

这一阶段从技术上讲就是 SFT，乏善可陈。

4) 通用强化学习

通用强化学习阶段旨在广泛提升模型在多样化场景中的能力与稳定性。

研究团队建立了覆盖 20 余项专项任务的精细化奖励体系，每个任务均设有定制化评分标准。这些任务重点强化以下核心能力：

(1) 指令遵循：确保模型能准确理解并执行用户指令，包括对内容、格式、长度及结构化输出的要求，提供符合用户预期的响应。

(2) 格式规范：除显式指令外，模型需遵循特定格式约定。例如根据/`think` 和/`no_think` 标志切换思考模式与非思考模式，并在最终输出中始终使用指定标记（如`<think>`和`</think>`）分隔思考过程与应答内容。

(3) 偏好对齐：针对开放式查询，通过提升模型的有用性、互动性和表达风格，最终呈现更自然流畅的用户体验。

(4) 智能体能力：训练模型通过指定接口正确调用工具。在强化学习推演过程中，允许模型与真实环境执行反馈进行完整的多轮次交互循环，从而提升其在长周期决策任务中的表现和稳定性。

(5) 专业场景能力：在专业场景中设计了特定任务。例如在检索增强生成（RAG）任务中，引入奖励信号引导模型生成准确且符合语境的响应，从而降低幻觉风险。

采用了三种不同类型的奖励机制

(1) 基于规则的奖励

(2) 基于参考答案的奖励机制

为每个查询提供参考答案，引导 Qwen2.5-72B-Instruct 根据参考对模型响应评分

(3) 基于模型的奖励

对于没有参考答案的查询，基于人类偏好数据训练奖励模型，为策略模型的响应打分。

4.2.4.4 强模型到弱模型的蒸馏流程

强模型是指旗舰模型（教师模型），弱模型是指轻量模型（学生模型），包括 5 个稠密模型（Qwen3-0.6B、1.7B、4B、8B 和 14B）以及 1 个 MOE 模型（Qwen3-30B-A3B）。

蒸馏分为两个阶段：

1) 离策略蒸馏

将教师模型通过思考和非思考两种模式生成的输出结果与查询进行整合，对学生模型进行监督微调。损失函数分为学生模型预测思维链（如果有的话）+教师模型响应的交叉熵损失。

这一阶段是为了培养学生模型的基础推理能力以及在不同思维模式之间切换的能力。

2) 同策略蒸馏

学生模型和教师对于相同的查询生成概率分布，这一阶段的损失函数是学生模型和教师模型的 KL 散度。

3) 总结

离策略蒸馏相当于老师给了习题和答案，学生对着习题记答案，学习基础能力。同策略蒸馏相当于学生在具备一定解题能力之后，对没见过的问题输出思维过程和答案，然后和老师的思维过程和答案对齐，进一步提升解题能力。

第 5 章 附录

5.1 Common Crawl 介绍

5.1.1 简介

Common Crawl 是一个开源爬虫项目，维护了一个免费、对所有人开放的网路爬虫数据仓库。官网：
<https://commoncrawl.org/>

5.1.2 数据量

成立于 2007 年，在过去的 18 年间，爬取了超过 2500 亿个网页，提供完全免费和开源的语料库，超过 10000 篇论文或研究报告使用了 Common Crawl 的数据集。

(1) 每月新增 30-50 亿网页

(2) 截至目前，2025 年 1-4 月每月新增网页数约为 30 亿，每月压缩后的原始数据约为 100TB，压缩后的纯文本数据不到 10TB。

5.1.3 入门指南

任何人都可以免费从任何地方获取 Common Crawl 的数据。

数据由亚马逊网络服务的开放数据集赞助计划托管，存储的桶为`s3://commoncrawl/`，位于美国东部 1 区（北弗吉尼亚）AWS 区域。

若需将数据下载至本地机器或本地集群，可使用任何 HTTP 下载工具（如 cURL 或 wget）。数据可通过 [https://data.commoncrawl.org/\[...\]](https://data.commoncrawl.org/[...]) URL 格式访问。

5.1.4 数据类型

25 年 4 月新增数据概览如下

Common Crawl April 2025 Crawl Archive (CC-MAIN-2025-18)			
The April 2025 crawl archive contains 2.74 billion pages, see the announcement for details.			
Data Size and File Listings			
Data Type	File List	#Files	Total Size Compressed (TiB)
Segments	segment.paths.gz	100	
WARC	warc.paths.gz	100000	99.00
WAT	wat.paths.gz	100000	18.49
WET	wet.paths.gz	100000	7.35
Robots.txt files	robotstxt.paths.gz	100000	0.15
Non-200 responses	non200responses.paths.gz	100000	3.30
URL index files	cc-index.paths.gz	302	0.21
Columnar URL index files	cc-index-table.paths.gz	900	0.24

数据类型介绍如下

数据类型	主要用途	适用场景
WARC	原始网页存档	网页内容分析、数据恢复
WAT	链接和元数据分析	网络图谱构建、爬虫优化
WET	文本处理	NLP、机器学习训练
CDX	快速 URL 查找	数据检索、去重
元数据	统计分析	数据质量评估、采样
Segments	划分抓取分片，协调分布式处理	大数据任务分片调度
Robots.txt	记录网站爬取规则	爬虫合规性、反爬策略研究
Non-200 Responses	标记抓取失败的页面	链接失效分析、网络健康监测
URL 索引(CDX)	快速 URL 定位	小规模精确检索(如特定网页历史版本)
URL 索引(Parquet)	高效聚合分析	统计域名覆盖率、内容类型分布

1) WARC (Web ARChive) 文件

- (1) 格式: .warc 或.warc.gz (压缩)
- (2) 用途: 存储原始爬取的网页内容, 是 Common Crawl 的核心数据格式。
- (3) 内容:
 - WARC 头: 包含元数据 (URL、抓取时间、MIME 类型等)。
 - HTTP 响应头: 服务器返回的 HTTP 状态码和头部信息。
 - 页内容: HTML、二进制文件 (如图片、PDF) 等原始数据。
- (4) 常见字段:

WARC-Type:response
WARC-Target-URI:<URL>
WARC-Date:<抓取时间戳>
Content-Type:<MIME 类型>

2) WAT (WebArchiveTransformation) 文件

- (1) 格式: .wat.gz (压缩的 JSON 格式)
- (2) 用途: 提供 WARC 文件的元数据和结构化信息, 不含原始内容, 用于快速分析链接和文本特征。
- (3) 内容:
 - 提取的元数据 (如链接、锚文本、HTTP 头)。
 - 使用 JSON 格式存储, 每条记录对应一个 WARC 记录。

(4) 示例字段:

```
{
  "Envelope":{
    "Payload-Metadata":{"HTTP-Response-Metadata":{"Headers":{"...}}},
    "WARC-Header-Metadata":{"WARC-Target-URI":"https://example.com"}
  }
}
```

3) WET (WebExtractText) 文件

(1) 格式: .wet.gz (压缩的纯文本)

(2) 用途: 从 WARC 文件中提取的纯文本内容, 去除了 HTML 标签, 适用于自然语言处理 (NLP) 任务。

(3) 内容:

- 每行对应一个 WARC 记录, 包含 URL 和提取的文本。
- 文本经过简单清洗 (如去除导航栏、广告等重复内容)。

(4) 示例:

WARC/1.0

WARC-Type:conversion

WARC-Target-URI:<URL>

Content-Type:text/plain

Thisistheextractedtextfromthewebpage...

4) URL 索引文件(CDX)

(1) 格式: .cdx 或.cdx.gz

(2) 用途: 提供 WARC 文件中 URL 的索引, 支持快速查找特定网页。

(3) 内容:

- 每行包含 URL、时间戳、WARC 文件偏移量等。
- 格式为空格分隔的字段:

<URL><时间戳><WARC 文件名><偏移量><压缩类型>

(4) 示例:

com,example)/20200101000000example.warc.gz1234gzip

5) 元数据文件(列式格式)

(1) 格式: Parquet/JSON

(2) 用途: 存储数据集的高级统计信息 (如抓取日期、域名分布、语言检测结果)。

(3) 内容:

- 列式存储 (Parquet) 优化分析性能。
- 可能包含网页数量、语言分布、内容类型统计等。

6) Segments('segment.paths.gz')

(1) 格式: GZIP 压缩的文本文件

(2) 用途:

➤ 存储数据分片 (Segment) 的路径列表, 每个分片对应一个独立的抓取单元 (通常包含多个 WARC/WAT/WET 文件)。

- 用于协调分布式处理, 例如 MapReduce 任务的分片划分。

(3) 特点:

仅包含文件路径, 无实际内容数据。

(4) 示例内容:

crawl-data/CC-MAIN-2025-18/segments/145623456789.12/

crawl-data/CC-MAIN-2025-18/segments/145623456790.34/

7) Robots.txt 文件('robotstat.paths.gz')

(1) 格式: GZIP 压缩的文本文件 (路径列表)

(2) 用途:

- 列出所有抓取的 'robots.txt' 文件路径, 这些文件记录了网站的爬取规则。
- 用于分析爬虫合规性, 或研究网站管理员对爬虫的限制策略。

(3) 关联数据:

- 实际文件内容存储在对应的 WARC 文件中 (通过路径定位)。

8) Non-200 响应('non200responses.paths.gz')

(1) 格式: GZIP 压缩的文本文件 (路径列表)

(2) 用途:

- 记录所有 HTTP 状态码非 200 (如 404、301、503 等) 的网页响应路径。
- 用于分析抓取失败原因、链接失效统计或网络拓扑变化。

(3) 注意:

- 实际响应内容需通过路径找到对应的 WARC 文件解析。

9) URL 索引文件(`cc-index.paths.gz`和`cc-index-table.paths.gz`)

(1) 统一 CDX 索引(`cc-index.paths.gz`)

① 格式: GZIP 压缩的文本文件 (CDX 格式)

② 用途:

- 提供 URL、时间戳、WARC 文件偏移量的映射, 支持快速检索特定 URL。
- 适用于小规模精确查询。

(2) 列示索引(`cc-index-table.paths.gz`)

① 格式: Parquet 列式存储 (通过路径列表定位)

② 用途:

- 优化大规模分析 (如 Spark、Hive), 支持高效聚合查询 (如按域名、语言筛选)。
- 包含增强字段 (如内容语言、MIME 类型、跳转链)。

5.2 基于哈夫曼树的分层 softmax

分层 softmax, 英文全称为 hierarchical softmax

5.2.1 实现思路

1) 构建哈夫曼树

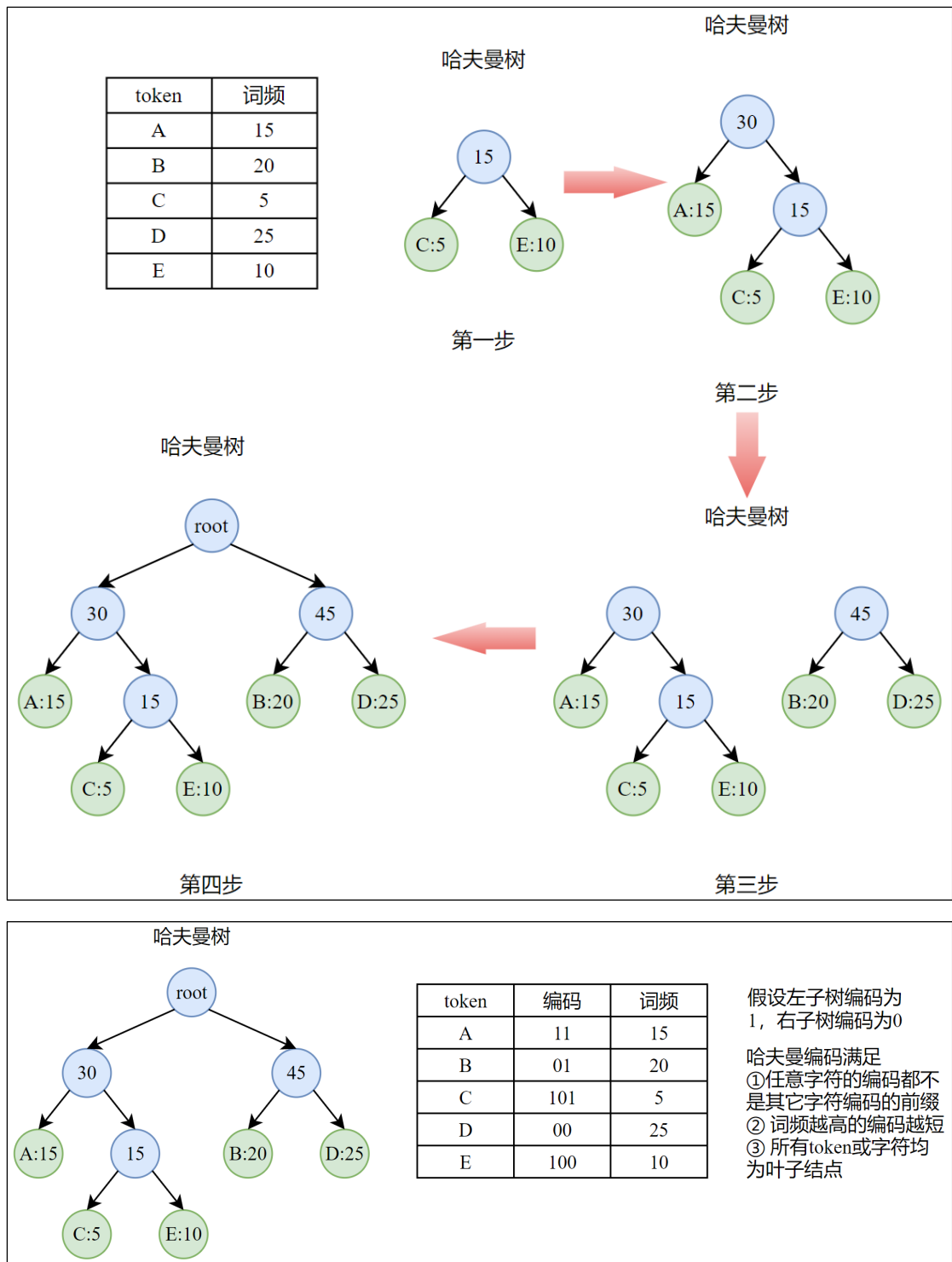
(1) 遍历训练或测试的全量文档, 统计所有词的词频

(2) 按照哈夫曼树的构建思路基于词频将所有词构建为一棵哈夫曼树

具体来说, 每次选取频次最小的两个节点合并为一个结点, 新结点的频次等于原先的两个节点频次之和。然后从列表中将原先的两个节点移除, 将新的节点加入。循环这一过程, 直至只有一个节点。

除了叶子结点, 每个哈夫曼树的结点都有一个向量权重, 维度与隐藏层的维度相同。

哈夫曼树处理完成后, 每个词都有唯一的编码, 且任何一个词的编码都不是其它词的编码的前缀。



注意：构建完成后，通常会将词到路径的映射（包括路径节点序列和每个节点的方向 dir）存储起来，供训练和推理时快速查找。

2) 概率计算过程

分层 softmax 的精髓是将 softmax 算法转换为沿路径的一系列 sigmoid。

首先假设到达目标 token w 的路径 $path(w) = \{n_1, n_2, \dots, n_L\}$ ，最后一个结点为 token 对应的叶子结点，除此之外的结点都对应一个维度为 d ，**可学习**的权重向量 w_{n_j} 。

(1) 路径上每一个结点对应一个二分类概率

$$P(n_j \rightarrow n_{j+1} | h) = \sigma\left(\left(h^T w_{n_j}\right) \cdot \text{dir}(n_j)\right)$$

其中， $\text{dir}(n_j) \in \{1, -1\}$ 表示从 n_j 向下的路径方向，向左为1，向右为-1。

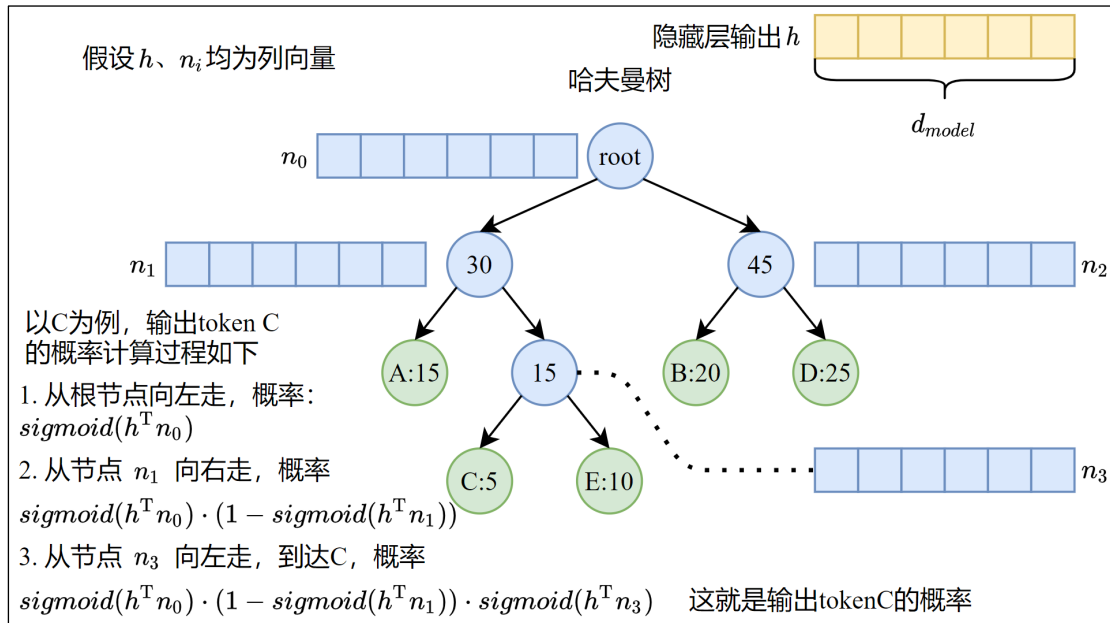
此处的 σ 是 sigmoid 函数的简写。

上式表示每个非叶子结点向左子树前进的概率为 $\text{sigmoid}(h^T w_{n_i}) = \frac{1}{1+e^{-h^T w_{n_i}}}$ ，向右前进的概率为 $1 -$

$$\text{sigmoid}(h^T w_{n_i}) = 1 - \frac{1}{1+e^{-h^T w_{n_i}}}。$$

(2) 最终某个词的概率为路径上所有概率的乘积

$$P(w|h) = \prod_{j=1}^{L-1} \sigma\left((h^T w_{n_j}) \cdot \text{dir}(n_j)\right)$$



5.2.2 训练阶段时间复杂度分析

我们对于单个词概率的计算进行时间复杂度分析。

传统的 sigmoid ，在计算概率之前要通过线性层将维度由隐藏层维度 d_h 转换为词表维度 d_{vocab} ，那么时间复杂度为 $O(d_h \times d_{vocab})$ ，至于最终的 sigmoid 计算，时间复杂度为 $O(d_{vocab})$ ，综上，时间复杂度为 $O(d_h \times d_{vocab})$ 。

而基于哈夫曼树的分层 softmax ，只需要计算沿着预先缓存的目标 token 路径，计算其概率即可。路径不超过树高，哈夫曼树是二叉树的一种，假设叶子结点个数为 n ，理想情况下二叉树的树高是 $\log_2 n$ ，越不平衡树高越大，所有非叶子结点都只有一个子结点时退化为链表，此时树高为 n 。哈夫曼树通常是不平衡的二叉树，在自然语言的词频分布下，树高接近 $\log_2 n$ ，二者在同一数量级。因此，分层 softmax 计算的时间复杂度为 $O(d_h \times \log_2(d_{vocab}))$ 。

效率显著提升。

5.2.3 训练阶段损失计算

训练阶段会记录所有 token 的路径信息，通常直接检索目标 token 的路径，计算目标概率，然后通过交叉熵定义损失函数

$$\mathcal{L} = -\log(P(w|h))$$

$$\mathcal{L} = - \sum_{i=1}^{L-1} \log \left(\sigma \left(\left(h^T w_{n_j} \right) \cdot \text{dir}(n_j) \right) \right)$$

5.2.4 推理阶段时间复杂度分析

1) 直接计算

推理阶段并不知道目标 token，如果像传统的 softmax 一样计算每个 token 的概率，那么时间复杂度将是 $O(d_{vocab} \times d_h \times \log_2(d_{vocab}))$ ，甚至要高于传统的 softmax，显然是不行的。

2) 优化

(1) 概率剪枝 (Beam Search, 束搜索)

思想：

在哈夫曼树的搜索过程中动态保留概率最高的 k 条路径，逐步扩展

优点：

减少计算量，找到近似的全局最优解，以精度换时间

过程：

束宽 (beam Width) 为 k

1. 从根结点开始，向下搜索候选集的每个子结点，初始候选集为根结点本身。

2. 保留扩展后的候选集中概率最高的 k 个结点 (不超过 k 则全部保留)

当前结点 n_j 的概率为 $P(n_j) = P(n_{j-1}) \cdot \sigma \left(\left(h^T w_{n_{j-1}} \right) \cdot \text{dir}(n_{j-1}) \right)$

3. 重复这一过程，保留最终的 k 个结点，即 k 个候选 token

最终，通过贪心策略取概率最大的 token 作为输出，或通过概率采样确定输出的 token

如果 k=1 则退化为贪心搜索，每一步选局部最优，但可能错过全局最优

k 越大，结果越接近全局最优，但计算开销越高

哈夫曼树的优化：高频词路径短，可能优先被保留，低频词概率大，可能被剪枝。

(2) 按概率采样 (贪心搜索)

每层处理为简单的二分类，选择概率最大的子树，最终选择为局部最优，可能并非全局最优

(3) 近似最近邻 (Approximately Nearest Neighbors Search, ANN)

略。

5.3 FastText 模型

5.3.1 简介

FastText 是 Facebook 人工智能研究院提出一种简单高效的模型，可以用于 **文本分类问题** 或 **构建词表征**，训练和推理都是在 CPU 上完成的。

5.3.2 核心思想

假设一个序列由 N 个 token 构成 $X = \{w_1, w_2, w_3, \dots, w_N\}$

1) 通过 n-gram 将一个 token 分解为多个子词

通常在 token 前后添加特殊边界符号<和>, 然后提取长度为 min_n 和 max_n 的所有子串。其中, min_n 和 max_n 都是超参数, 分别表示子词的最小和最大长度。

如 apple 的 3-gram 子词最小长度和最大长度均为 3, 分词结果为

```
<ap, app, ppl, ple, le>
```

2) 获取 token 和子词的嵌入

(1) 过程

通过类似于字典的数据结构直接获取子词对应的嵌入, 如果 token 本身也有嵌入的话也拿过来(训练阶段一定有, 推理阶段未必有, 因为可能是 OOV 词), 然后求和, 得到 token 最终的嵌入

数学表示如下。

子词和 token 有单独的词表

子词嵌入矩阵为 $E_s \in R^{|V_s| \times d}$, 其中 $|V_s|$ 是子词词表大小, 当采用哈希桶精简子词数量时, $|V_s|$ 是子词哈希桶的数量, 而非子词数量, d 是向量维度

token 嵌入矩阵为 $E_t \in R^{|V_t| \times d}$, 其中 $|V_t|$ 是 token 词表大小, d 是向量维度

那么, 用 v_{w_i} 作为 token w_i 的最终表示, $G(w_i)$ 表示其所有子词的集合, 则有下式

$v_{w_i} = z_t + \sum_g z_g$, 其中 z_t 表示 token 的词表征, g 表示子词, z_g 表示子词的表征

(2) 优化

通过哈希桶优化子词嵌入的获取, 子词哈希值相同的对应同一个向量。主要是通过缩减子词嵌入矩阵的总量实现优化。**这会引入哈希冲突, 但实践表明是可接受的代价。**

3) 通过平均池化计算整个文档的等效嵌入

$$\vec{h} = \frac{1}{N} \sum_{i=1}^N v_{w_i}, \vec{h} \in R^d$$

此处的平均池化实际上是求文档中所有 token 表示的均值

4) 输出层

FastText 有两种典型的应用场景: 文档分类和词表征。这两种场景下模型的唯一区别在于隐藏层之后的线性层, 用作文档分类时该线性层权重矩阵的形状为 $d \times C$, d 是隐藏层维度, 而 C 是类别数。用于训练词表征时, 形状为 $d \times |V_t|$, $|V_t|$ 表示表示词表大小。

这两种应用场景下输出层的优化方式都是相同的。此处文档分类问题为例说明。

(1) 如果没有优化

为便于表示, 此处规定 $\vec{h}$ 为行向量

通过传统的输出权重矩阵+softmax 计算文档类别概率

对于单个文档的特征表示 $\vec{h}$ ，首先经过线性层处理得到输出向量 $\vec{y}$

$$\vec{y} = \vec{h} \cdot W_{d \times c}$$

然后通过 softmax 得到概率，对于 $\vec{y}$ 中的每一个元素 y_i ，对应的概率如下所示

$$prob_i = \frac{e^{y_i}}{\sum_{i=1}^d e^{y_i}}$$

这样的计算过程时间复杂度为 $O(C \cdot d)$

(2) 分层 softmax 优化

fasttext 采用了高效的基于哈夫曼树的分层 softmax (hierarchical softmax)，计算过程见上文。

(3) 负采样优化

为便于表示，此处规定 $\vec{h}$ 为列向量

负采样优化将 softmax 的多分类问题转换为二分类，因此激活函数替换为 sigmoid，训练阶段将目标 token 作为正例，其余的均为负例，选取若干负例，基于正例和负例计算交叉熵即可，优化了损失函数的计算过程

在 fasttext 分类问题中，输出权重矩阵形状为 $d \times C$ ，该矩阵的每一列对应一个类别的类别嵌入，表示为 $C = \{c_1, c_2, \dots, c_n\}$ ，目标类别为 c_o

负采样将问题转化为二元分类

➤ 正样本: (C, c_o)

➤ 负样本: (C, c_n)

定义单个类别的概率模型，即对于特定类别 c_i ，它为正例和负例时的概率分别如下所示

$D = 1$ 表示 c_i 为正样本 $D = 0$ 表示 c_i 为负样本

$$P(D = 1|C, c_i) = \sigma(h^T u_{c_i}) = \frac{1}{1 + \exp(-h^T u_{c_i})}$$

$$\begin{aligned} P(D = 0|C, c_i) &= 1 - \sigma(h^T u_{c_i}) \\ &= \sigma(-h^T u_{c_i}) \end{aligned}$$

负采样优化步骤

1. 正例只有一个

2. 从非目标类别中采样 k 个负例

3. 损失函数

$$\mathcal{L} = -\log(P(D = 1|C, c_o)) - \sum_{i=1}^k \log(P(D = 0|C, c_n))$$

$$\mathcal{L} = -\log(\sigma(h^T u_{c_o})) - \sum_{i=1}^k \log(\sigma(-h^T u_{c_{n_i}}))$$

此时只需要计算一个正例和 k 个负例

时间复杂度变为 $O(d \cdot (k + 1))$

以精度为代价，大幅降低计算开销。

5.3.3 优点

1) 高效训练与推理

- (1) 分词时通过哈希桶减少参数量
- (2) 分层 Softmax/负采样大幅降低时间复杂度

2) 强大的未登录词 (OOV) 处理能力

将 token 分解为子词，当 token 为 OOV 时，可以用其子词的求和作为 token 的表征。

3) 词向量质量优异

相同词根可以共享子词向量，可以更好捕捉形态学特征，低频次也可以受益于高频子词，获得准确的表示

4) 简洁统一的架构

分类和词向量训练共用同一套架构。

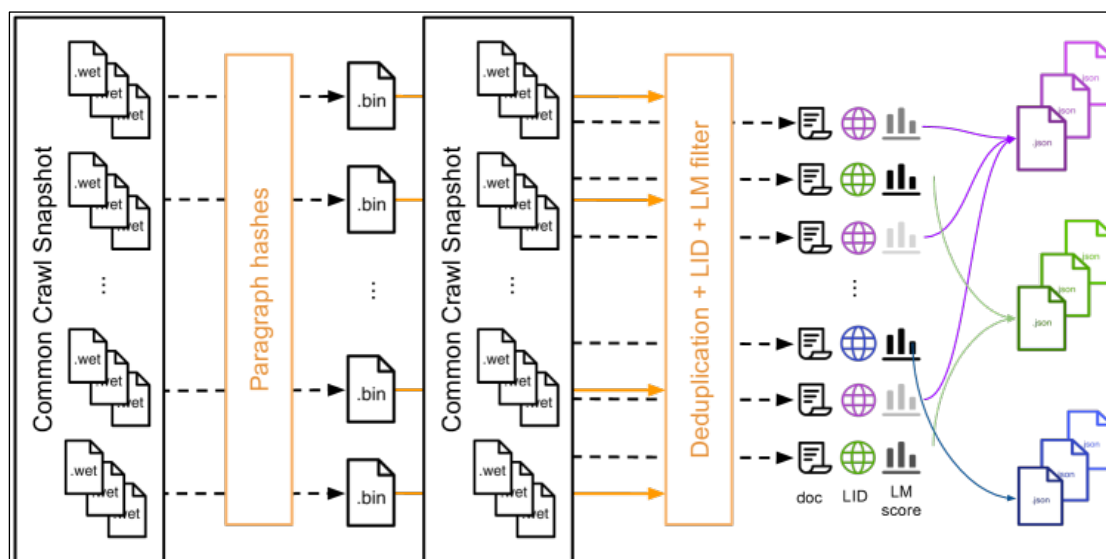
5) 分类性能卓越

通过平均池化聚合所有词向量获得词序无关的文档表征，这一特性在垃圾邮件检测、主题分类中表现鲁棒。在短文本分类任务上表现优异。

5.4 CCNet

Facebook 人工智能研究院提出的一种专门用于清洗 Common Crawl 数据集的神经网络。

5.4.1 架构



5.4.2 处理流程

1) 预处理 (Preprocessing)

将 WET 文件重新分组为 5GB 大小的分片 (shard)，每个分片以 JSON 格式存储数据，每条记录对应一个网页，分片是被压缩存储的，即 5GB 是压缩后的大小。

对应上图中最左侧的 .wet 文件。

2) 计算段落哈希

这一步的标准化处理、哈希值计算都是基于数据集的副本进行的，下游模型仍然可以看到原本的数据。

(1) 标准化处理

将所有字符转为小写，用占位符（即 0）替换数字，并移除所有 Unicode 标点符号和重音标记

将数字全部替换为 0 是为了数字不同的相同语义文本被判定为不同的文本。此处的处理只是为了计算哈希值用于去重，下游模型看到的是原始文本。

(2) 计算段落哈希

针对每个数据分片（shard），计算每个段落（each paragraph）的哈希码，并将其存入二进制文件（.bin 后缀的文件）。使用的哈希算法是 SHA-1，将 SHA-1 计算结果的前 64 位作为键用于去重。

上图中.bin 文件中记录的就是段落哈希。

3) 去重 (Duplication)

去除快照中不同网页间的重复段落

基于上一步得到的哈希 key，将每个分片与一个、部分或全部分片对比，实现分片级去重。

对应右侧的 Duplication。

4) 语言识别 (Language Identification, LID)

这一步是基于去重后的数据处理

使用 fasttext 语言分类器，后者支持 176 种语言分类，输出[0,1]区间的置信度分数。

对于每个网页，计算最可能的语言及对应分类分数，如果得分超过 0.5 则归入相应语言类别，否则视为无法识别丢弃页面。

对应右侧的 LID 和类似于网络的图标。

5) 语言模型过滤 (LM Filtering)

这一步基于 LID 后的数据处理

(1) 通过目标语言的高质量语料库（官方在 48 种语言的维基百科数据集上训练）训练了一个分词器和 5-gram Kneser-Ney 语言模型（一种 n-gram 模型）

(2) 首先对数据集中的每个页面执行分词操作

(3) 然后通过 5-gram Kneser-Ney 语言模型计算段落的困惑度

(4) 根据困惑度将数据划分为三类，head、middle 和 tail，困惑度依次增大。每种语言对应各自的 head、middle 和 tail 集合

对应上图中右侧的 LM Filter 和类似于柱状图的图标，清洗后的文件通过紫、绿、蓝三种不同的颜色标识，用同色线标识数据流向，对应不同困惑度的文件。这是清洗的最后一个环节，得到的文件直接用于下游处理。

5.5 旋转位置编码

旋转位置编码 (Rotary Position Embedding, RoPE) 是国内大神[苏剑林提出](#)的，旨在解决正余弦位置编码的不足。

5.5.1 正余弦位置编码的不足

1) 原版的正余弦位置编码只是与词向量简单相加，破坏了原始语义，并导致位置信息在注意力计算中被“稀释”

原版 PE 是直接词嵌入 e 相加 $h = e + p$

在注意力计算时， $q_i = (e_i + p_i) \cdot W_q$ ， $k_j = (e_j + p_j) \cdot W_k$

注意力分数 $q^T \cdot k$ 展开后包含四项 $W_q^T \cdot e_i^T \cdot e_j \cdot W_k$ (词-词交互) + $W_q^T \cdot e_i^T \cdot p_j \cdot W_k$ (词-位置交互) + $W_q^T \cdot p_i^T \cdot e_j \cdot W_k$ (位置-词交互) + $W_q^T \cdot p_i^T \cdot p_j \cdot W_k$ (位置-位置交互)。

模型必须同时学习区分词义交互、位置交互以及它们的交叉项，增加了学习难度。更重要的是，位置信息 p_i 和 p_j 被投影并与其它项混合，导致位置信息在最终的注意力分数中被稀释和干扰，模型难以精确捕捉纯粹的位置关系，尤其是相对位置。

2) 绝对位置编码对相对位置关系表达不够直接和鲁棒

虽然理论上正余弦编码允许模型学习相对位置，因为 $PE(pos + k)$ 可以表示为 $PE(pos)$ 的线性变换，但这种表示是隐式的，模型需要通过注意力层的权重 W_q 和 W_k 来学习如何从绝对位置中推导出相对位置关系。这个过程并非显式保证，增加了模型学习的负担和不确定性。

3) 长度外推性差

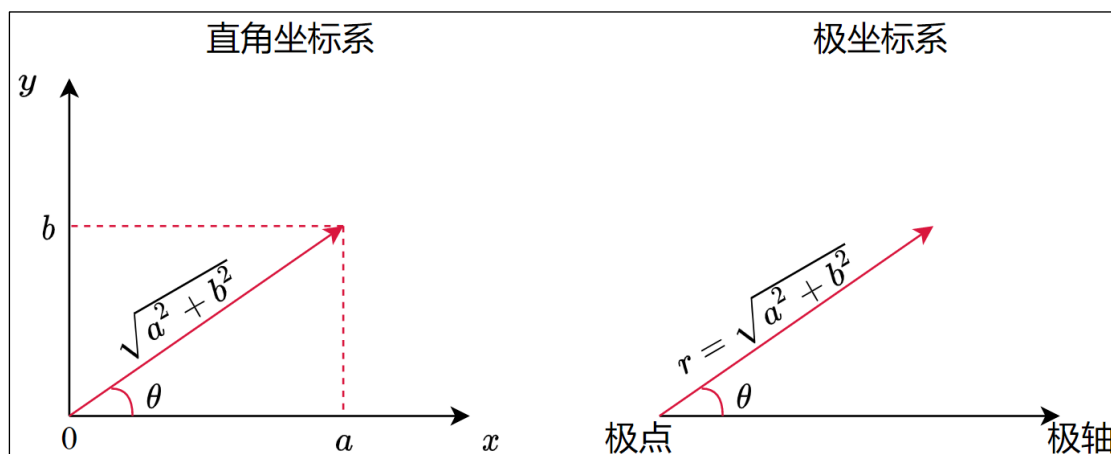
模型在训练时只见过最大长度（如 512、1024）以内的位置编码

当推理遇到显著长于训练最大长度的序列时，模型看到的 pos 值远大于训练时见过的值。

5.5.2 二维向量和复数极坐标系相关知识

二维向量可以被视为一个复数，具体来说对于向量 (a, b) ，可以表示为复数 $a + b \cdot i$ ，其中 i 不是索引， $i := \sqrt{-1}$ ，

复数可以表示为极坐标形式 $r \cdot e^{i\theta}$ ，其中 r 为模长， $r = \sqrt{a^2 + b^2}$ ， θ 是向量的旋转角， $\theta = \arctan\left(\frac{b}{a}\right)$ 。



向量的旋转可以通过旋转矩阵实现。

向量 (a, b) 逆时针旋转 ϕ ，通过极坐标表示为 $r \cdot e^{i \cdot (\theta + \phi)}$

$$r \cdot e^{i \cdot (\theta + \phi)} = r \cdot e^{i \cdot \theta} \cdot 1 \cdot e^{i \cdot \phi}$$

将极坐标表示转换为复数，则上式

$$= (a + bi)(\cos\phi + i \cdot \sin\phi)$$

$$= a \cdot \cos\phi + i \cdot a \cdot \sin\phi + i \cdot b \cdot \cos\phi + i^2 b \cdot \sin\phi$$

$$= a \cdot \cos\phi - b \cdot \sin\phi + i \cdot (a \cdot \sin\phi + b \cdot \cos\phi)$$

此处，再将复数表示转换为二维向量

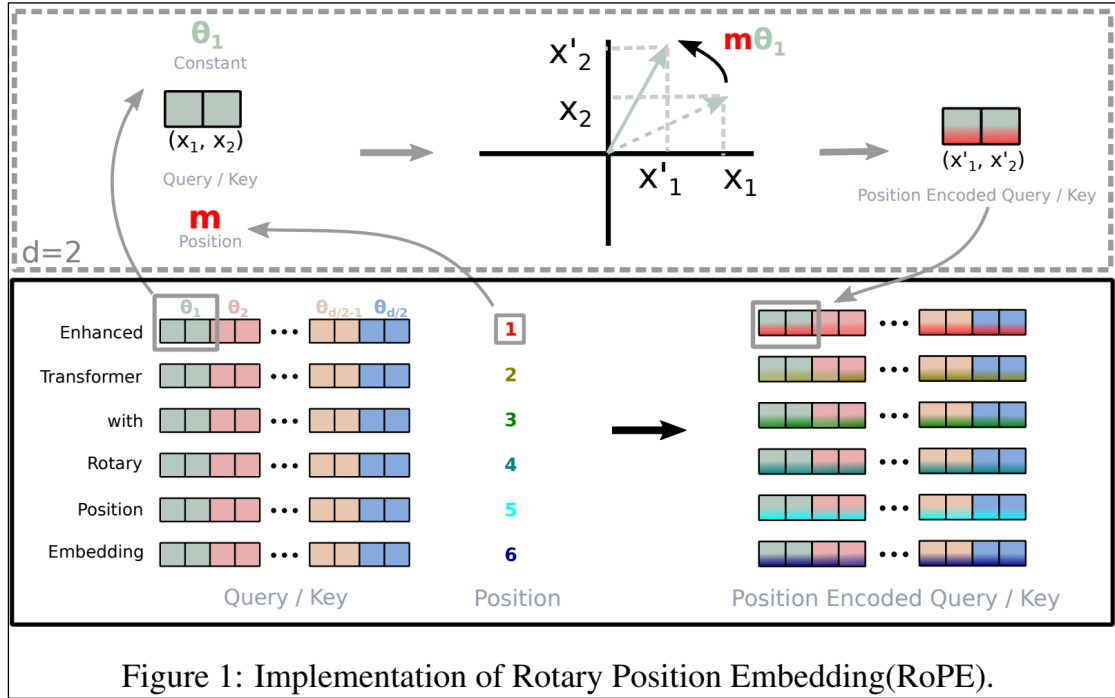
$$\begin{pmatrix} a \cdot \cos\phi - b \cdot \sin\phi \\ a \cdot \sin\phi + b \cdot \cos\phi \end{pmatrix} = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix}$$

由此可见，向量 (a, b) 旋转 ϕ 可以通过乘以一个 2×2 的矩阵实现。这个矩阵被称为旋转矩阵 R_θ 。

旋转矩阵可以在不改变原向量模长的情况下改变其角度。

5.5.3 RoPE 原理

RoPE 的核心思想是不修改词嵌入本身，而是通过一个旋转变换来修改计算查询向量 q 和键向量 k 的方式，将相对位置信息直接编码到注意力分数的计算过程中。



1) 将输入向量分为 $\frac{d}{2}$ 个向量

通常，向量维度 d 是偶数，第一步将向量视为 $\frac{d}{2}$ 个二维向量的堆叠。

$$(e_1, e_2, \dots, e_d) = \text{concat}((e_1, e_2), (e_3, e_4), \dots, (e_{d-1}, e_d))$$

2) 将每个向量旋转不同角度

$$\begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{\frac{d}{2}} & -\sin m\theta_{\frac{d}{2}} \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_{\frac{d}{2}} & \cos m\theta_{\frac{d}{2}} \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ \vdots \\ e_{d-1} \\ e_d \end{pmatrix}$$

左侧的矩阵记为 R_m ，表示位置 m 的旋转矩阵。

(1) θ_i 是定值，称为**旋转基数**，由 i 决定。

(2) m 表示**绝对位置编号**（如 $\{1, 2, \dots\}$ ）

(3) 对于特定的 θ_i 和 m ，**旋转角度为 $m\theta_i$** 。

应用旋转矩阵的结果就是拆分后的 $\frac{d}{2}$ 组向量各自旋转 $m\theta_i$ ，其中 $i \in [1, \frac{d}{2}]$ 。

3) θ 的取值

沿用正余弦编码的方案， $\theta_i = 10000^{-2i/d}$ 。

其中，10000 称为基础频率（Base Frequency），因为它可以控制正余弦函数的频率。

4) 在注意力计算中应用旋转

查询向量： $q_m = R_m W_q x_m$ （ x_m 是位置 m 的词嵌入）

键向量： $k_n = R_n W_k x_n$ （ x_n 是位置 n 的词嵌入）

5) 注意力分数体现相对位置

计算 m 位置查询对 n 位置键的注意力分数： $(q_m)^T * k_n = (R_m W_q x_m)^T * (R_n W_k x_n)$

根据旋转矩阵的性质 $R_m^T R_n = R_{n-m}$ （ $R_\theta^T = R_{-\theta}$ ， $R_{\theta_1} R_{\theta_2} = R_{\theta_1 + \theta_2}$ ），上式可以化为： $(W_q x_m)^T R_{n-m}^T (W_k x_n)$ （这里 R_{n-m} 是相对位置 $(n - m)$ 的旋转矩阵）

因此，注意力分数 $(q_m)^T * k_n$ 只依赖于：

(1) 原始的词嵌入内容 x_m 和 x_n

(2) 投影矩阵 W_q, W_k

(3) 相对位置 $(n - m)$ （通过旋转矩阵 R_{n-m} 体现）。

5.5.4 RoPE 如何改进原版的缺点

1) 避免信息稀释，显式编码相对位置

这是最核心的改进，RoPE 不改变词嵌入向量，而是通过旋转操作直接修改 q 和 k 的计算过程。

注意力分数 $q_m^T k_n$ 天然地、显式地包含了词向量投影 $(W_q x_m)$ 和 $(W_k x_n)$ 之间的相对位置差 $(n - m)$ （通过 R_{n-m} ）。模型不再需要费力地从混合项中学习和解耦位置信息。

位置信息以一种几何旋转的方式（保持向量模长不变）融入 q 和 k ，避免了与词义信息的直接相加和后续投影带来的稀释。

2) 提升表达能力

旋转操作是一种线性变换，并且保持了向量的模长，这被认为是一种更“自然”和数学上更优雅的融入位置信息的方式。

显式地建模相对位置差 $(n - m)$ ，使得模型更容易精确捕捉不同距离的词之间的关系。

3) 显著改善长度外推性

虽然 RoPE 在训练时也定义了最大位置（如 2048），但其外推性优于原版 PE。

RoPE 的泛化依赖于旋转操作的连续性和周期性。

(1) 连续性：旋转角度是连续变化的。即使遇到训练时未见过的较大位置 m ，旋转操作 R_m 在数学上仍然是定义良好的。

无论 m 多大（即使是训练时从未见过的巨大值，如 $m = 1000000$ ），旋转操作 R_m 在数学上都是严格定义的。 R_m 仅仅是一个角度为 $m\theta$ 的旋转矩阵。

这保证了对于任意位置 m （无论是否在训练范围内），我们都能计算出一个有效的、符合几何意义的旋转表示。模型至少有一个“合理”的数学对象可以处理。

相比之下正余弦位置编码：当 pos 远大于训练长度时， $PE(pos)$ 的值可能落入函数行为完全不同的区域（如低频维度从准线性突变为高频振荡），其“合理性”或“可解释性”大大降低。RoPE 的旋转操作则始终是几何上明确定义的旋转。

(2) 周期性：旋转 2π 弧度（360 度）后回到原点。模型在训练时已经学会了如何处理这种周期性（处理角度 $\theta \bmod 2\pi$ ）。当遇到长序列时，虽然绝对位置 m 很大，但 $m\theta \bmod 2\pi$ 的值域仍在 $[0, 2\pi)$ 内，模型在训练时见过这个范围内的各种角度组合。模型需要学习的是如何利用这种周期性来表示更长的依赖。

相比之下，原版 PE 在 pos 很大时，其函数值 $\sin(pos / \dots), \cos(pos / \dots)$ 可能落在训练时完全未覆盖的、函数特性（如高频振荡）不同的区域。

注意：RoPE 的外推性并非完美无缺，但普遍认为比原版 PE 好很多。也有后续工作进一步优化 RoPE 的外推能力。

5.5.5 实践中如何实现 RoPE

由于 R_m 的稀疏性，直接用矩阵乘法会浪费算力，通常使用一下方式实现。

$$\begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \vdots \\ q_{d/2-2} \\ q_{d/2-1} \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_0 \\ \cos m\theta_0 \\ \cos m\theta_1 \\ \cos m\theta_1 \\ \vdots \\ \cos m\theta_{d/2-1} \\ \cos m\theta_{d/2-1} \end{pmatrix} + \begin{pmatrix} -q_1 \\ q_0 \\ -q_3 \\ q_2 \\ \vdots \\ -q_{d/2-1} \\ q_{d/2-2} \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_0 \\ \sin m\theta_0 \\ \sin m\theta_1 \\ \sin m\theta_1 \\ \vdots \\ \sin m\theta_{d/2-1} \\ \sin m\theta_{d/2-1} \end{pmatrix}$$

参考苏剑林的[博客](#)。

5.6 温度超参数

5.6.1 数学原理

假设模型最终的线性层输出（Logits）为 $\vec{h} = \{h_1, h_2, \dots, h_n\}$ ，接下来通过 softmax 将 Logits 转换为概率分布 $o_i = \frac{e^{h_i}}{\sum_{j=1}^n e^{h_j}}$ 。

温度超参数 t 可用于调整输出的概率分布，具体来说，输出调整为 $o_i = \frac{e^{h_i/t}}{\sum_{j=1}^n e^{h_j/t}}$ ，即每个 logit 除以 t 再执行 softmax 归一化。当 $t \rightarrow +\infty$ 时， $\frac{h_i}{t} \rightarrow 0$ ， $e^{h_i/t} \rightarrow 1$ ，此时所有 token 的概率无限接近，不确定性极高。当 $t \rightarrow 0^+$ 时， $\frac{h_i}{t} \rightarrow \pm\infty$ ，差异被放大，原先 logits 更大的元素趋近于 1，而更小的元素趋近于 0，预测的下一个 token 确定性更大。

5.6.2 总结

- （1）温度越高，输出的 token 不确定性越大，类比丹炉的温度越高，反应越剧烈，产物越随机。
- （2）温度越低，输出的 token 不确定性越小，类比丹炉的温度越低，反应越缓慢，产物更加确定。

5.7 预热和退火

预热和退火都属于学习率调度策略。它们的目标都是在模型训练的不同阶段，动态调整学习率（Learning Rate，LR）这个超参数，以优化训练过程，提升模型最终性能。

5.7.1 预热（Warmup）

1) 定义

预热是指在训练初期（通常是前几个 epoch 或前几万个/几十万个 step），将学习率从一个非常小的值（甚至 0）逐步线性地（或其他方式）增加到预设的初始学习率（或峰值学习率）的过程。

2) 作用

（1）稳定训练初期

模型参数在初始化时通常是随机的（如 Xavier，He）。一开始就使用较大的学习率可能导致梯度更新过大、过猛，造成参数剧烈震荡，甚至导致训练不稳定（梯度爆炸或数值溢出）或陷入糟糕的局部极小点。预热让模型“温和地”开始学习，先用小步探索数据分布和梯度方向。

（2）适应优化器状态

对于像 Adam 这样带有动量（一阶矩估计 m ）和自适应学习率（二阶矩估计 v ）的优化器，其内部状态（ m, v ）在初始阶段是零或有偏估计（偏向 0）。预热期给了优化器时间积累更准确的历史梯度信息（更新 m 和 v ），避免初始几步基于不可靠的 m/v 做出过大的更新。

（3）避免早期过拟合/崩溃

在大模型训练初期，模型容量巨大但尚未学到有效表示，过大的学习率可能导致它过早地拟合训练数据中的噪声或特定模式，甚至导致训练崩溃。预热降低了这种风险。

（4）为后续训练奠定基础

提供一个更平滑、更稳定的起点，让模型在进入主要训练阶段时处于一个更好的状态。

5.7.2 退火（Annealing）

1) 定义

退火是指在训练的中后期，当模型学习逐渐收敛时，将学习率从预设的峰值（或初始值）按照某种预定的策略（如阶梯式下降、线性下降、余弦衰减）逐步降低到接近 0 的过程。

2) 作用

（1）促进精细收敛

随着训练的进行，模型参数逐渐接近损失函数的某个极小值区域。此时如果学习率仍然很大，参数更新步长过大，可能会在最优解附近反复震荡，甚至跳出最优解区域，无法真正收敛到精细的极小值点。退火通过逐步减小学习率，使模型能够进行更小、更精细的参数调整，最终稳定在一个（希望是更好的）局部或全局极小值点附近。

（2）跳出局部极小值点

虽然主要目的是精细收敛，但在某些策略（如带重启的余弦退火）中，周期性地将学习率重置到一个相对较高的值，可以帮助模型跳出当前可能陷入的次优局部极小值点，探索损失曲面上其他可能的更优区域。

（3）提高泛化能力

理论（如随机梯度下降的收敛分析）和大量实践表明，在训练后期降低学习率通常有助于模型找到更平坦的极小值点。平坦极小值点对参数扰动不敏感，被认为具有更好的泛化能力（在未见过的数据上表现更好）。

（4）稳定训练末期

避免在接近收敛时因学习率过大而产生不必要的波动。

5.7.3 协同使用

预热和退火通常是协同使用的：

（1）预热开局

训练开始，学习率从 0 逐步增加到峰值学习率（例如 $1e-4$ 或 $5e-5$ ），确保训练稳定启动。

（2）稳定训练

在预热结束后的一段时间内（有时会保持恒定），使用峰值学习率进行主要训练，让模型快速学习。

（3）退火收尾

在训练后半程，按照预定的退火策略（如余弦衰减），逐步降低学习率，使模型参数精细收敛到最优解附近。

5.7.4 为什么对大模型尤其重要

（1）训练成本极高

大模型训练一次动辄耗费数十万、数百万美元的计算资源和数周、数月时间。任何训练失败（如早期崩溃）或次优收敛（导致最终性能差）的代价都极其巨大。预热和退火是提高训练成功率和最终模型质量的关键保障。

（2）模型复杂度极高

数十亿、数百亿参数的模型，其损失函数曲面极其复杂，存在大量局部极小点、鞍点。精细的学习率调度（预热+退火）是引导模型穿越这片复杂地形、找到良好解的必要导航工具。

（3）优化器依赖

现代大模型几乎都使用 Adam 或 AdamW 等自适应优化器，这些优化器对初始状态非常敏感，预热对它们至关重要。

总而言之，预热和退火是大模型训练中不可或缺的学习率调度策略。预热为高风险的训练初期保驾护航，确保稳定起步；退火则在训练后期引导模型精细收敛，追求更优的性能和泛化能力。两者共同作用，极大地提升了大规模深度学习模型训练的效率和最终效果。

5.8 稠密模型和混合专家模型

“稠密模型”和“混合专家模型”是当前大型语言模型架构中两种重要的范式。它们代表了不同的设计理念，在模型容量、计算效率和训练难度上各有优劣。

5.8.1 稠密模型

在稠密模型（Dense Model）中，每个输入样本都会激活（用到）模型的所有参数。

5.8.2 混合专家模型

混合专家模型（Mixture of Experts Model, MOE Model）的核心思想是将模型的不同部分设计成“专家”，并引入一个门控网络。对于每一个输入样本（词元），门控网络会动态选择一小部分最相关的“专家”来处理该输入，而其他专家保持非活跃状态。这是一种稀疏激活的机制。

5.9 大模型的缩放定律

5.9.1 定义

在大模型领域，缩放定律（Scaling Laws）的本质是揭示模型性能与资源投入之间的定量关系，它通过数学规律证明：系统性地扩大模型规模（N）、数据规模（D）和计算量（C），可持续提升模型能力（如降低损失、提高准确率）。其核心是建立“资源-性能”的幂律函数关系，为大模型研发提供可预测的科学框架。

注意：

（1）模型性能还会受到**训练批次**和**训练步数**的影响，但主要的影响因素是 N、D、C，此处不讨论其它因素。

（2）许多研究团队在各自的研究环境下做了缩放定律的研究，由于硬件条件、数据集、模型架构等的不同，得出的规律也各不相同。

本节以 OpenAI 团队的 Kaplan 等人在 2020 年发表的论文《Scaling Laws for Neural Language Models》中所做的研究对缩放定律进行介绍。该论文用交叉熵损失 L 衡量模型性能。

5.9.2 数学表示

1) 模型规模

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}$$

（1）在数据规模和计算量无限的前提下，性能随模型规模的增长按幂率提升。

此处的**无限**是指数据规模和计算量**不会成为性能瓶颈**。不会因为数据太少模型太大而过拟合，也不会因为计算资源不够而无法训练。

(2) N 表示非嵌入参数量（不考虑嵌入层参数量，那么参数规模随 Transformer Block 层数线性增长，更容易揭示模型性能随参数规模的变化规律）。

(3) N_c 是模型规模相关的常数。

(4) α_N 是模型规模相关的缩放指数。

2) 数据规模

$$L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}$$

(1) 在模型规模和计算量不做限制的前提下，性能随数据规模的增长按幂率提升。

模型规模不限制是指模型要有足够的容量从海量的数据中提炼知识，不会因为模型过小数据过多而欠拟合。

(2) D 表示数据规模，即 token 的数量。

(3) D_c 是数据规模相关的常数。

(4) α_D 是数据规模相关的缩放指数。

3) 计算量

$$L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}$$

(1) 在模型规模和数据规模不受限制的前提下，性能随计算量的增长按幂率提升。

数据够多，模型够大。

(2) C 是前向传播和反向传播的总计算量，不考虑嵌入和偏置项所需的计算（可以忽略）。

(3) C_c 是计算量相关的常数。

(4) α_C 是计算量相关的缩放指数。

4) 数据规模和参数规模的共同作用

寻找特定参数规模下最优的数据规模或反过来，对于最大化模型性能，避免过拟合具有重要意义。

$$L(N, D) = \left[\left(\frac{N_c}{N}\right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$$

其中， N_c 和 D_c 是与模型架构相关的常数。

由上述公式可知，损失同时受到参数规模和数据规模的影响。

(1) 当 $N \rightarrow \infty$ 时， $L(N, D)$ 等价于 $L(D)$ ，即 $L(\infty, D) = L(D)$

(2) 当 $D \rightarrow \infty$ 时， $L(N, D)$ 等价于 $L(N)$ ，即 $L(N, \infty) = L(N)$

即单独增大 N 或 D ，模型性能并不能无限提升，即存在过拟合现象。

过拟合系数定义为

$$\delta L(N, D) \equiv \frac{L(N, D)}{L(N, \infty)} - 1$$

$$\delta L(N, D) \equiv \left[1 + \frac{D_c}{D} \left(\frac{N}{N_c} \right)^{\frac{\alpha_N}{\alpha_D}} \right] - 1$$

当数据集充足时，过拟合系数为 0，要将过拟合系数控制在一定范围内，数据量存在下限，在原文的研究环境下， $D \gtrsim (5 \times 10^3) N^{0.74}$ ，即 $D \propto N^{0.74}$ 。

总结

(1) 只要按比例同步扩展 N 和 D ，性能就会按预期提升，模型扩大 8 倍时，数据量扩大为原先的 5 倍即可。

(2) 如果单独增大 N 和 D ，就会进入收益递减阶段。

5) 根据参数规模和数据规模估算计算量

不考虑嵌入层和偏置项所需的计算，单个 token 前向传播的计算量约为

$$C_{\text{forward}} = 2N + 2n_{\text{layer}}n_{\text{ctx}}d_{\text{model}}$$

其中，

- (1) C_{forward} 表示前向传播的计算量
- (2) N 表示模型的总参数量
- (3) n_{layer} 表示模型的层数
- (4) n_{ctx} 表示模型的上下文长度
- (5) d_{model} 表示模型维度，即单个 token 嵌入向量的长度

当 $d_{\text{model}} > \frac{n_{\text{ctx}}}{12}$ 时， $2n_{\text{layer}}n_{\text{ctx}}d_{\text{model}}$ 占比较小。

当 $d_{\text{model}} \gg \frac{n_{\text{ctx}}}{12}$ 时，第二项可以忽略，此时的 $C_{\text{forward}} \approx 2N$

考虑到反向传播的计算量约为前向传播的两倍，单次训练（单个 token 前向传播+反向传播）计算量如下

$$C \approx 6N$$

上式乘以数据量即计算总量，如下：

$$C \approx 6ND$$

5.9.3 核心结论

1) 性能无饱和提升

只要持续增加 N, D, C ，模型性能几乎无限提升。

2) 计算最优边界

计算量不可能无限供给，需要平衡模型规模与数据规模，以在固定的预算下获得最强的模型。即在 C 固定的情况下，根据以下两个公式，计算最优的数据量和模型规模

$$\delta L(N, D) \equiv \left[1 + \frac{D_c}{D} \left(\frac{N}{N_c} \right)^{\frac{\alpha_N}{\alpha_D}} \right] - 1$$

$$C \approx 6ND$$

固定 C ，调整 N 和 D ，以最小化 $\delta L(N, D)$ ，避免过拟合。此时可以获得 L 最小、性能最优的模型。

3) 数据与模型的等效性

数据不足时，盲目增大模型规模反而损害性能。

高质量数据可等效为更多 token， D 应理解为有效数据量，数据质量越高，有效数据量越大。

4) 架构无关性

缩放定律适用于 Transformer、MOE 等多种架构。